

Training dynamics in overparameterized neural networks

Training dynamics in overparameterized neural networks

Student Meeting Presentation

Training dynamics in overparameterized neural networks

Student Meeting Presentation

Shreyas Kalvankar
MSc Computer Science

The supervised learning framework

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

How: find f^*

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

How: find f^* that minimizes the population risk

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

How: find f^* that minimizes the population risk

$$\mathcal{L}_\rho(f) := \mathbb{E}_\rho[(f(X) - Y)^2]$$

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

How: find f^* that minimizes the population risk

$$\mathcal{L}_\rho(f) := \mathbb{E}_\rho[(f(X) - Y)^2]$$

Problem: ρ is unknown!

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

How: find f^* that minimizes the population risk

$$\mathcal{L}_\rho(f) := \mathbb{E}_\rho[(f(X) - Y)^2]$$

Problem: ρ is unknown!

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

How: find f^* that minimizes the population risk

$$\mathcal{L}_\rho(f) := \mathbb{E}_\rho[(f(X) - Y)^2]$$

Solution:
$$\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2$$

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

How: find f^* that minimizes the population risk

$$\mathcal{L}_\rho(f) := \mathbb{E}_\rho[(f(X) - Y)^2]$$

Solution: $\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2$ **Minimize the empirical risk**

The supervised learning framework

Data: n observations $(x_i, y_i) \in \mathbb{R}^d \times \mathbb{R}$, $i = 1, \dots, n$, i.i.d. $\sim \rho$

Prediction function: $f(x) \in \mathbb{R}$

Aim: given a new x , predict its label y (generalize)

How: find f^* that minimizes the population risk

$$\mathcal{L}_\rho(f) := \mathbb{E}_\rho[(f(X) - Y)^2]$$

Solution: $\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2$ **Minimize the empirical risk**

Introduction & Problem setup

In order to minimize the empirical risk we consider a family of parameterized functions

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}$$

Introduction & Problem setup

In order to minimize the empirical risk we consider a family of parameterized functions

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}$$

Linear Networks: $f_{\theta}(x) = x^{\top} \theta$

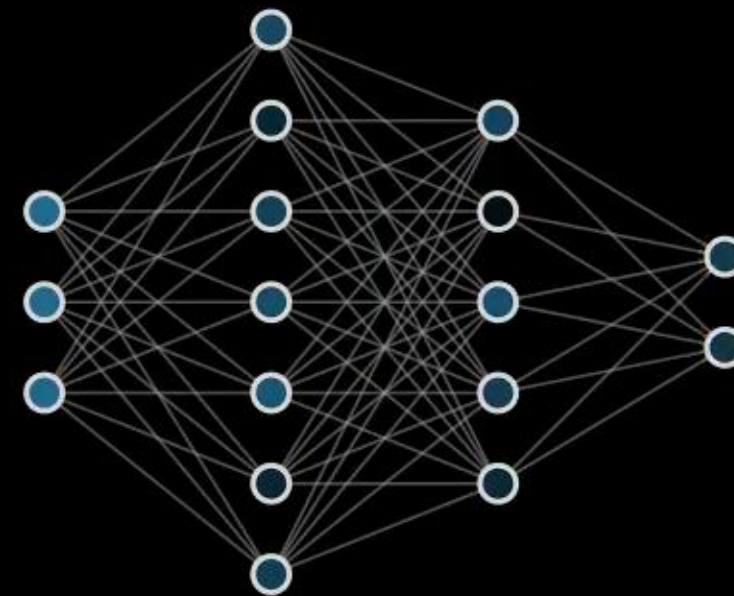
Introduction & Problem setup

In order to minimize the empirical risk we consider a family of parameterized functions

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}$$

Linear Networks: $f_{\theta}(x) = x^{\top} \theta$

Neural Networks: $f_{\theta}(x) =$



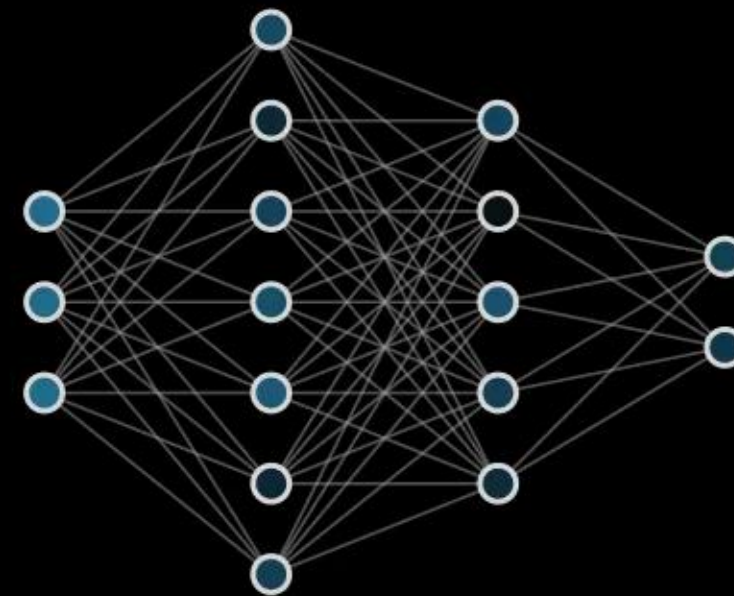
Introduction & Problem setup

In order to minimize the empirical risk we consider a family of parameterized functions

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}$$

Linear Networks: $f_{\theta}(x) = x^{\top} \theta$

Neural Networks: $f_{\theta}(x) =$



**Expressivity
power**

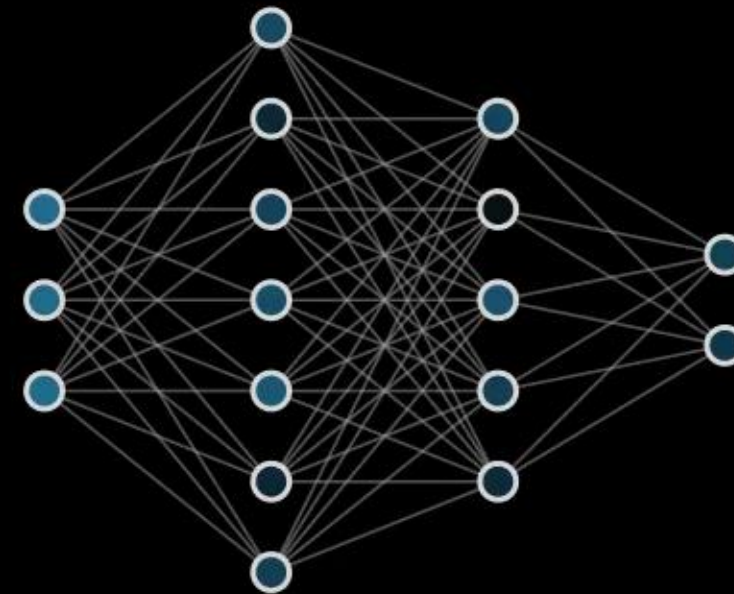
Introduction & Problem setup

In order to minimize the empirical risk we consider a family of parameterized functions

$$f_{\theta} : \mathbb{R}^d \rightarrow \mathbb{R}$$

Linear Networks: $f_{\theta}(x) = x^{\top} \theta$

Neural Networks: $f_{\theta}(x) =$



**Expressivity
power**

**Simplicity
of
analysis**

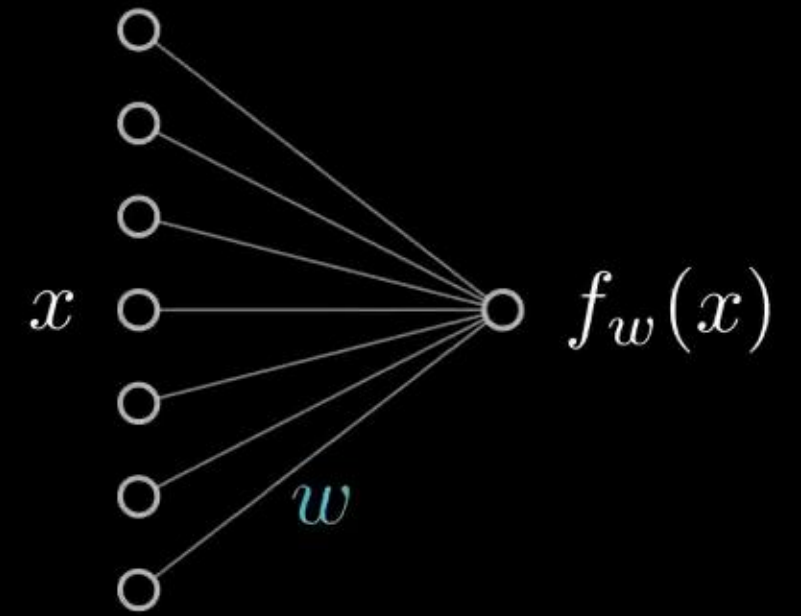
Linear models and gradient descent

Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

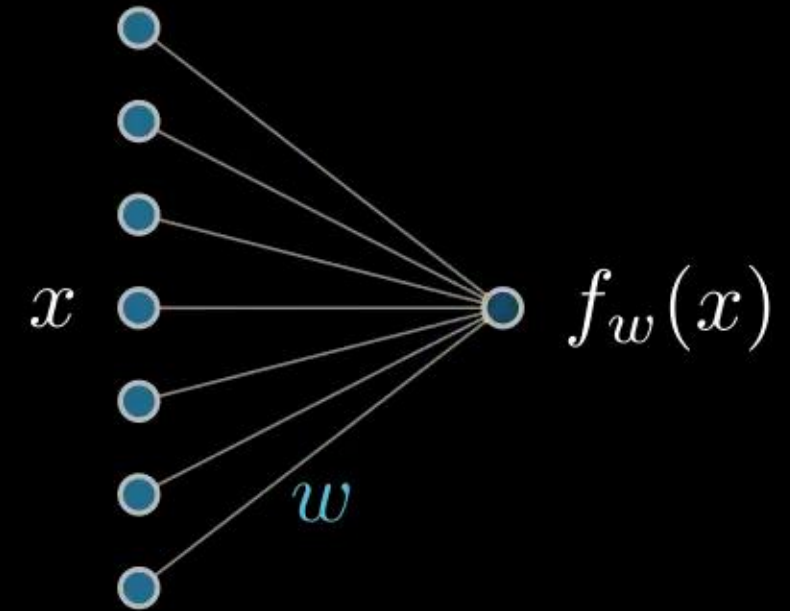
Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$



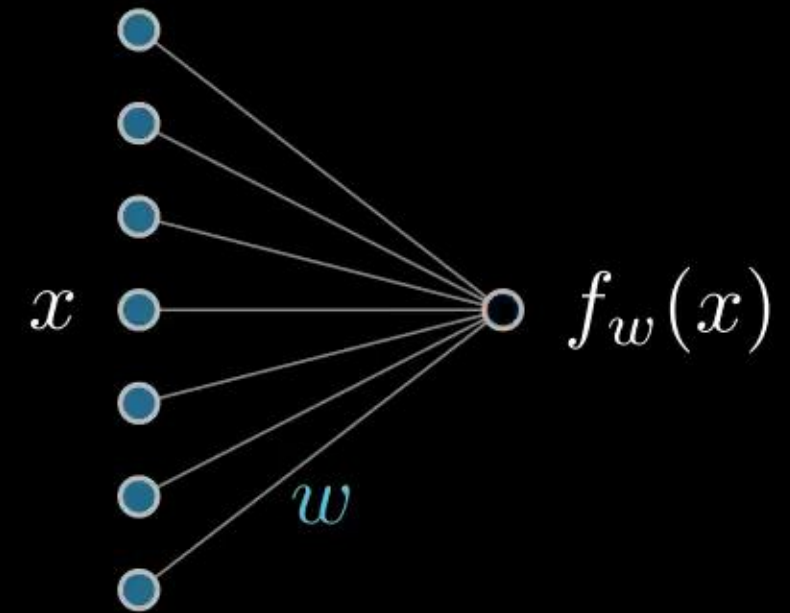
Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$



Linear models and gradient descent

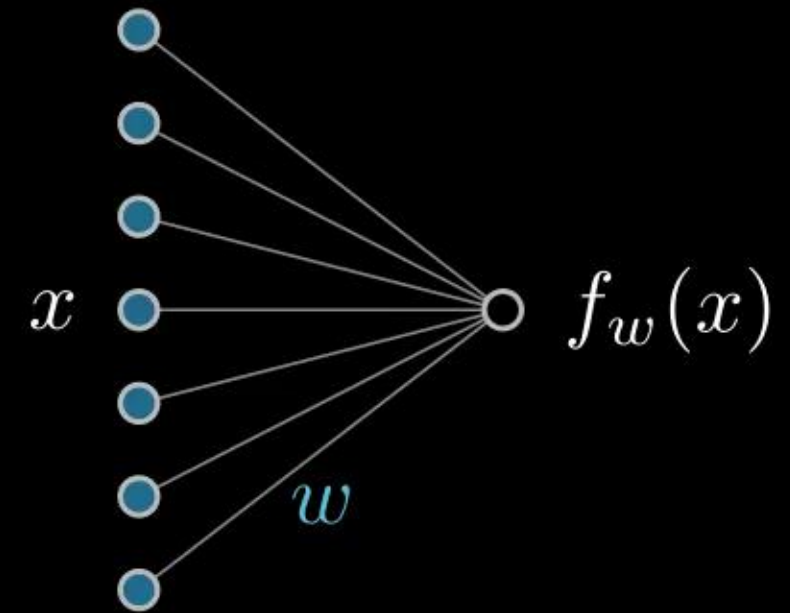
Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

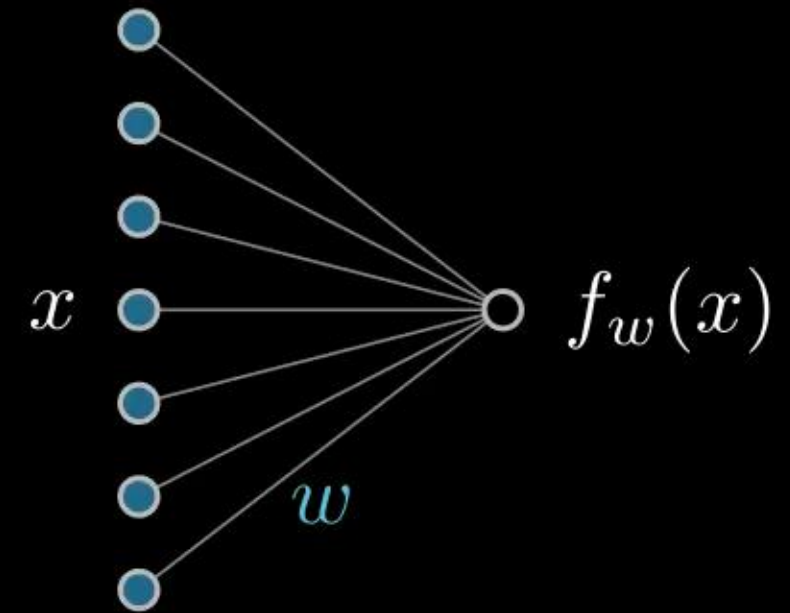


Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:



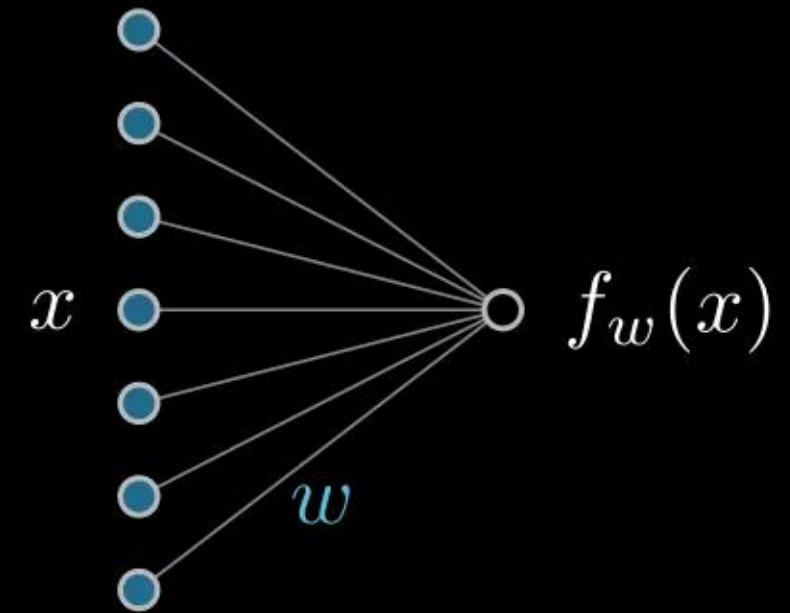
Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$



Linear models and gradient descent

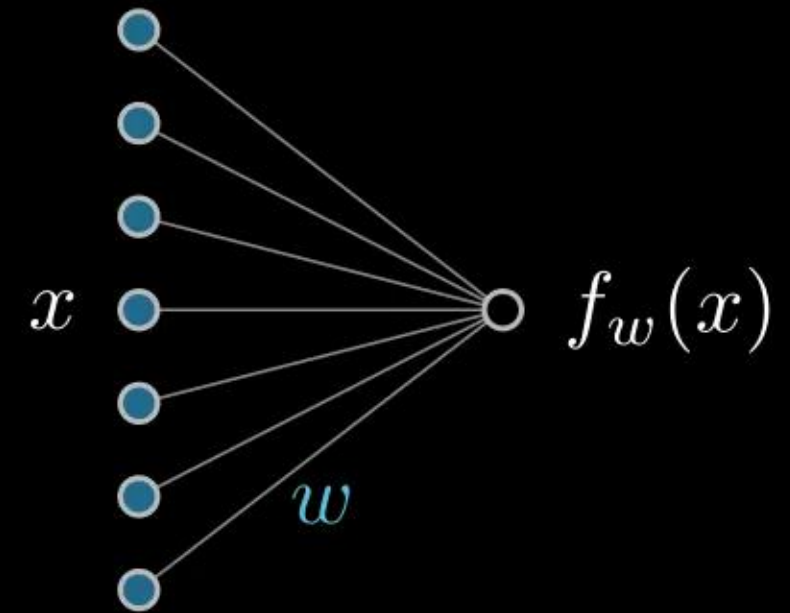
Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

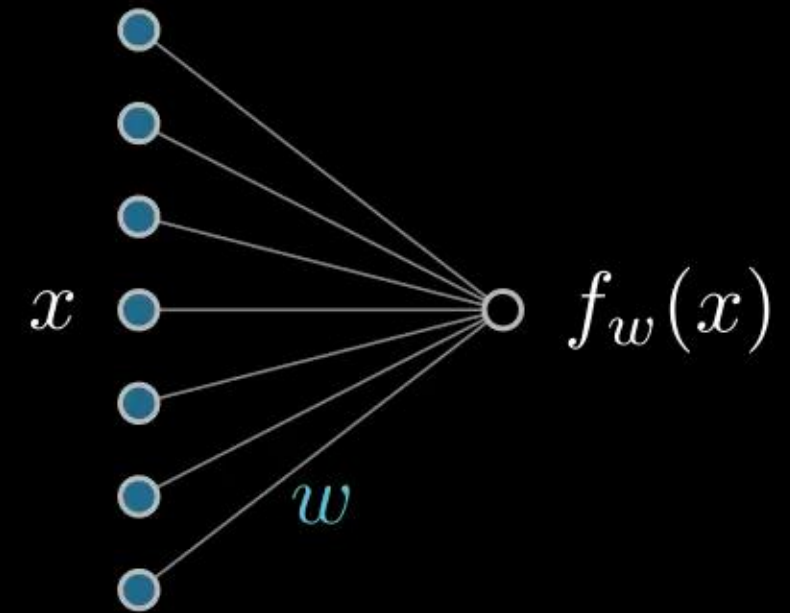
$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$

$$\mathcal{L} \text{ is convex} \Rightarrow \nabla_w \mathcal{L}(w^*) = 0$$

$$XX^\top w^* = Xy$$

(unique if $XX^\top \succ 0$)



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

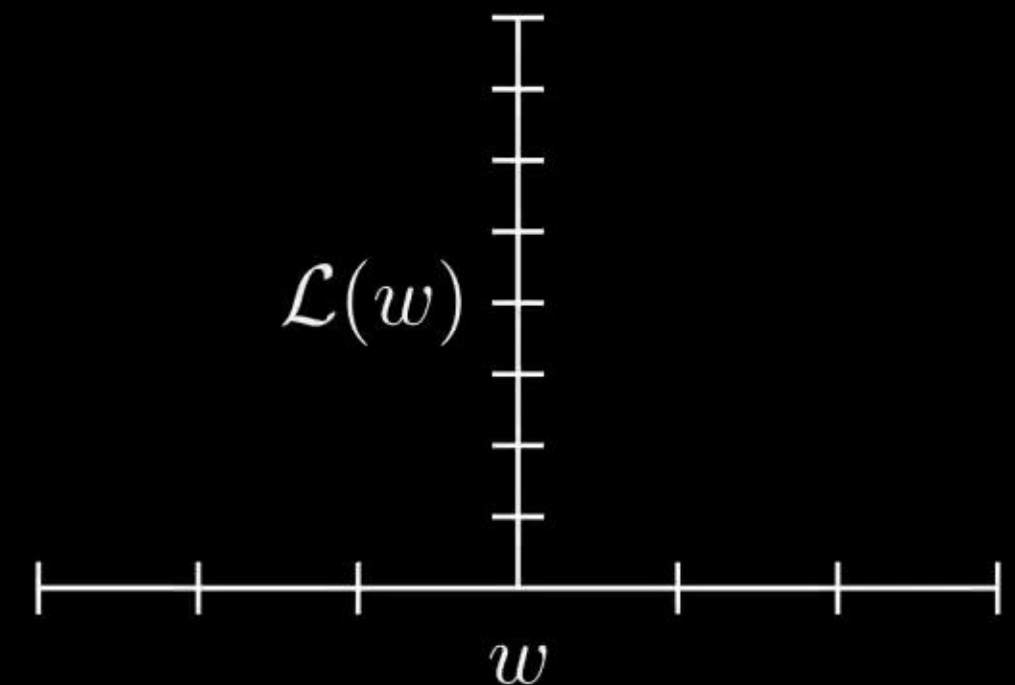
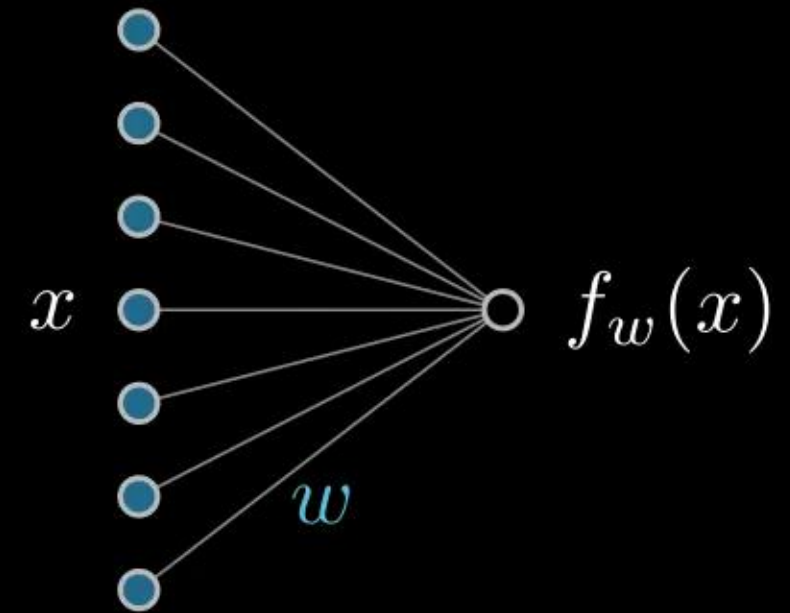
$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$

$$\mathcal{L} \text{ is convex} \Rightarrow \nabla_w \mathcal{L}(w^*) = 0$$

$$XX^\top w^* = Xy$$

(unique if $XX^\top \succ 0$)



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

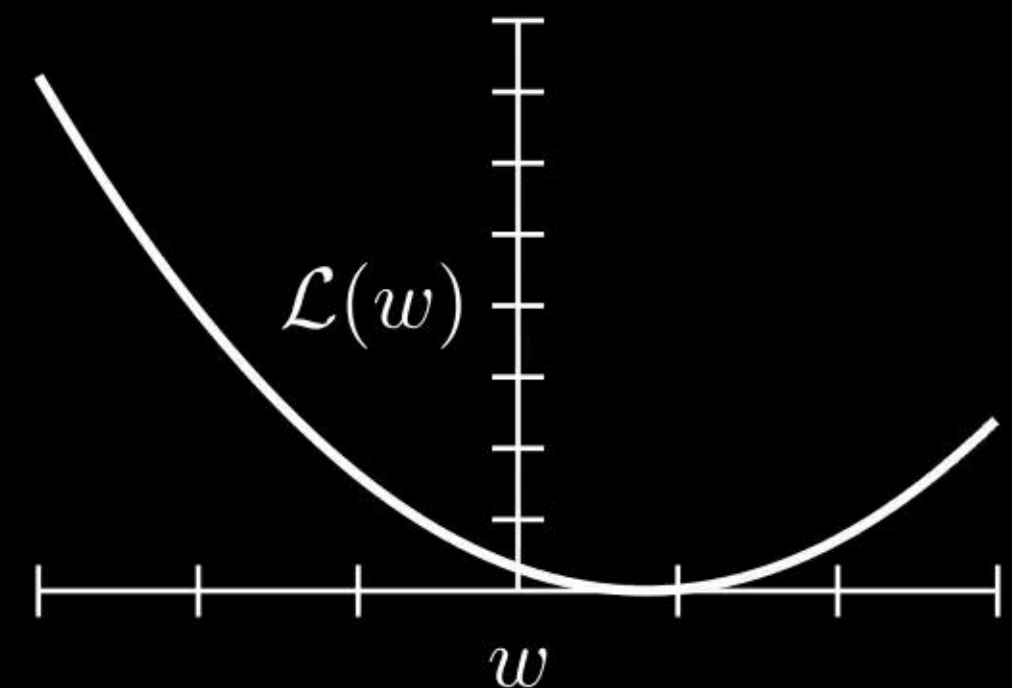
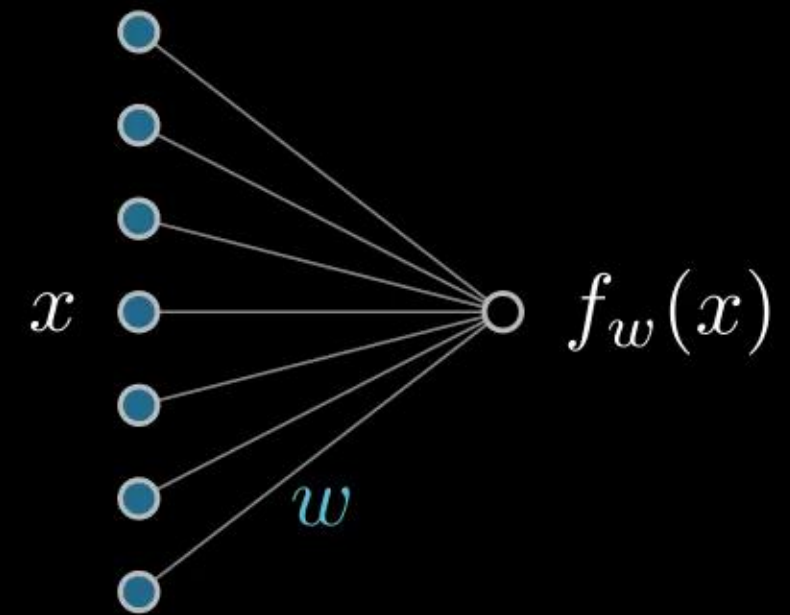
$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$

$$\mathcal{L} \text{ is convex} \Rightarrow \nabla_w \mathcal{L}(w^*) = 0$$

$$XX^\top w^* = Xy$$

(unique if $XX^\top \succ 0$)



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

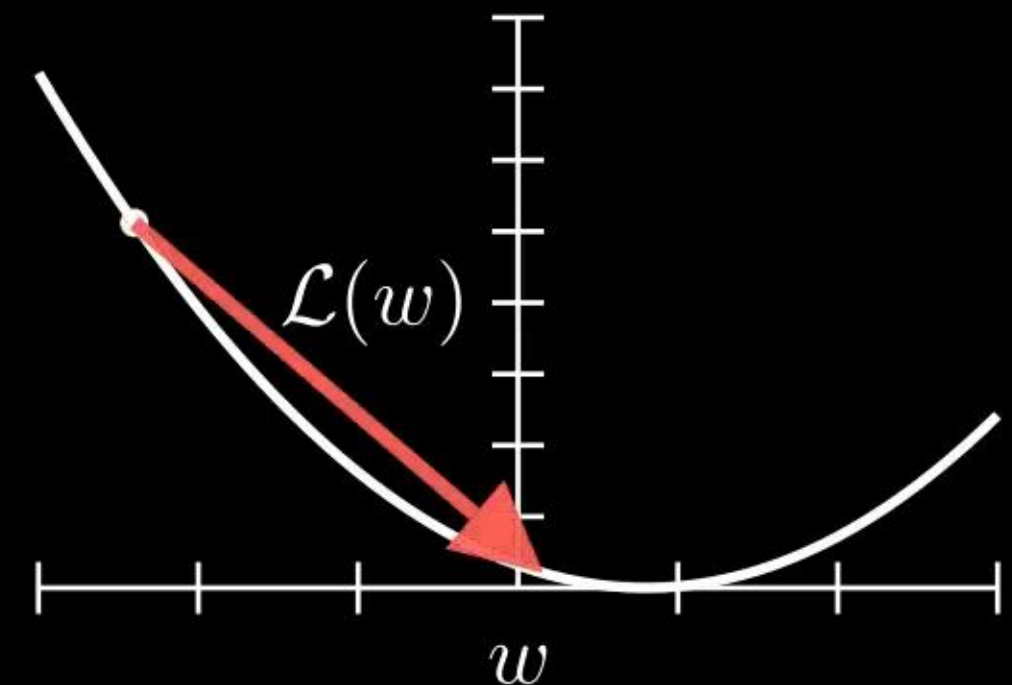
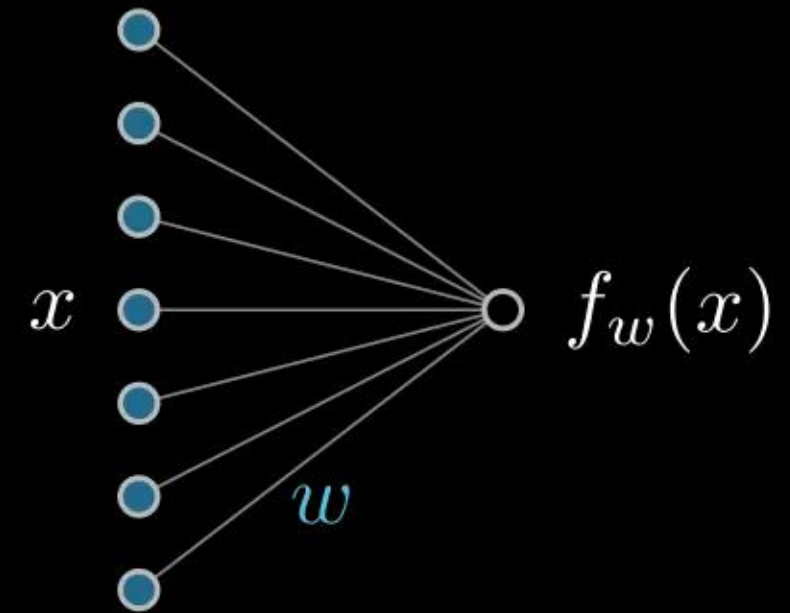
$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$

$$\mathcal{L} \text{ is convex} \Rightarrow \nabla_w \mathcal{L}(w^*) = 0$$

$$XX^\top w^* = Xy$$

(unique if $XX^\top \succ 0$)



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

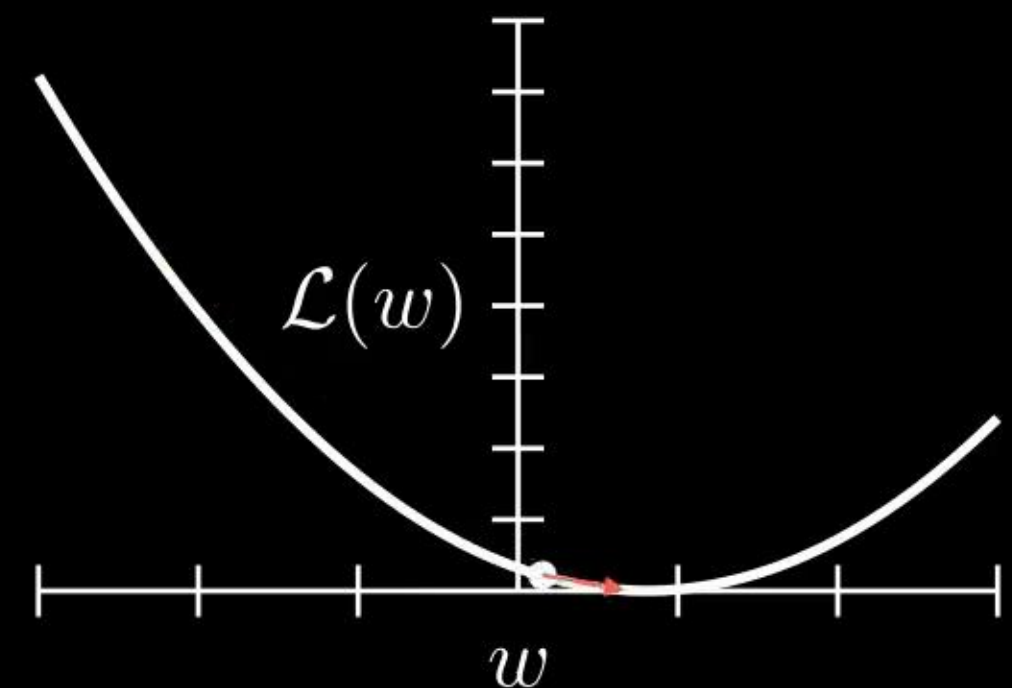
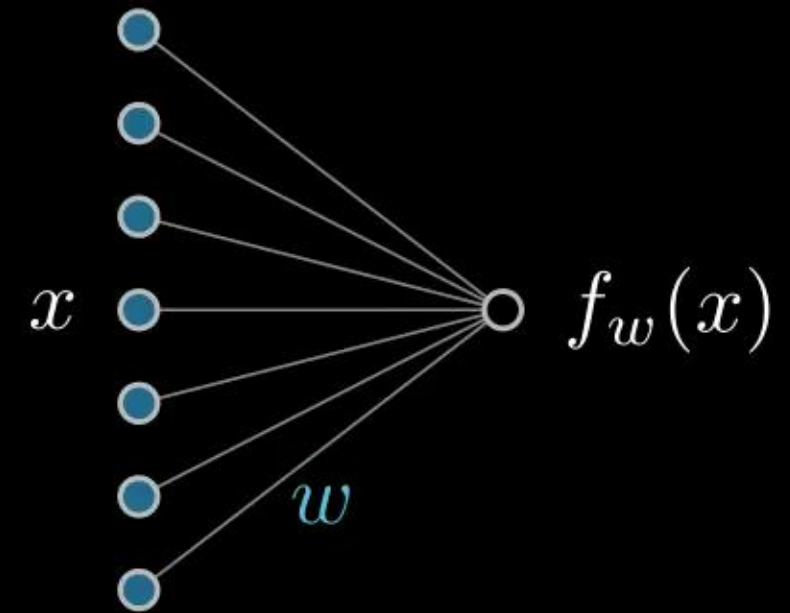
$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$

$$\mathcal{L} \text{ is convex} \Rightarrow \nabla_w \mathcal{L}(w^*) = 0$$

$$XX^\top w^* = Xy$$

(unique if $XX^\top \succ 0$)



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

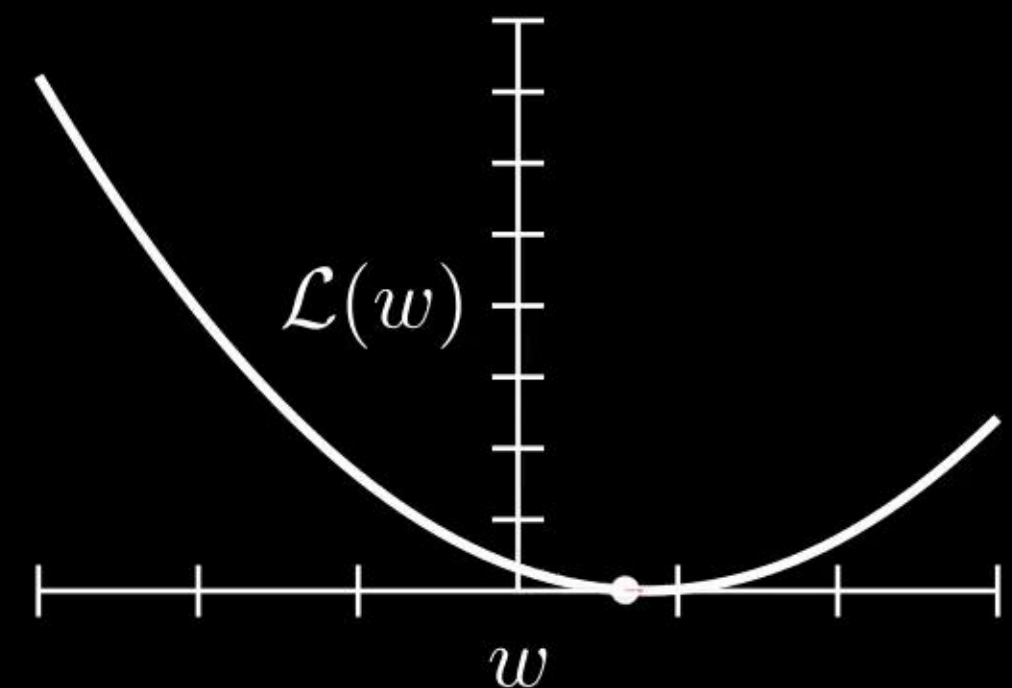
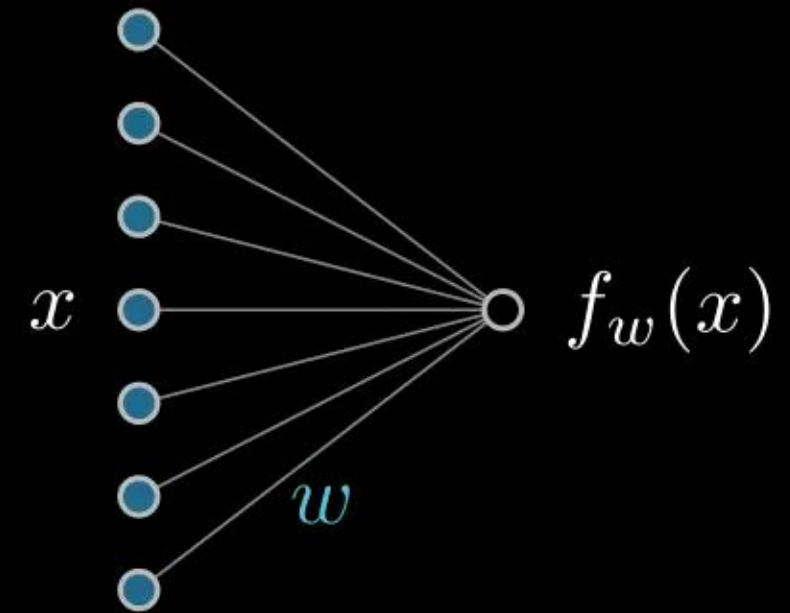
$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$

$$\mathcal{L} \text{ is convex} \Rightarrow \nabla_w \mathcal{L}(w^*) = 0$$

$$XX^\top w^* = Xy$$

(unique if $XX^\top \succ 0$)



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

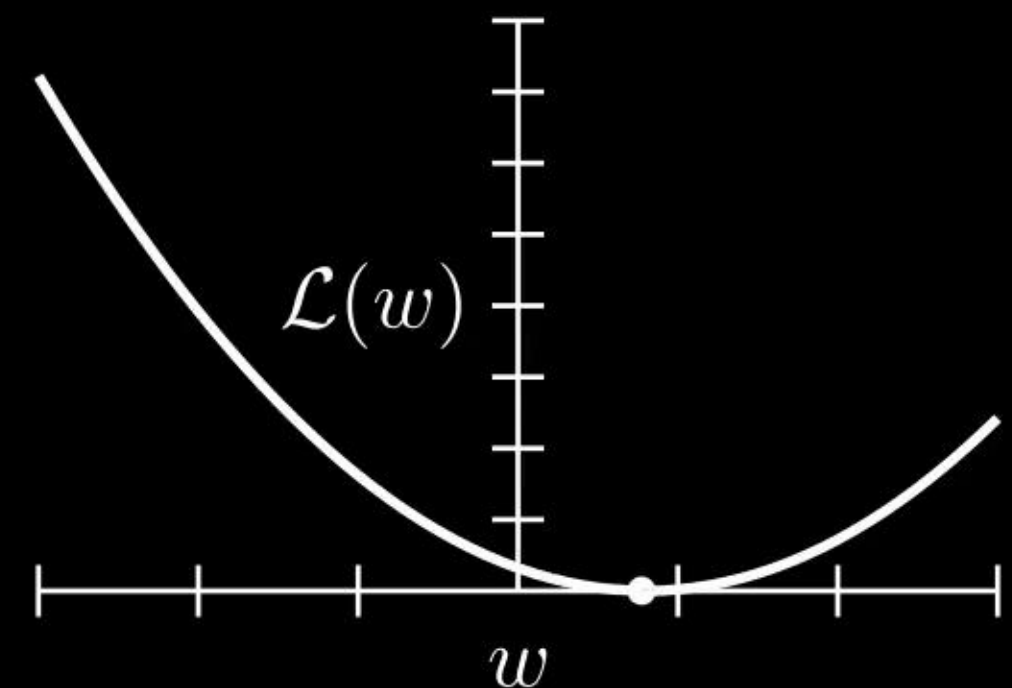
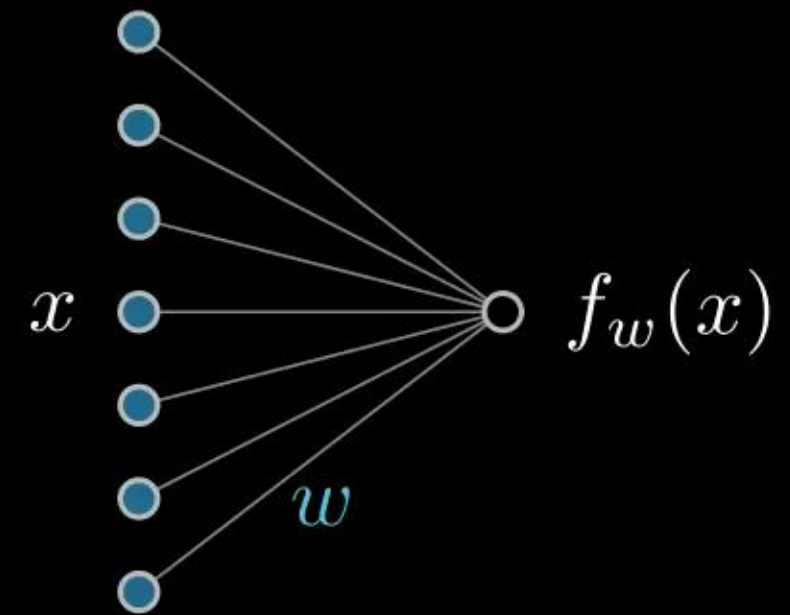
$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$

$$\mathcal{L} \text{ is convex} \Rightarrow \nabla_w \mathcal{L}(w^*) = 0$$

$$XX^\top w^* = Xy$$

(unique if $XX^\top \succ 0$)



Linear models and gradient descent

Setup: $f_w(x) = x^\top w, \quad w \in \mathbb{R}^d$

Loss: $\mathcal{L}(w) = \frac{1}{2} \|X^\top w - y\|_2^2$

GD update:

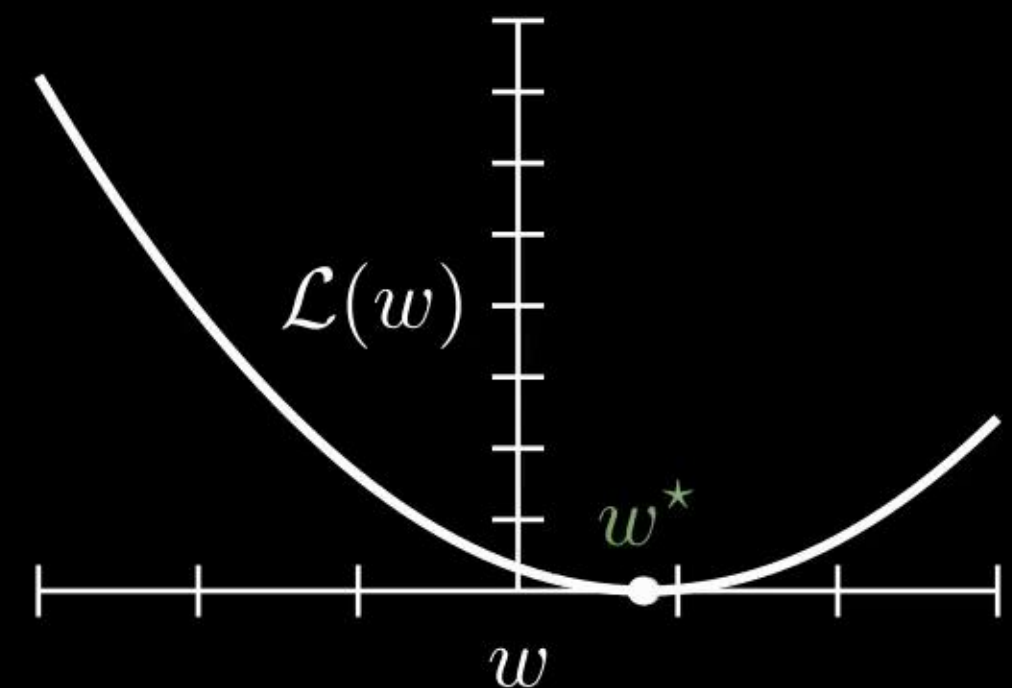
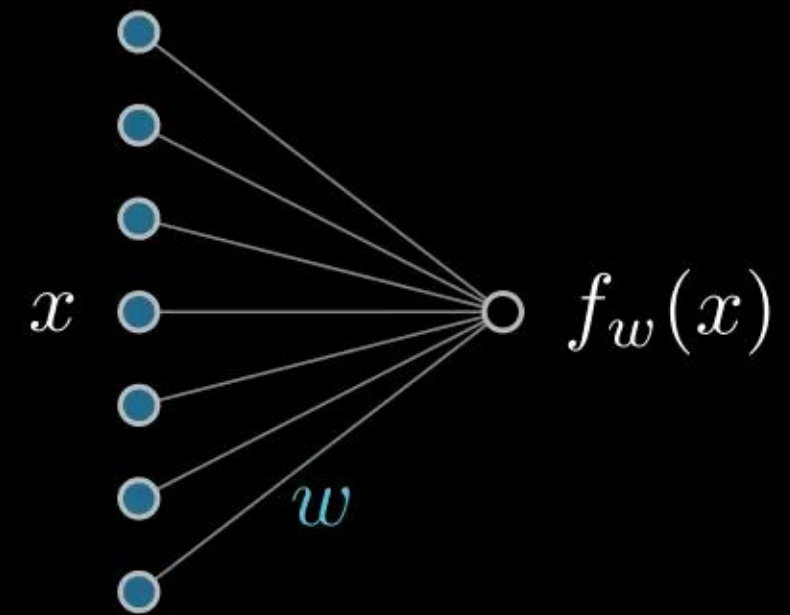
$$w_{t+1} = w_t - \eta \nabla_w \mathcal{L}(w_t)$$

$$\nabla_w \mathcal{L}(w) = X(X^\top w - y) = XX^\top w - Xy$$

$$\mathcal{L} \text{ is convex} \Rightarrow \nabla_w \mathcal{L}(w^*) = 0$$

$$XX^\top w^* = Xy$$

(unique if $XX^\top \succ 0$)



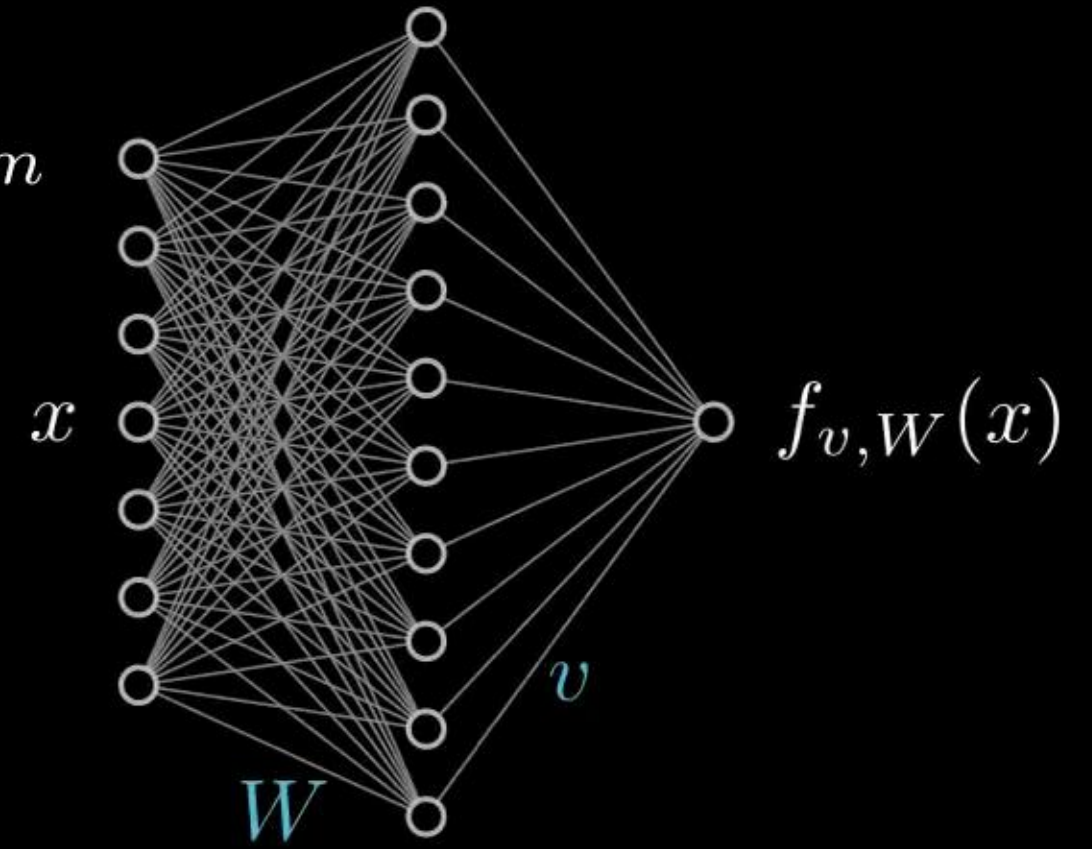
Deep linear models and gradient descent

Deep linear models and gradient descent

Setup: $f_{v,W}(x) = v^\top(Wx), \quad W \in \mathbb{R}^{m \times d}, \quad v \in \mathbb{R}^m$

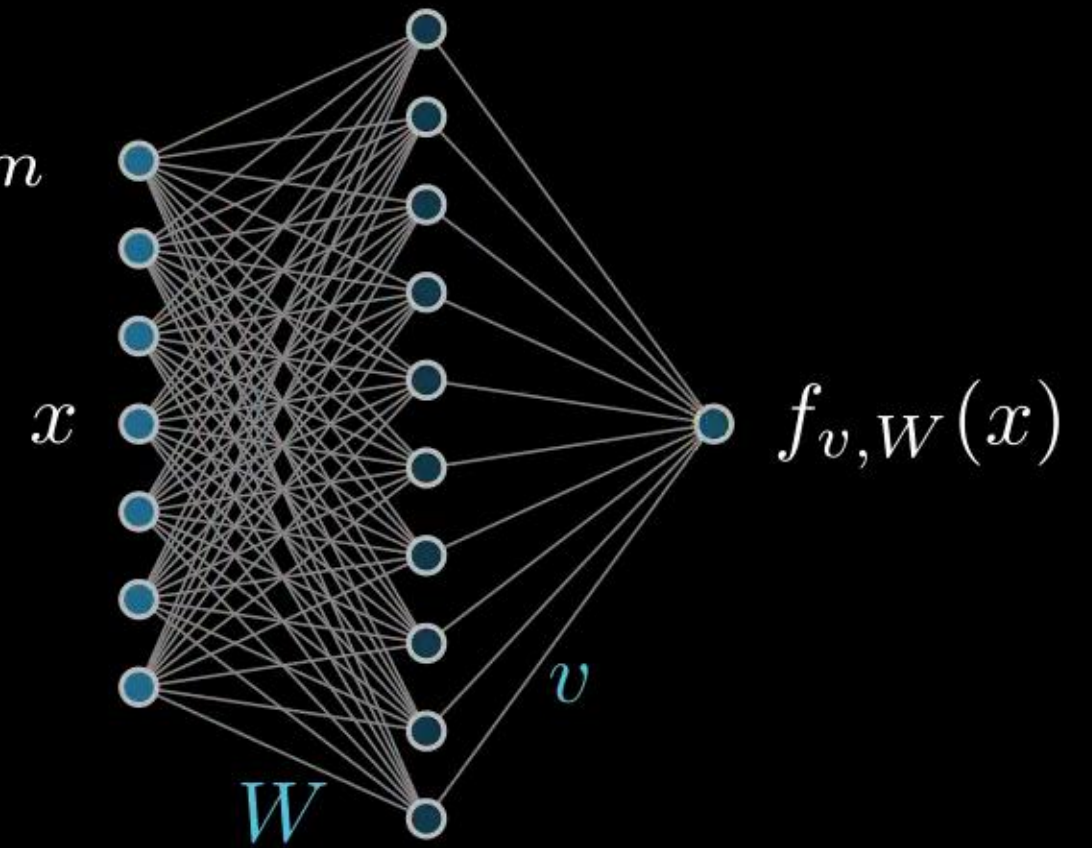
Deep linear models and gradient descent

Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$



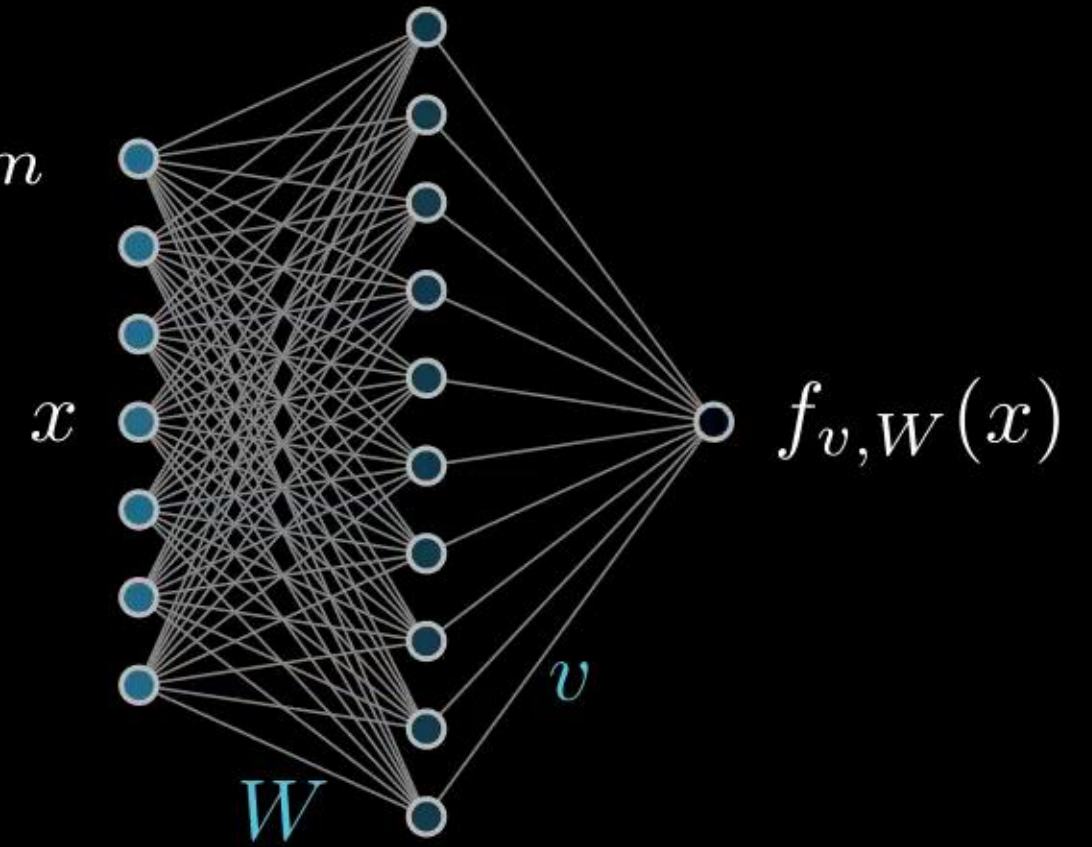
Deep linear models and gradient descent

Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$



Deep linear models and gradient descent

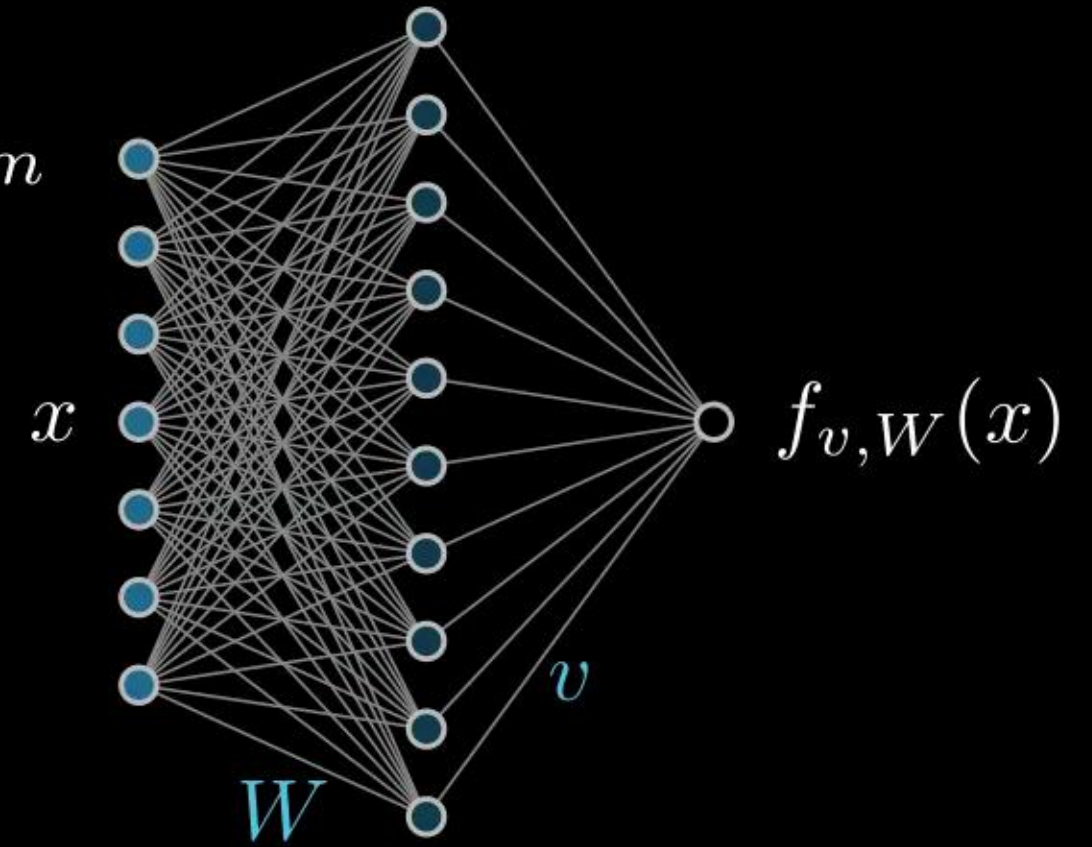
Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$



Deep linear models and gradient descent

Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

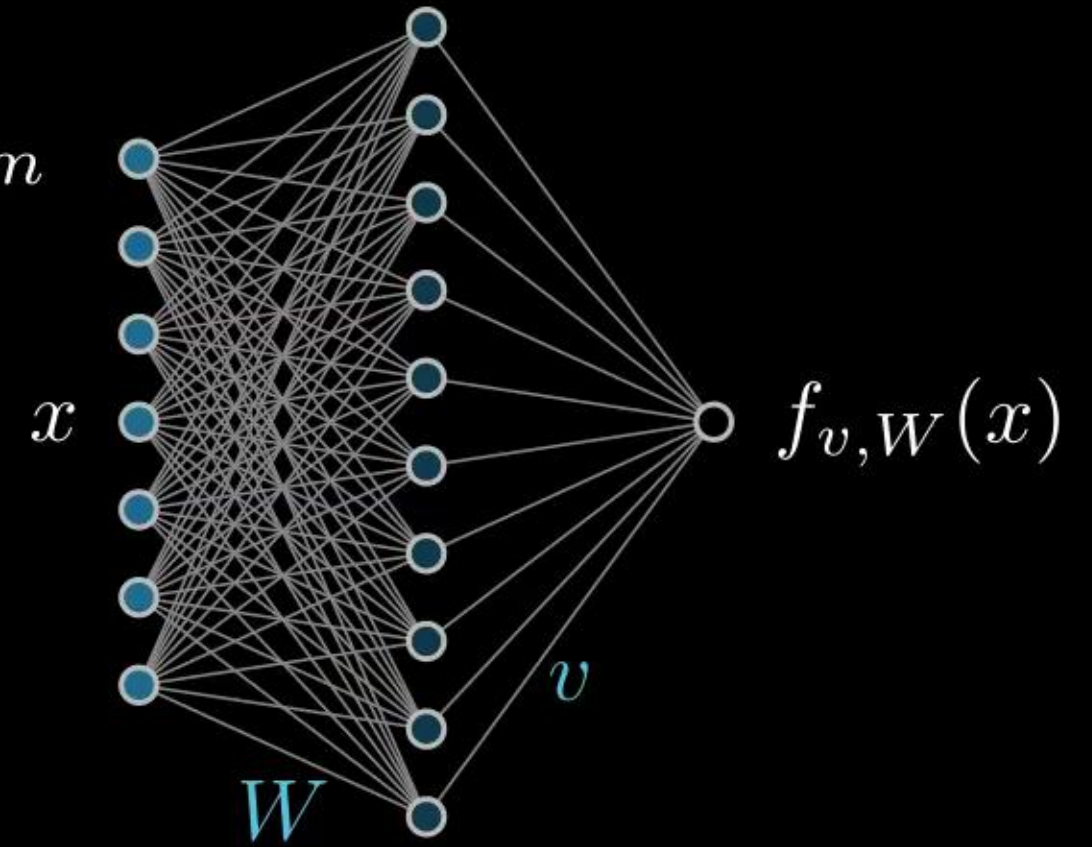


Deep linear models and gradient descent

Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

GD update:



Deep linear models and gradient descent

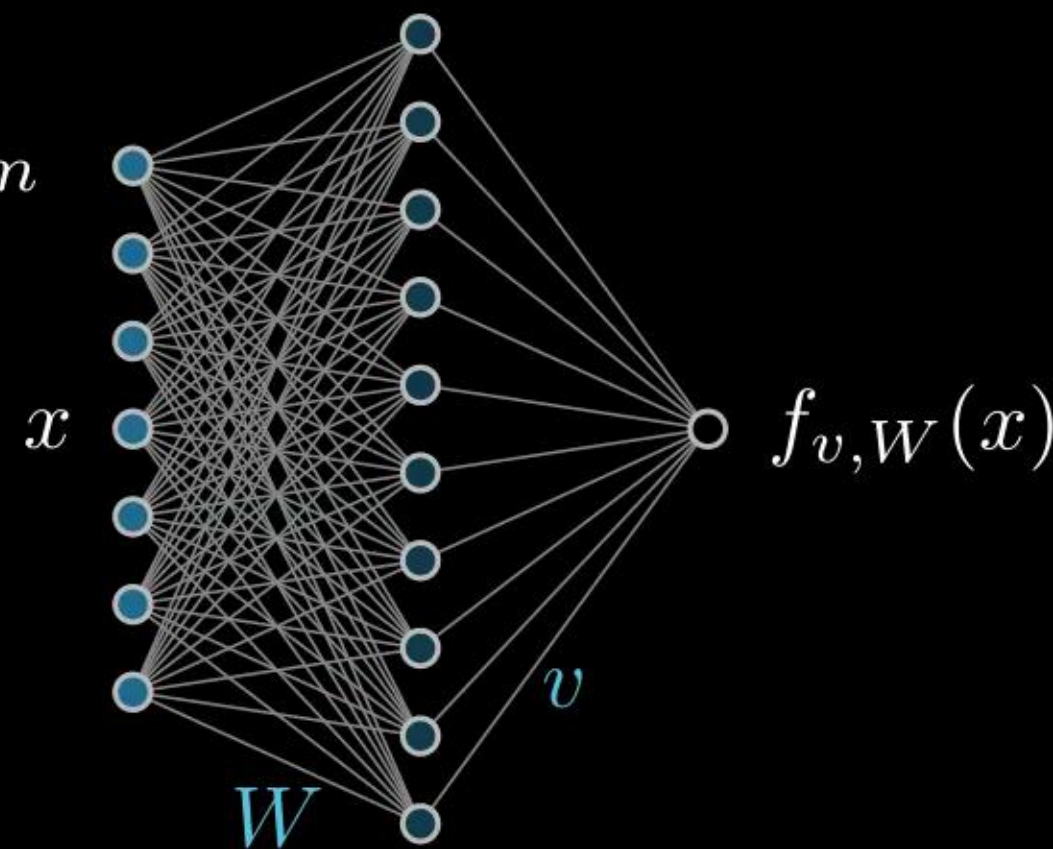
Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$



Deep linear models and gradient descent

Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

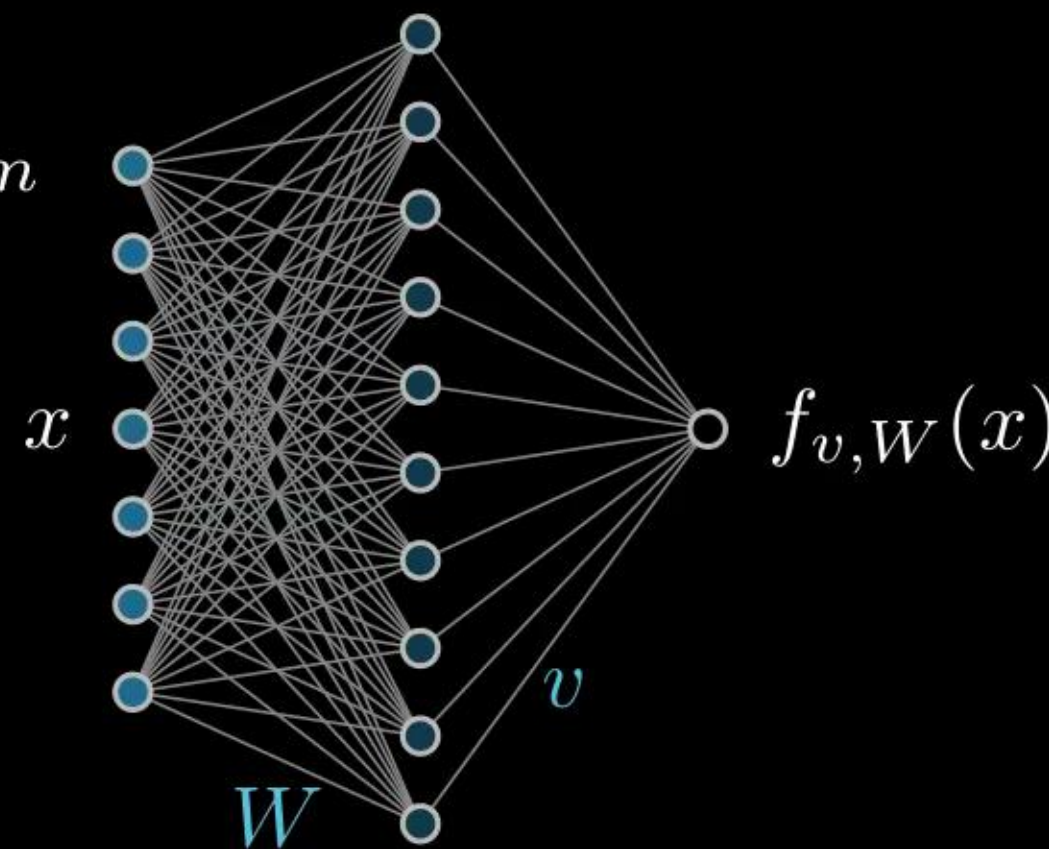
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = W_t X (X^\top W_t^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = v_t (X^\top W_t^\top v_t - y)^\top X^\top$$



Deep linear models and gradient descent

Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

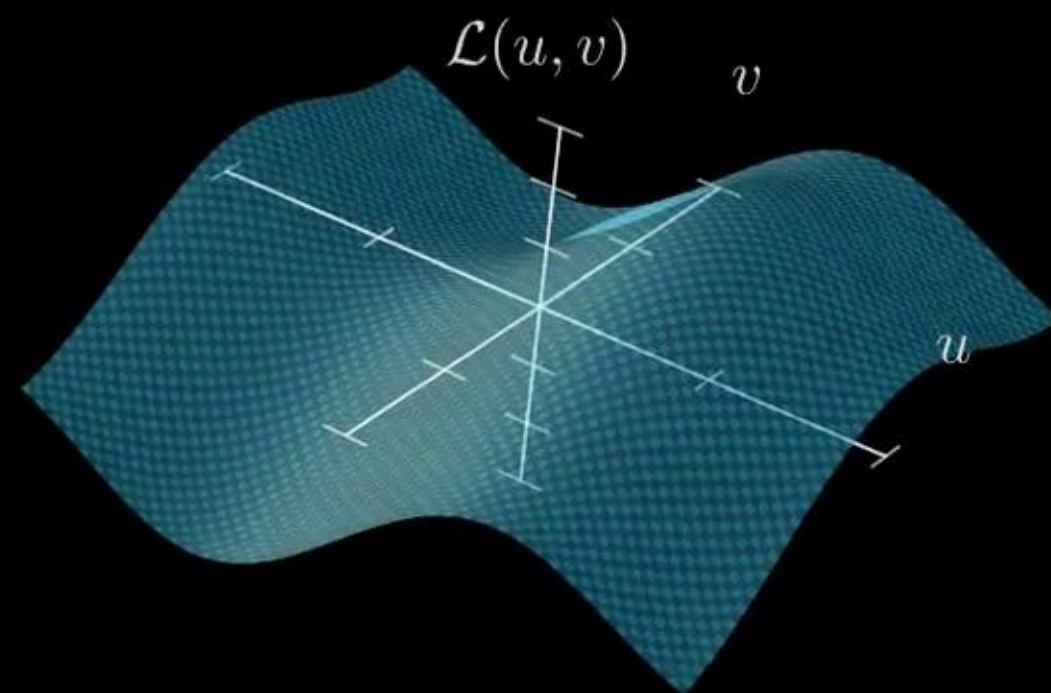
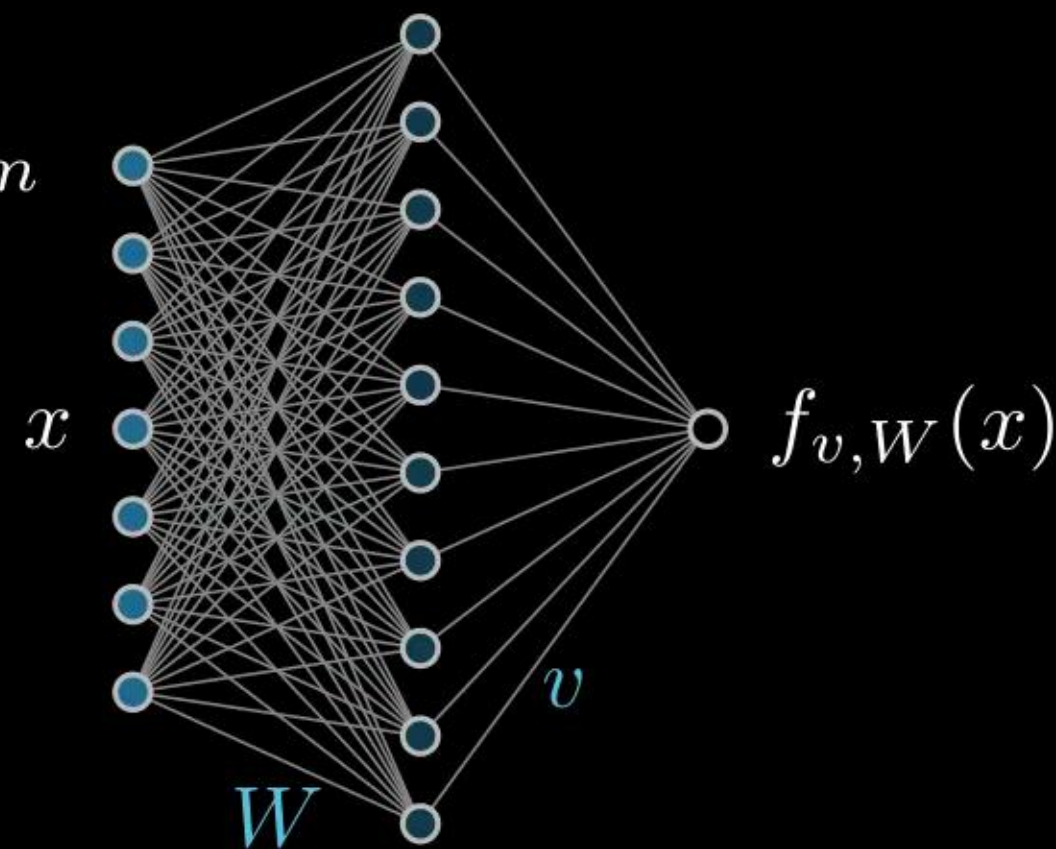
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = W_t X (X^\top W_t^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = v_t (X^\top W_t^\top v_t - y)^\top X^\top$$



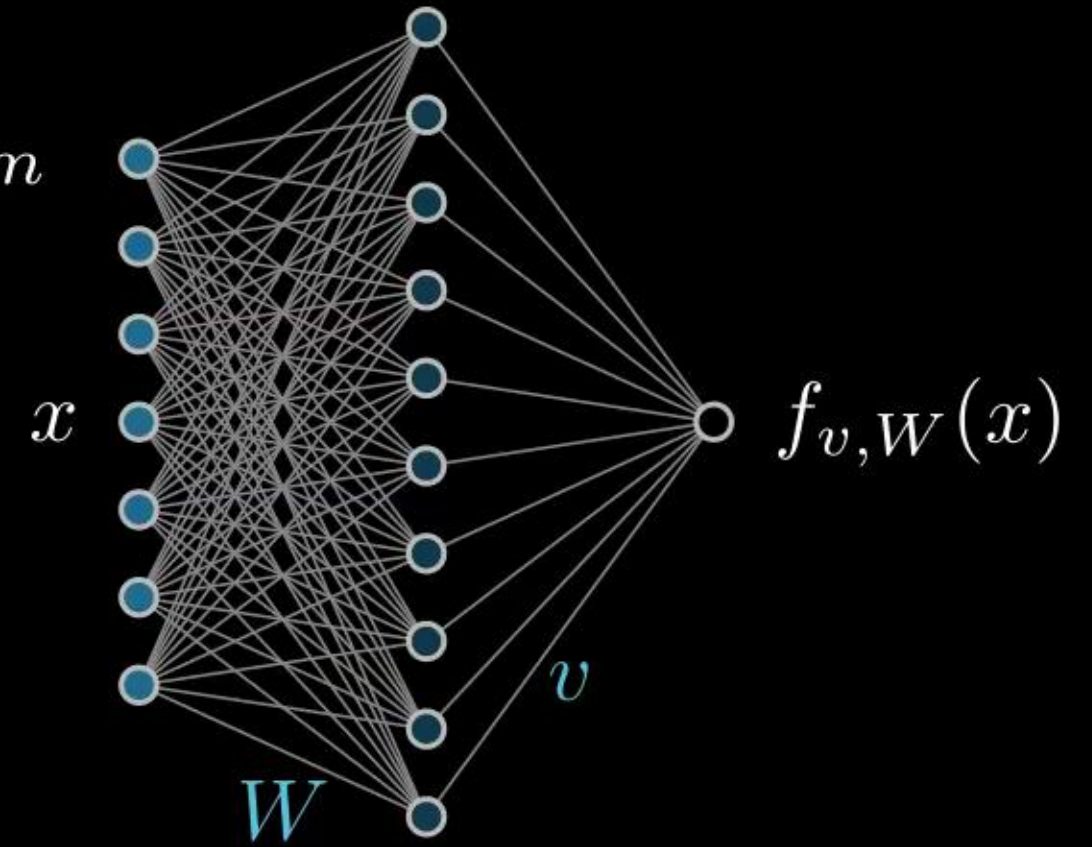
Nonlinear Models and gradient descent

Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

GD update:



Nonlinear Models and gradient descent

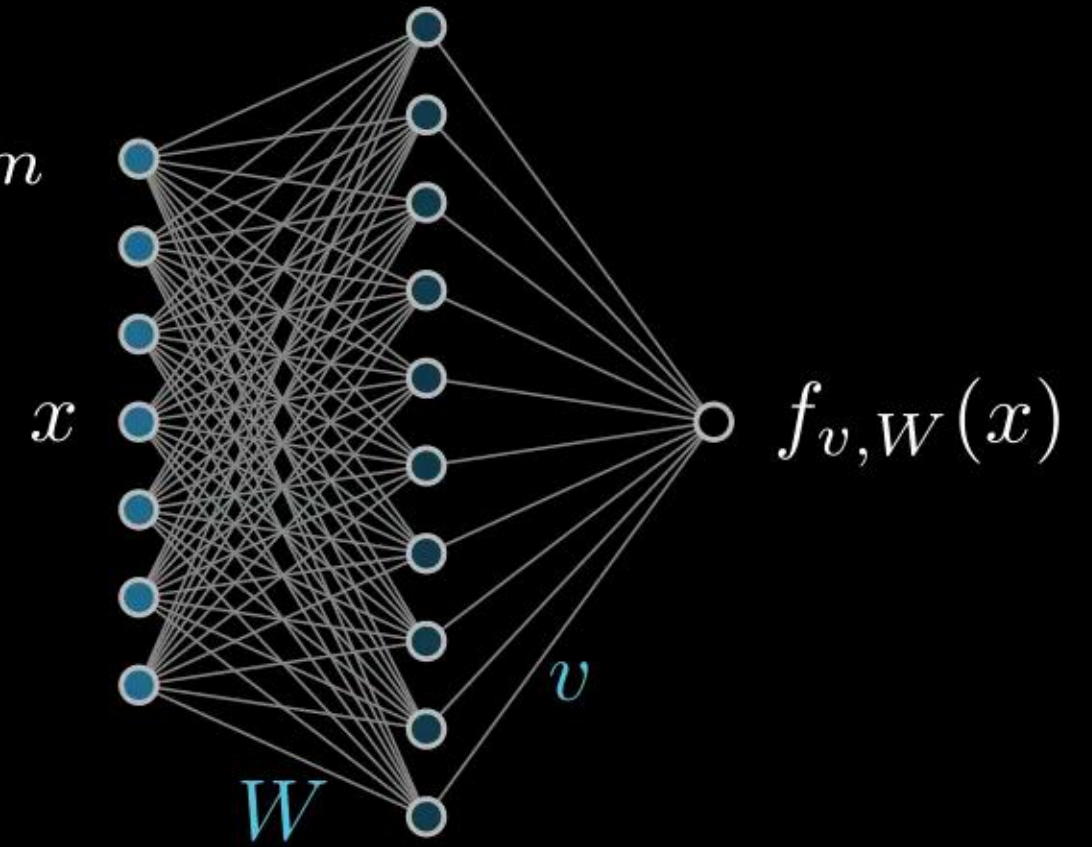
Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$



Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top (Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

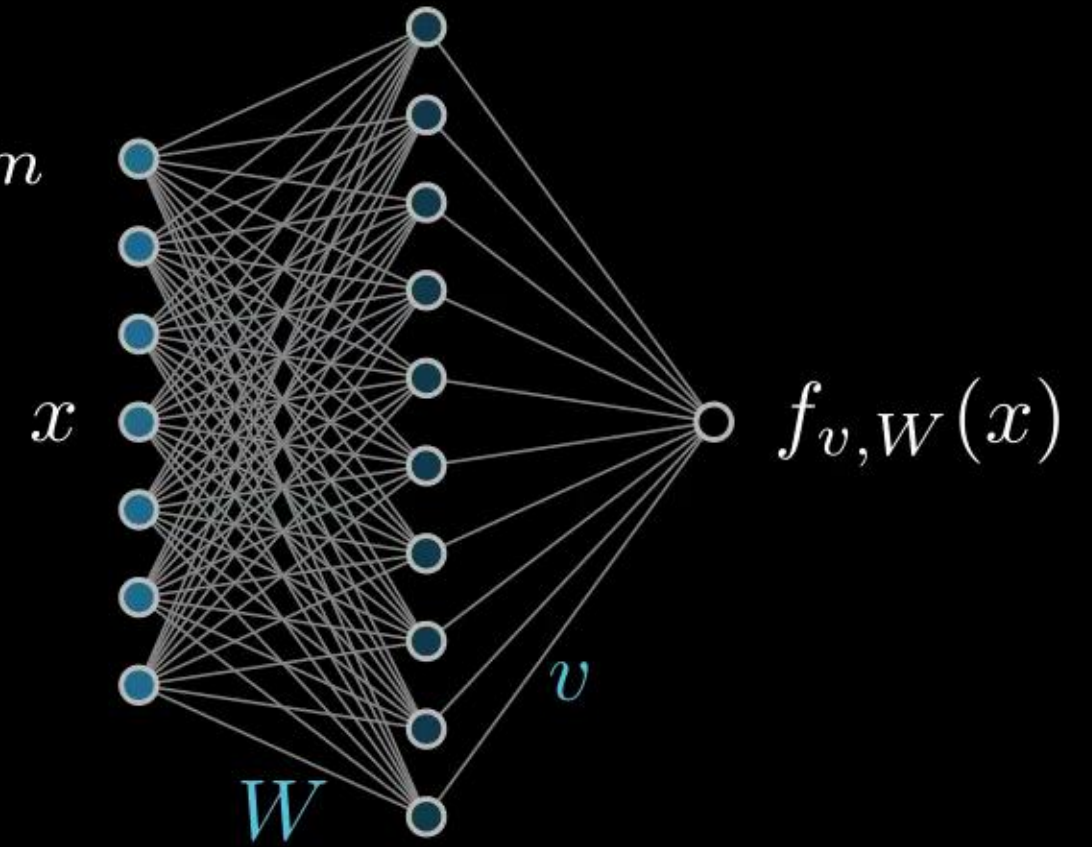
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = W_t X (X^\top W_t^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = v_t (X^\top W_t^\top v_t - y)^\top X^\top$$



Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top \phi(Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|X^\top W^\top v - y\|_2^2$

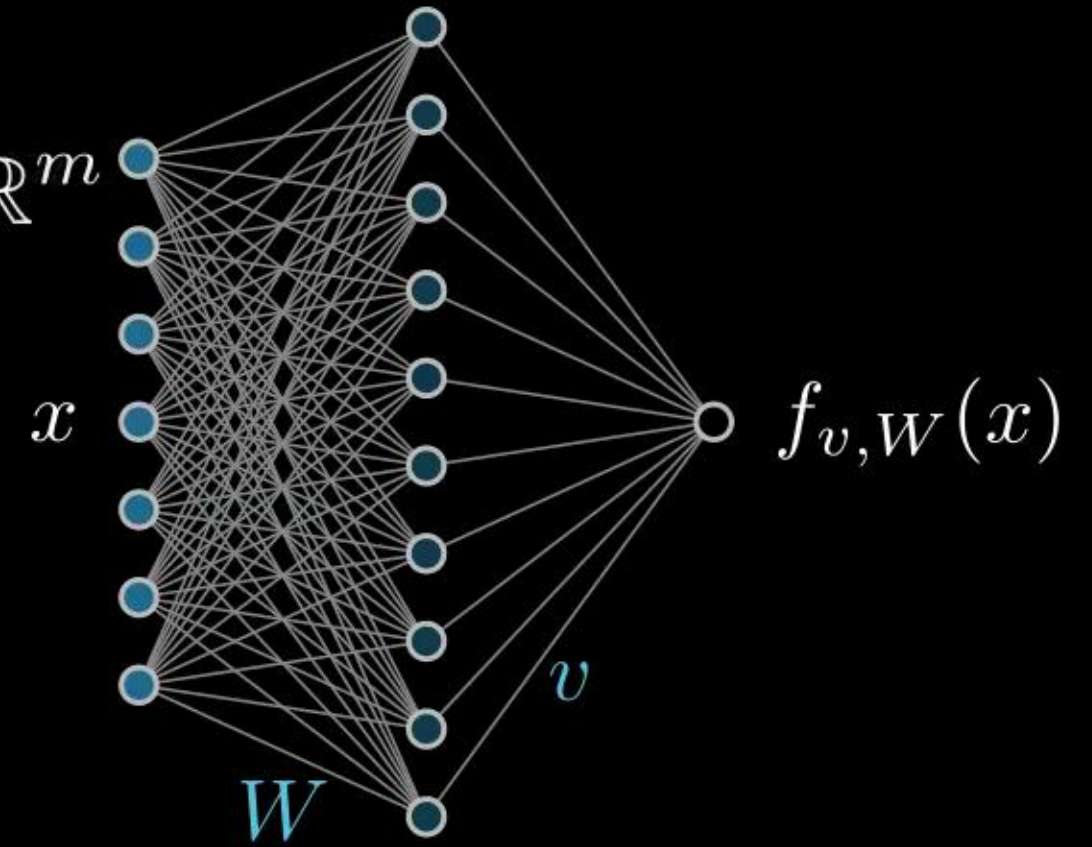
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = W_t X (X^\top W_t^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = v_t (X^\top W_t^\top v_t - y)^\top X^\top$$



Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top \phi(Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|\phi(WX)^\top v - y\|_2^2$

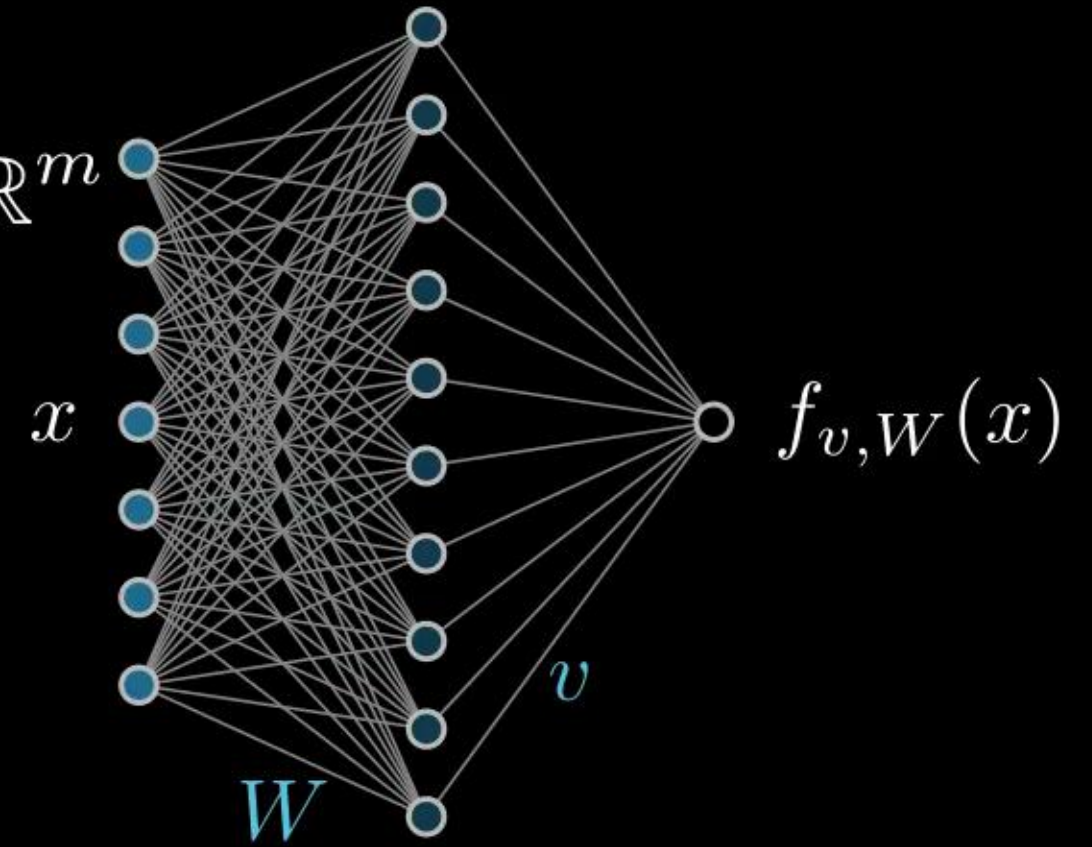
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = W_t X (X^\top W_t^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = v_t (X^\top W_t^\top v_t - y)^\top X^\top$$



Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top \phi(Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|\phi(WX)^\top v - y\|_2^2$

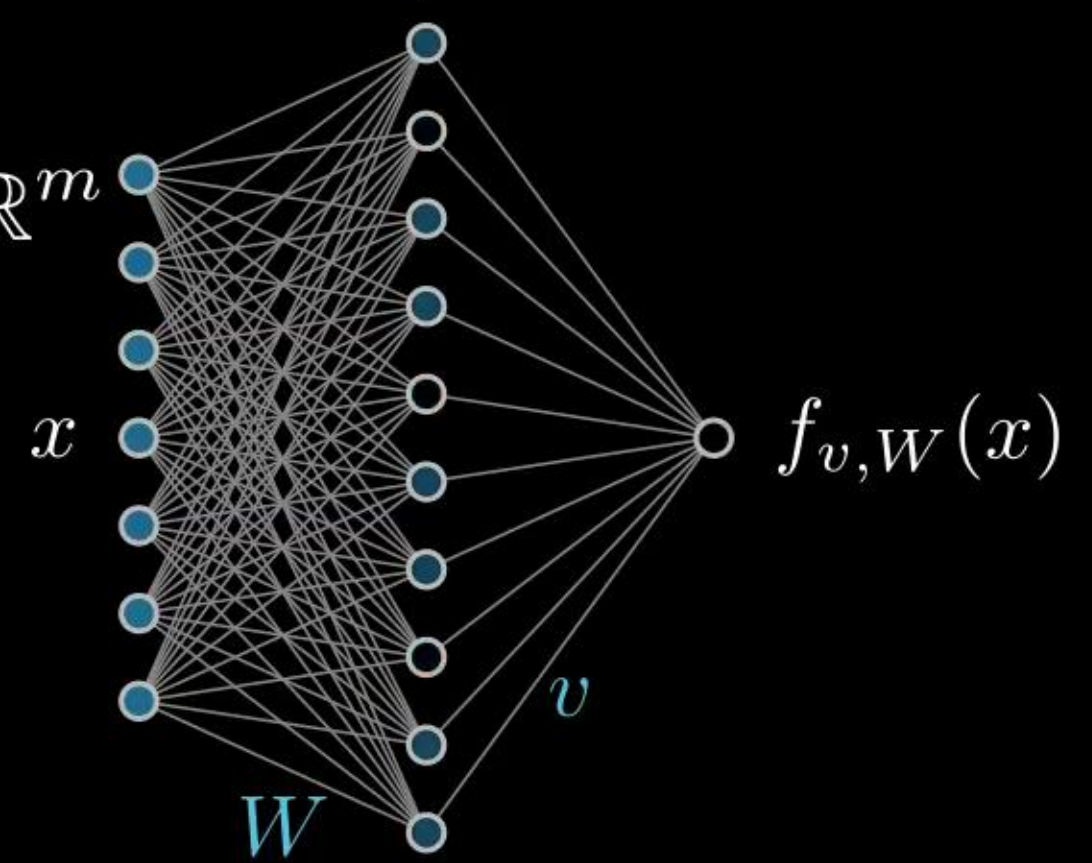
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = W_t X (X^\top W_t^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = v_t (X^\top W_t^\top v_t - y)^\top X^\top$$



Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top \phi(Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|\phi(WX)^\top v - y\|_2^2$

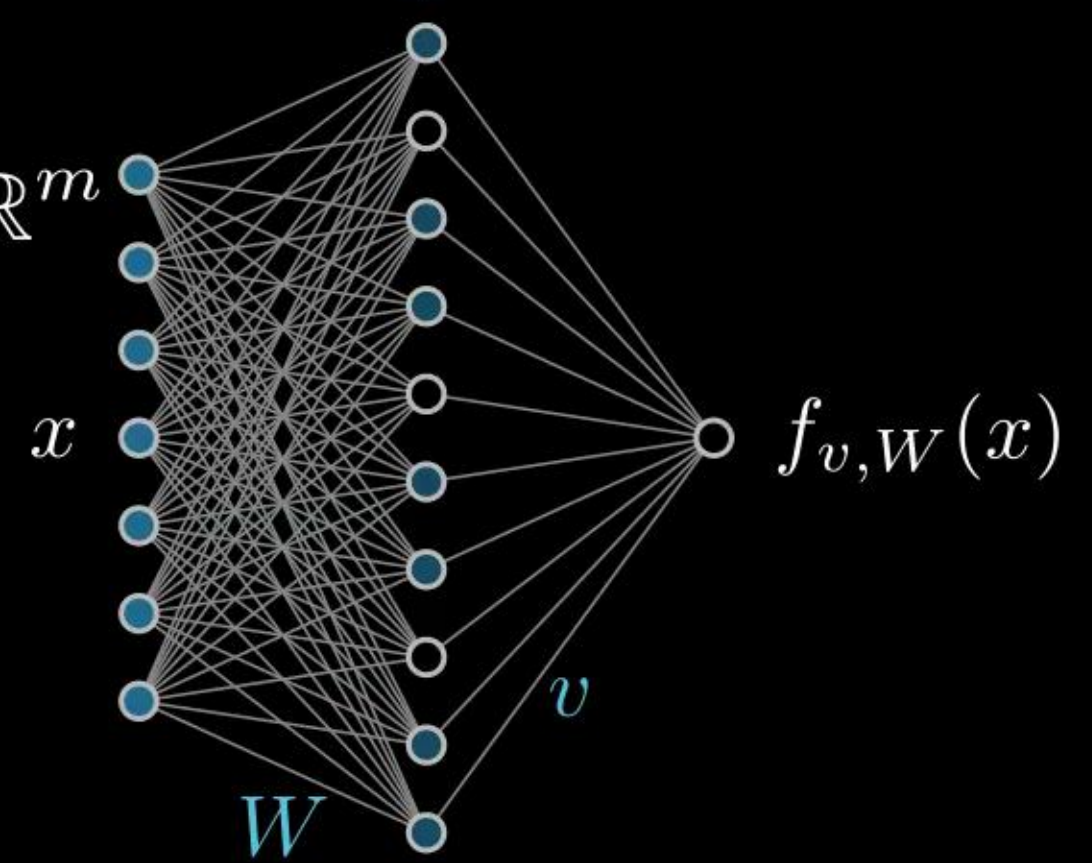
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = \phi(W_t X) (\phi(W_t X)^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = v_t (X^\top W_t^\top v_t - y)^\top X^\top$$



Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top \phi(Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|\phi(WX)^\top v - y\|_2^2$

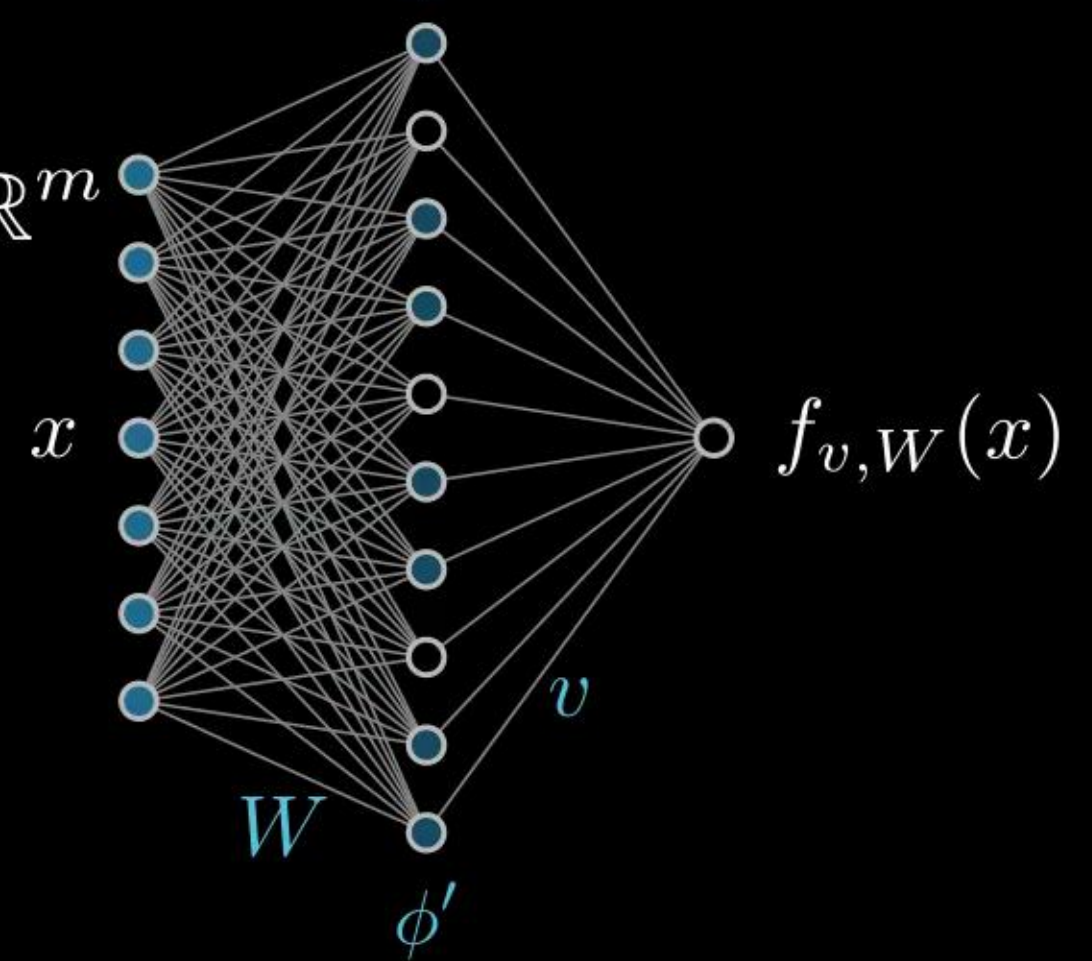
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = \phi(W_t X) (\phi(W_t X)^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = (\phi'(W_t X) \odot (v_t (\phi(W_t X)^\top v_t - y)^\top)) X^\top$$



Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top \phi(Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|\phi(WX)^\top v - y\|_2^2$

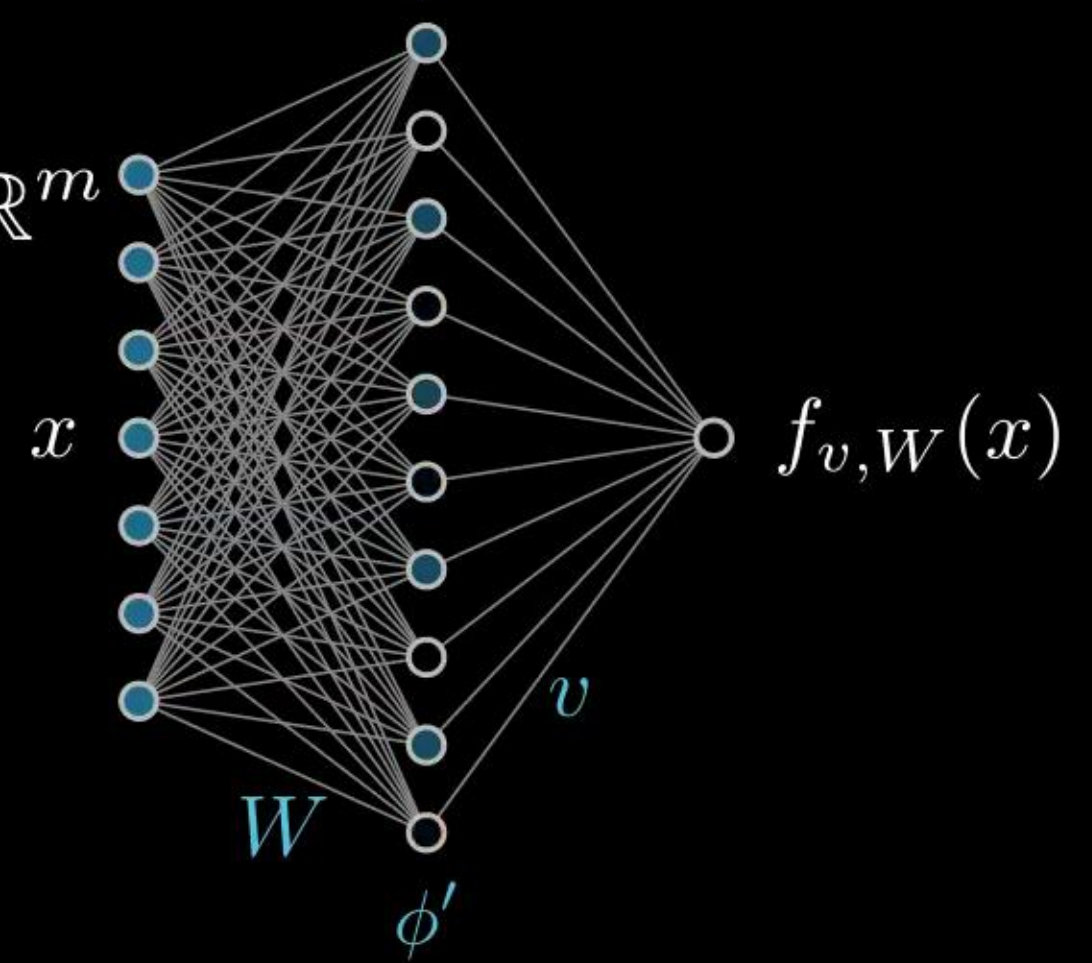
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = \phi(W_t X) (\phi(W_t X)^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = (\phi'(W_t X) \odot (v_t (\phi(W_t X)^\top v_t - y)^\top)) X^\top$$



Nonlinear Models and gradient descent

Setup: $f_{v,W}(x) = v^\top \phi(Wx)$, $W \in \mathbb{R}^{m \times d}$, $v \in \mathbb{R}^m$

Loss: $\mathcal{L}(v, W) = \frac{1}{2} \|\phi(WX)^\top v - y\|_2^2$

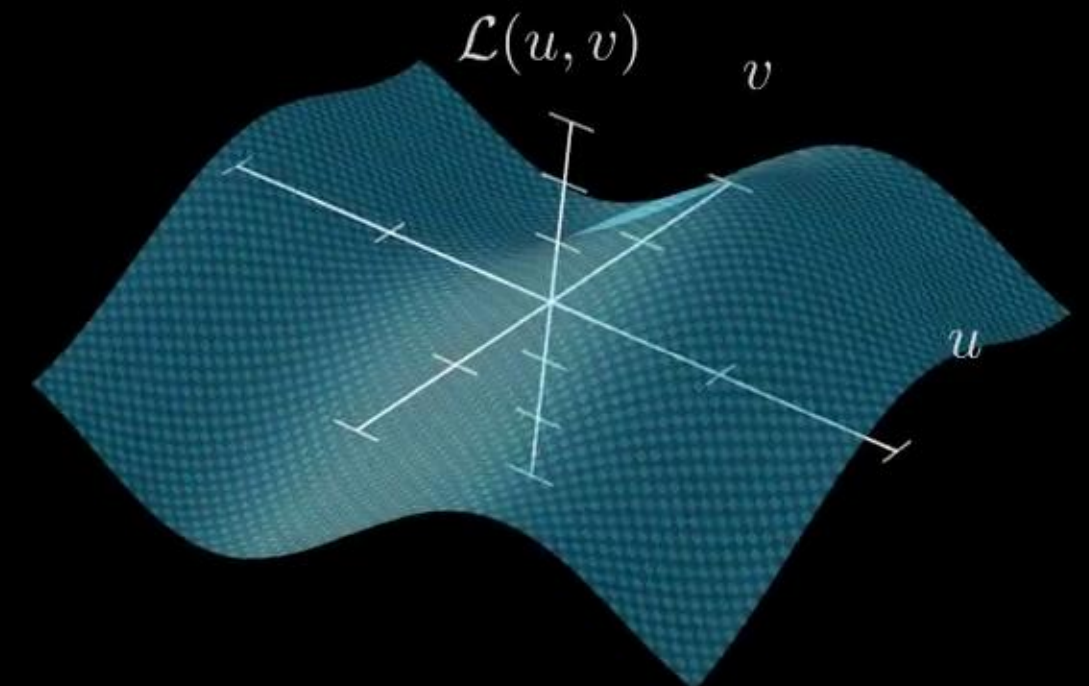
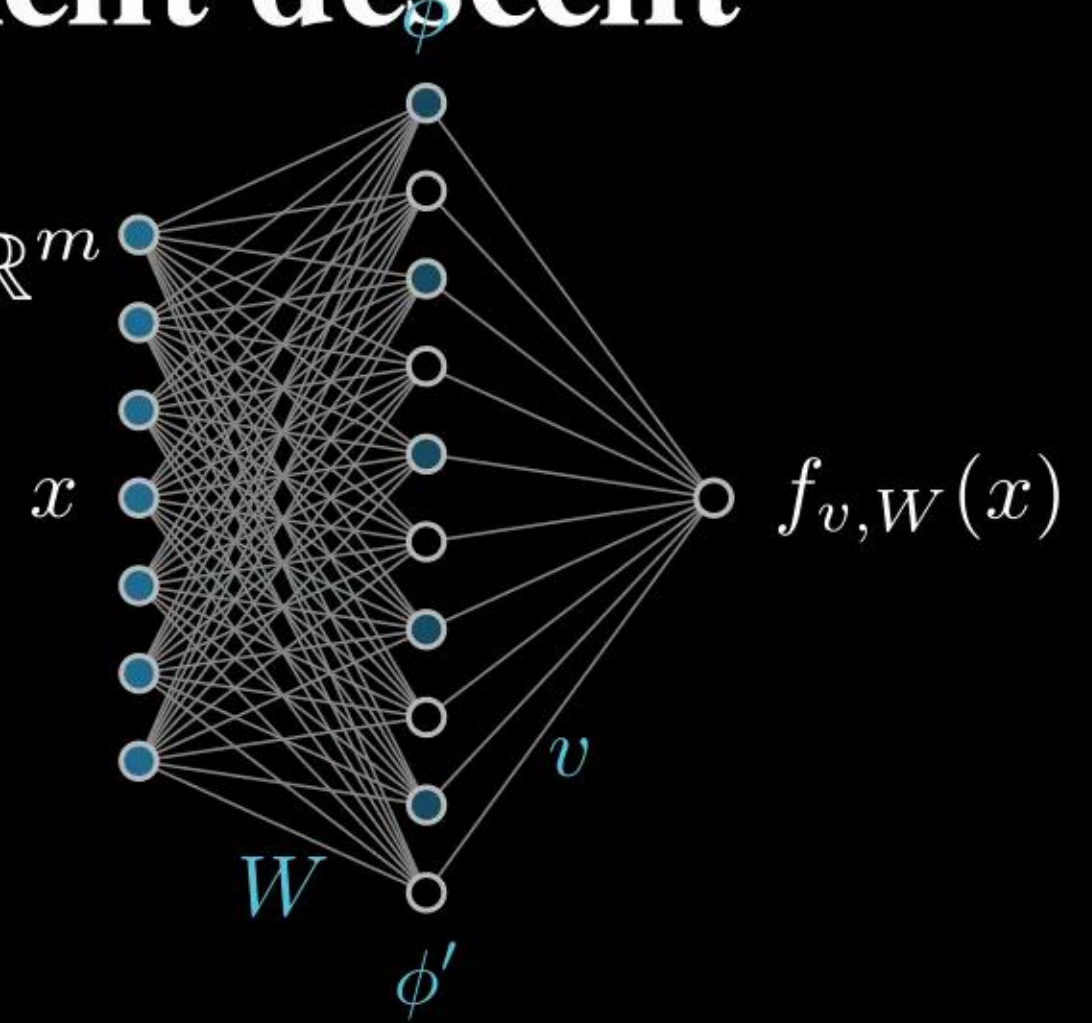
GD update:

$$v_{t+1} = v_t - \eta \nabla_v \mathcal{L}(v_t, W_t)$$

$$W_{t+1} = W_t - \eta \nabla_W \mathcal{L}(v_t, W_t)$$

$$\nabla_v \mathcal{L}(v_t, W_t) = \phi(W_t X) (\phi(W_t X)^\top v_t - y)$$

$$\nabla_W \mathcal{L}(v_t, W_t) = (\phi'(W_t X) \odot (v_t (\phi(W_t X)^\top v_t - y)^\top)) X^\top$$



Why analyze training dynamics?

Why analyze training dynamics?

Start with a linear model. For convenience, use gradient flow.

Why analyze training dynamics?

Start with a linear model. For convenience, use gradient flow.

$$\dot{r}(t) = -A r(t), \quad r(t) = f(t) - y$$

Why analyze training dynamics?

Start with a linear model. For convenience, use gradient flow.

$$\dot{r}(t) = -A r(t), \quad r(t) = f(t) - y$$

How does this system evolve as we train?

Why analyze training dynamics?

Start with a linear model. For convenience, use gradient flow.

$$\dot{r}(t) = -A r(t), \quad r(t) = f(t) - y$$

How does this system evolve as we train?

Homogeneous linear ODE $\Rightarrow r(t) = e^{-At} r(0)$

Why analyze training dynamics?

Start with a linear model. For convenience, use gradient flow.

$$\dot{r}(t) = -A r(t), \quad r(t) = f(t) - y$$

How does this system evolve as we train?

Homogeneous linear ODE $\Rightarrow r(t) = e^{-At} r(0)$

$$A = Q\Lambda Q^\top \Rightarrow |r_i(t)| = e^{-\lambda_i t} |r_i(0)|$$

Why analyze training dynamics?

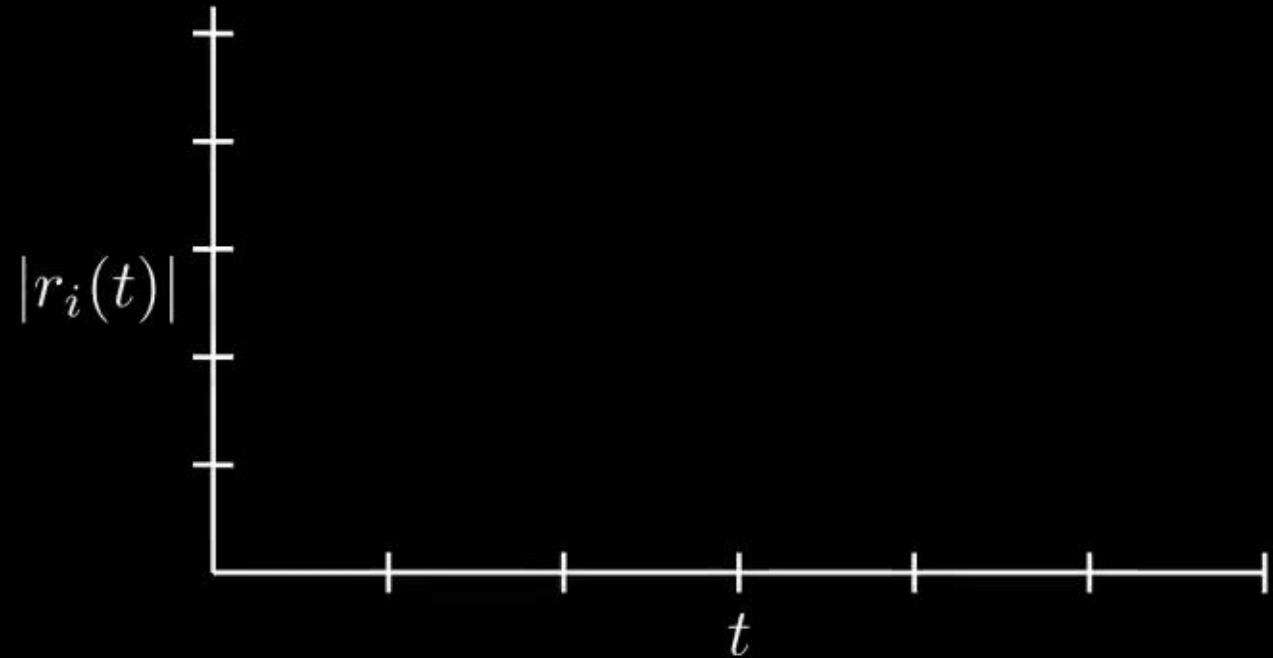
Start with a linear model. For convenience, use gradient flow.

$$\dot{r}(t) = -A r(t), \quad r(t) = f(t) - y$$

How does this system evolve as we train?

Homogeneous linear ODE $\Rightarrow r(t) = e^{-At} r(0)$

$$A = Q\Lambda Q^\top \Rightarrow |r_i(t)| = e^{-\lambda_i t} |r_i(0)|$$



Why analyze training dynamics?

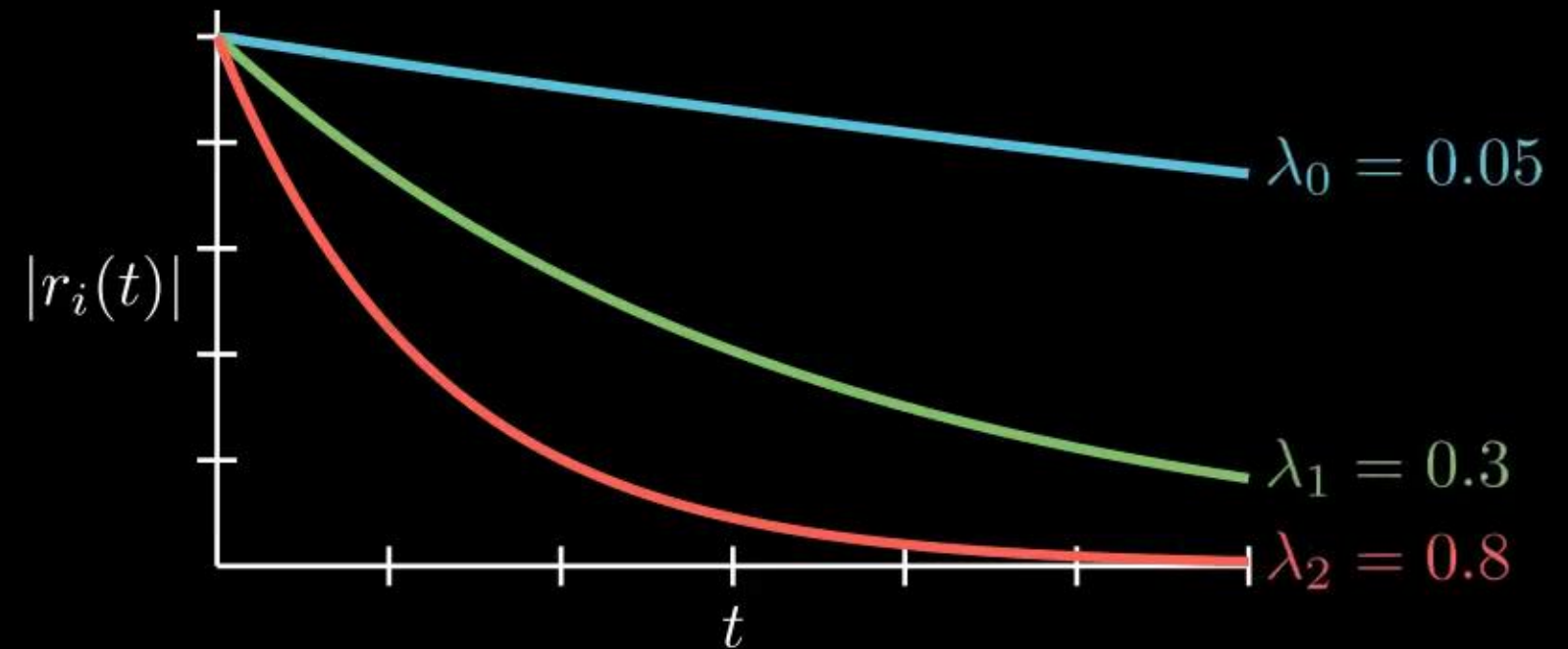
Start with a linear model. For convenience, use gradient flow.

$$\dot{r}(t) = -A r(t), \quad r(t) = f(t) - y$$

How does this system evolve as we train?

Homogeneous linear ODE $\Rightarrow r(t) = e^{-At} r(0)$

$$A = Q\Lambda Q^\top \Rightarrow |r_i(t)| = e^{-\lambda_i t} |r_i(0)|$$



Why analyze training dynamics?

Start with a linear model. For convenience, use gradient flow.

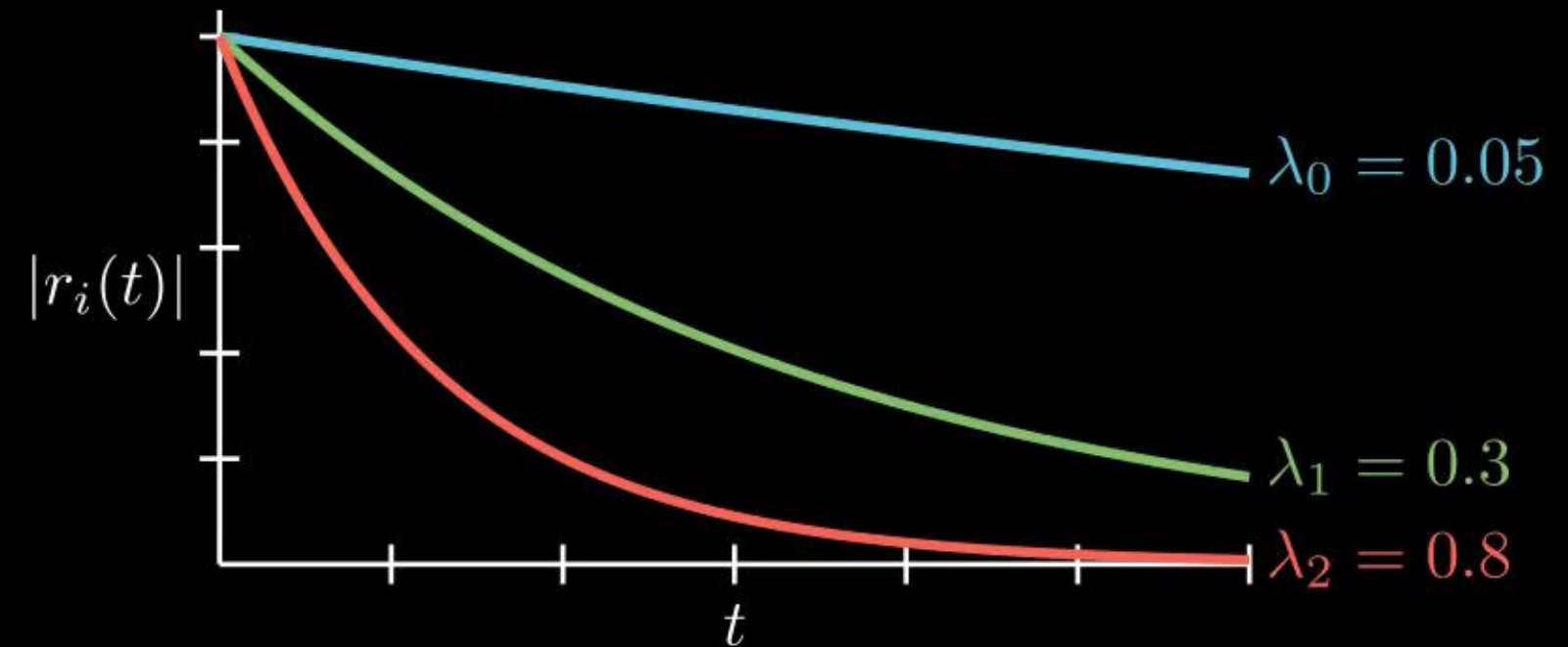
$$\dot{r}(t) = -A r(t), \quad r(t) = f(t) - y$$

How does this system evolve as we train?

Homogeneous linear ODE $\Rightarrow r(t) = e^{-At} r(0)$

$$A = Q\Lambda Q^\top \Rightarrow |r_i(t)| = e^{-\lambda_i t} |r_i(0)|$$

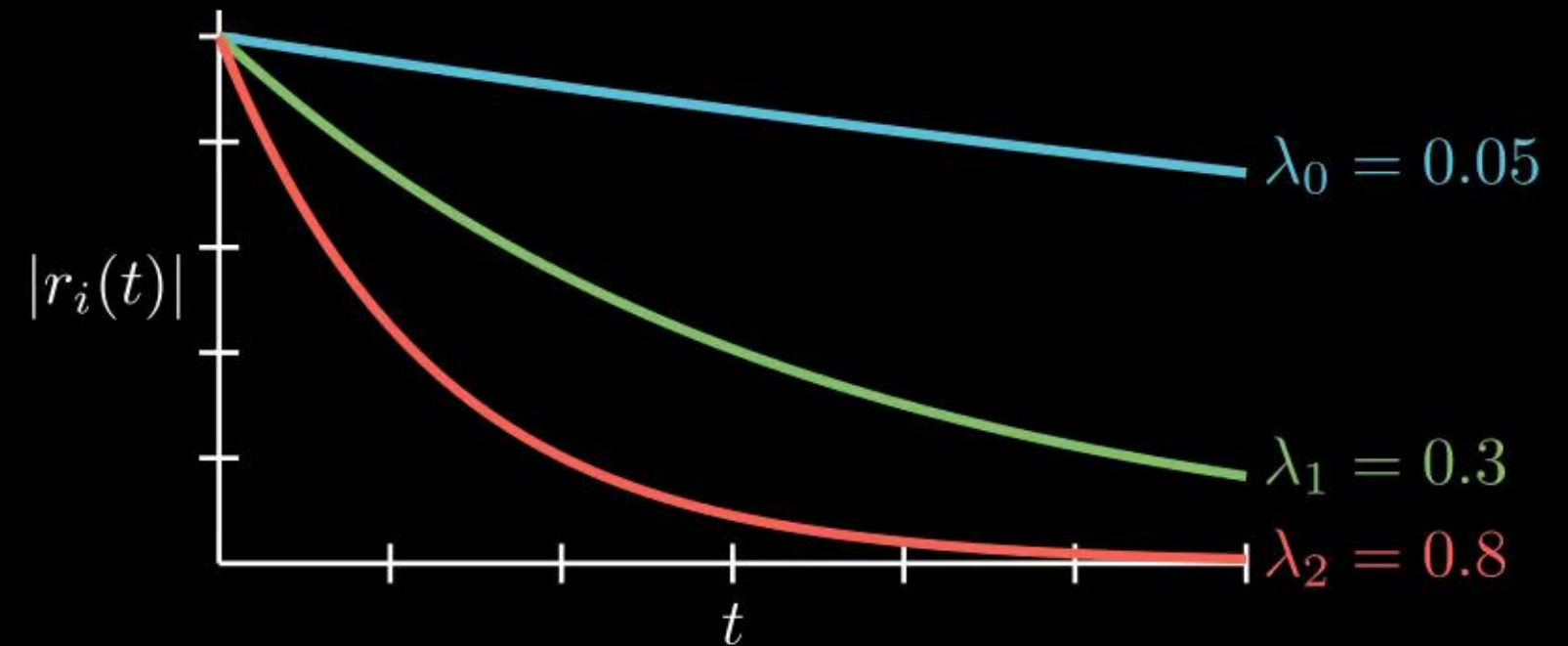
Decay rates are set by the spectrum of A .



Why analyze training dynamics?

- Predict **training time / compute**:
which components take longest?

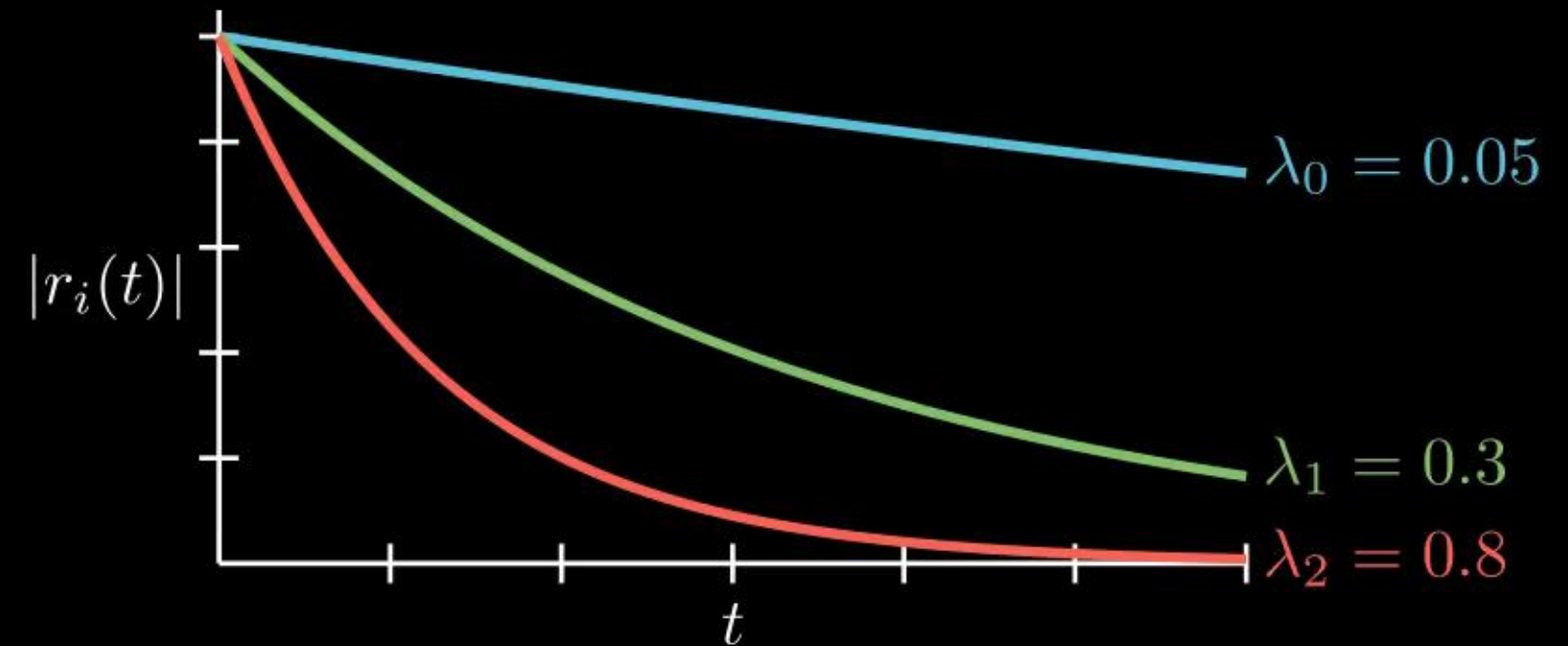
Decay rates are set by the spectrum of A .



Why analyze training dynamics?

- Predict **training time / compute**:
which components take longest?
- Explain **what learns first**:
learning order (**spectral bias**).

Decay rates are set by the spectrum of A .



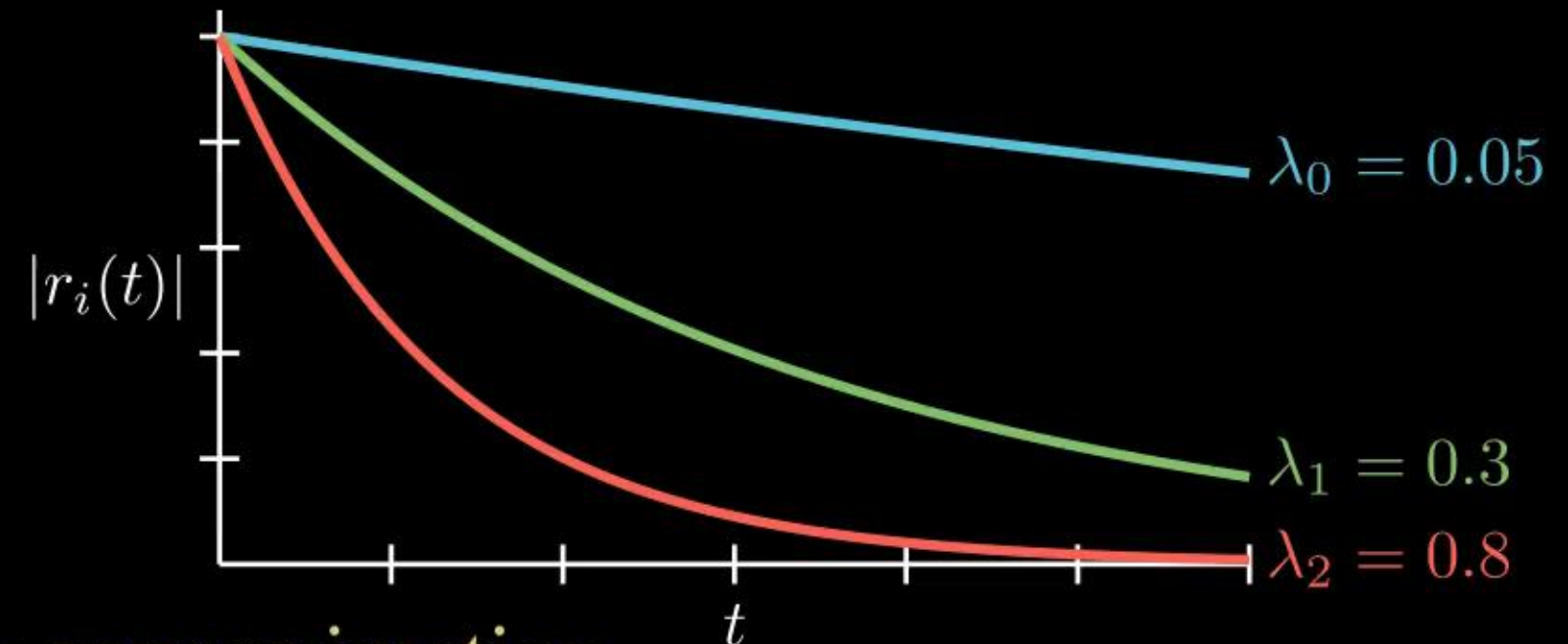
Why analyze training dynamics?

- Predict **training time / compute**:
which components take longest?

- Explain **what learns first**:
learning order (**spectral bias**).

- Bridge theory $\leftrightarrow$ practice:
turn a nonlinear problem into a **tractable approximation**.

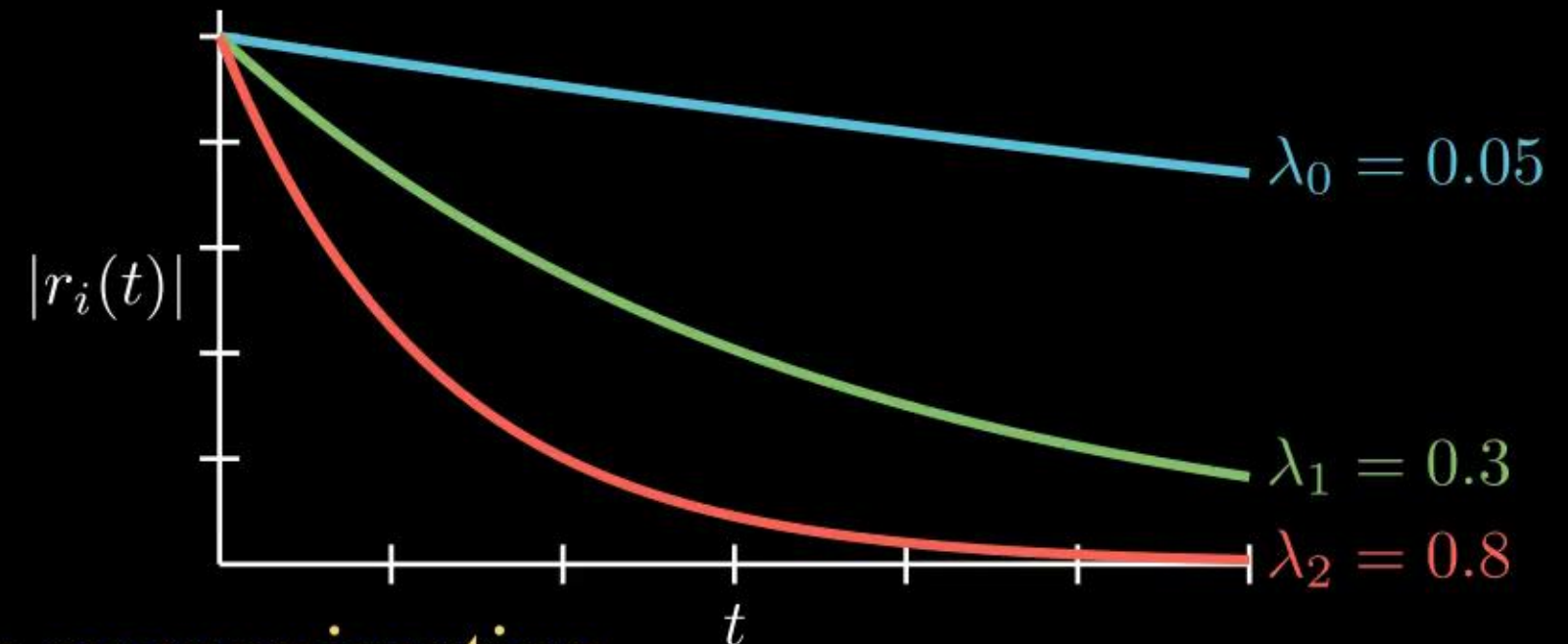
Decay rates are set by the spectrum of A .



Why analyze training dynamics?

- Predict **training time / compute**:
which components take longest?
- Explain **what learns first**:
learning order (**spectral bias**).
- Bridge theory $\leftrightarrow$ practice:
turn a nonlinear problem into a **tractable approximation**.

Decay rates are set by the spectrum of A .



So we look for regimes where the dynamics become linear again.

Linearizing a function

Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

tangent plane

θ

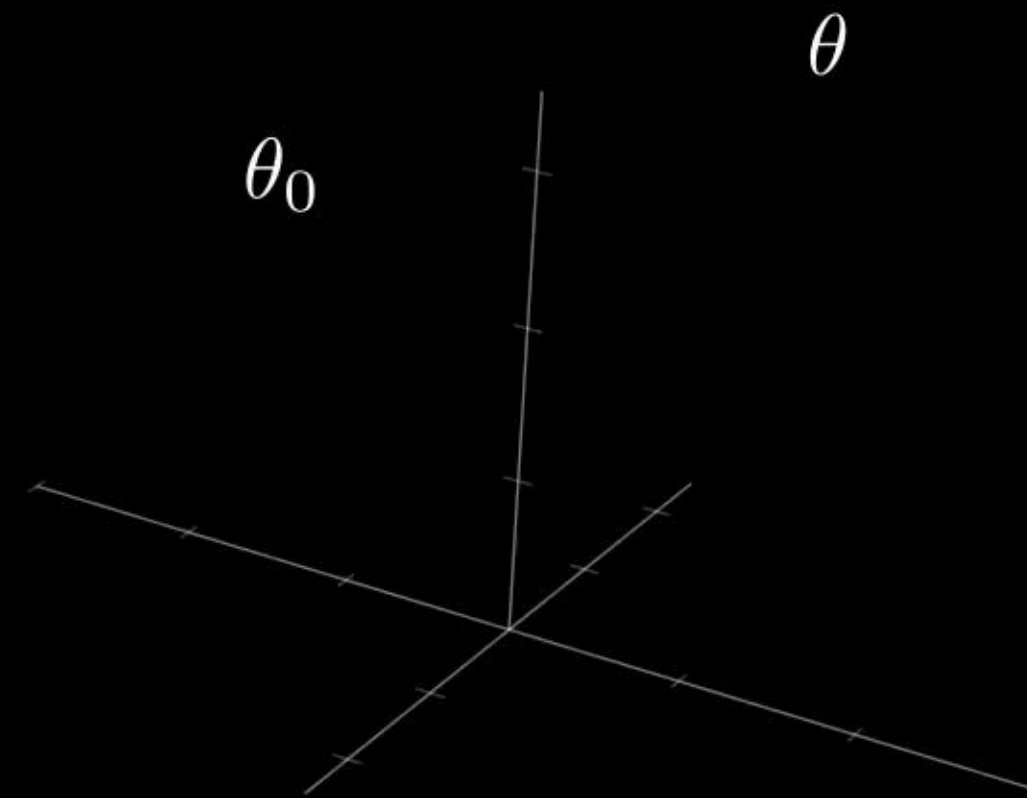
θ_0

Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

tangent plane



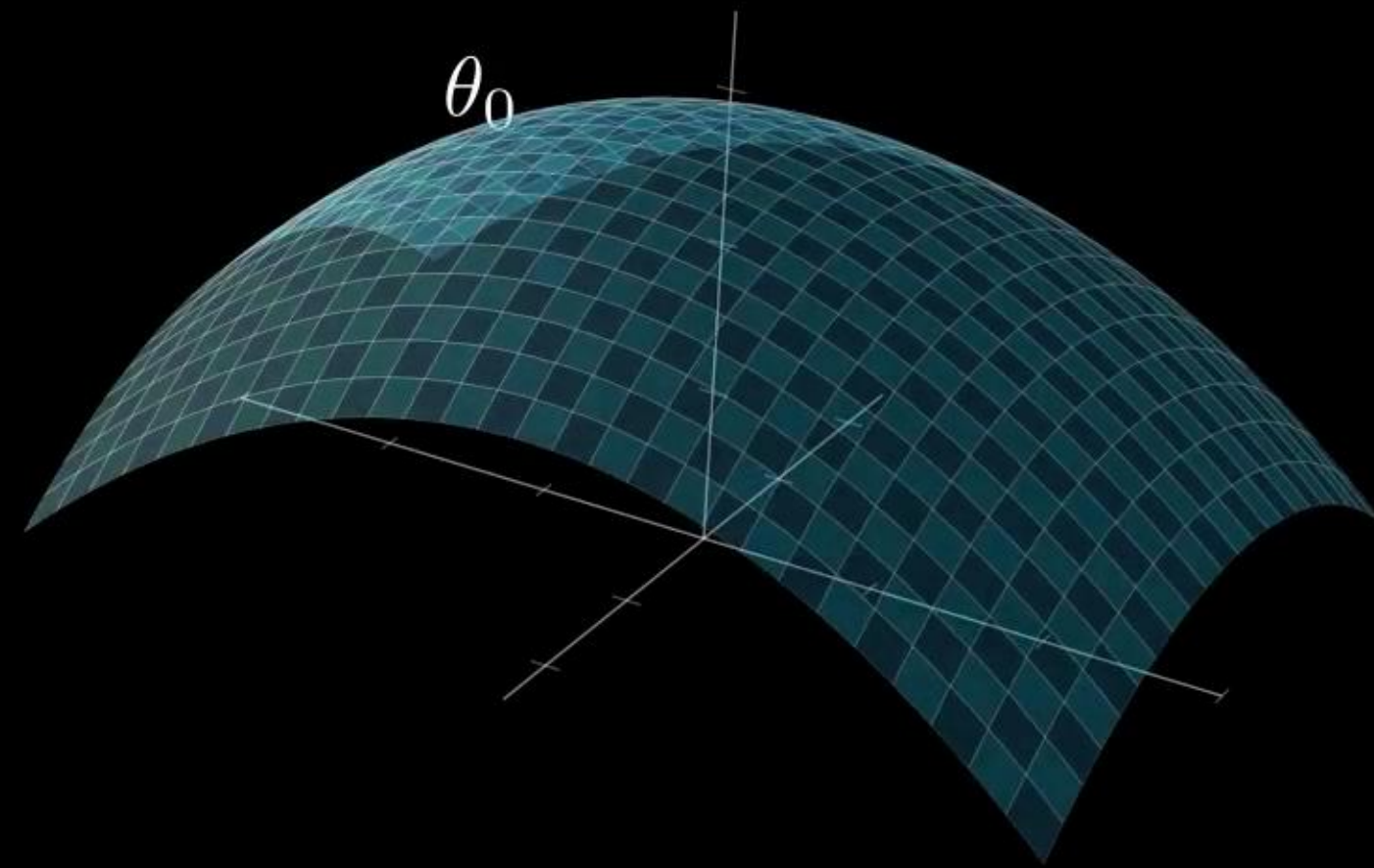
Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

tangent plane

θ



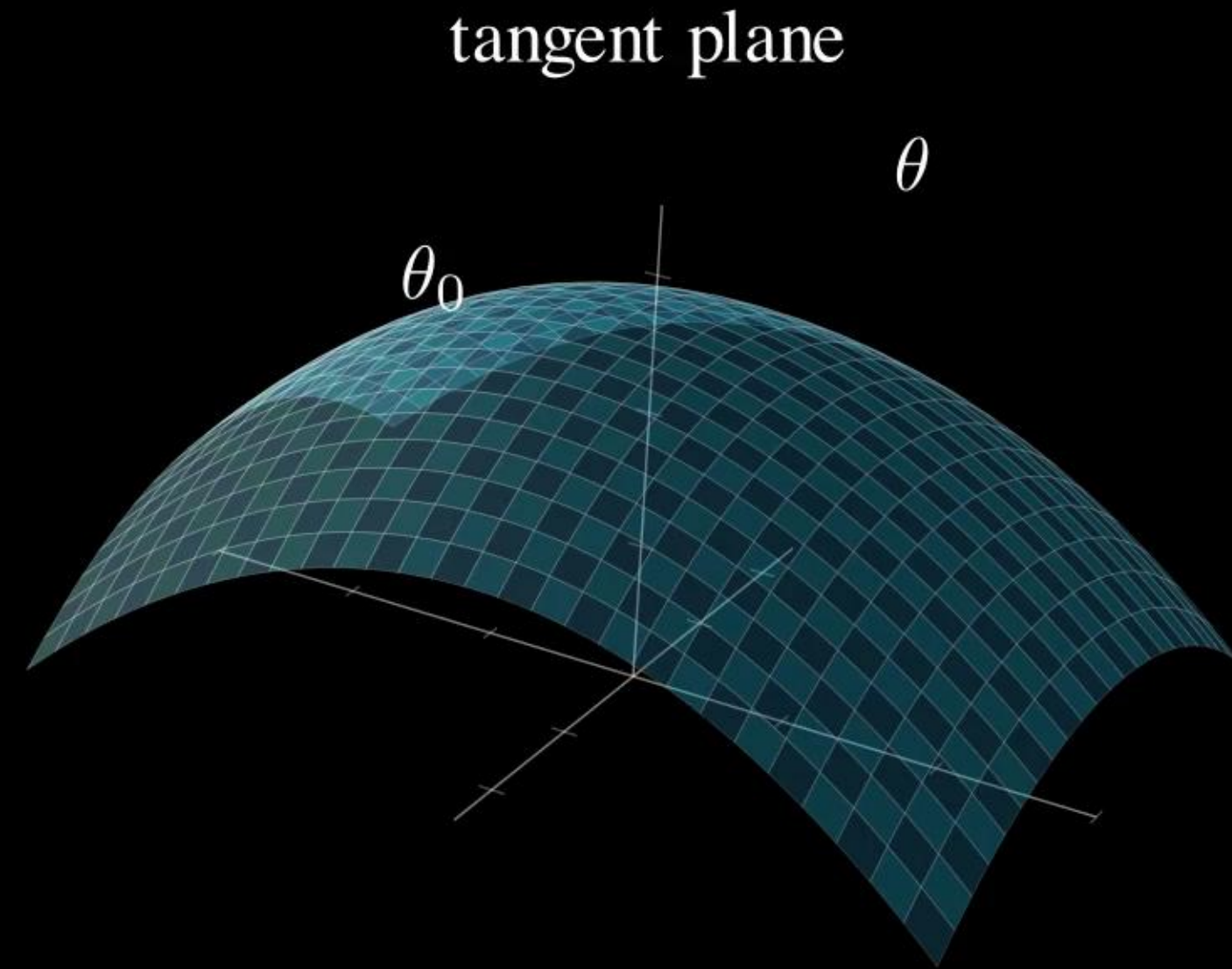
Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

So we linearize around a point:

$$f(\theta) \approx f(\theta_0) + \nabla_{\theta} f(\theta_0) (\theta - \theta_0)$$



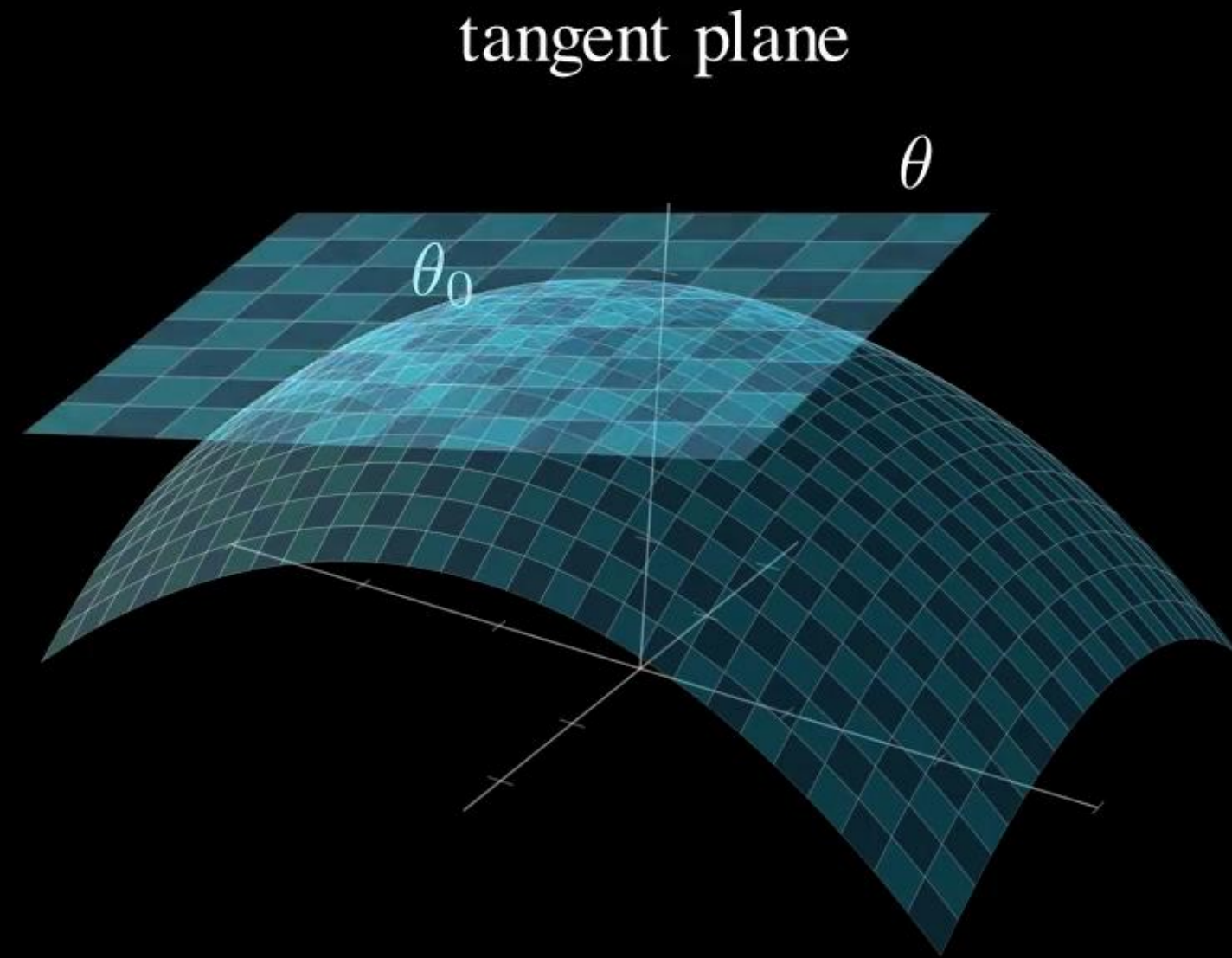
Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

So we linearize around a point:

$$f(\theta) \approx f(\theta_0) + \nabla_\theta f(\theta_0) (\theta - \theta_0)$$



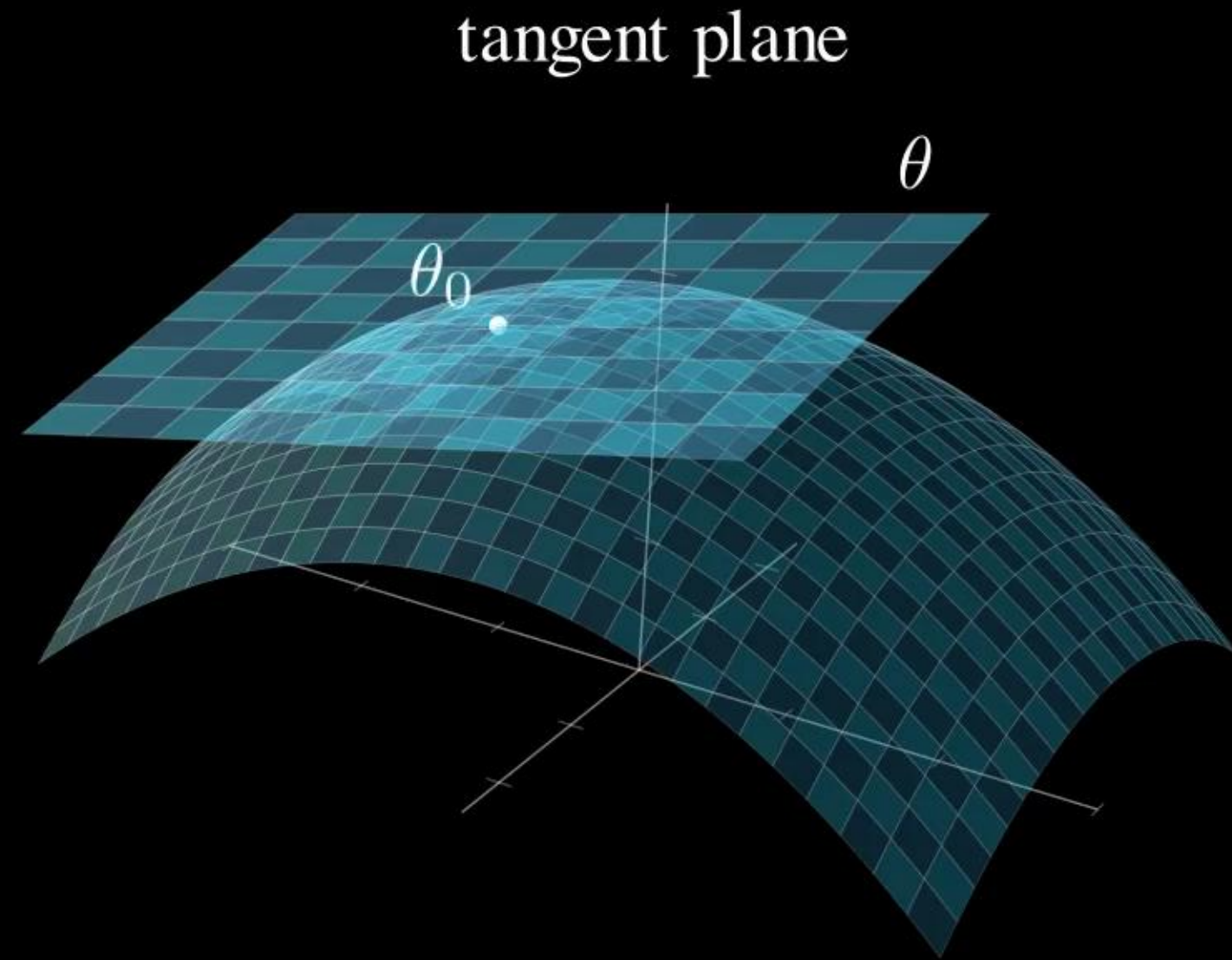
Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

So we linearize around a point:

$$f(\theta) \approx f(\theta_0) + \nabla_\theta f(\theta_0) (\theta - \theta_0)$$



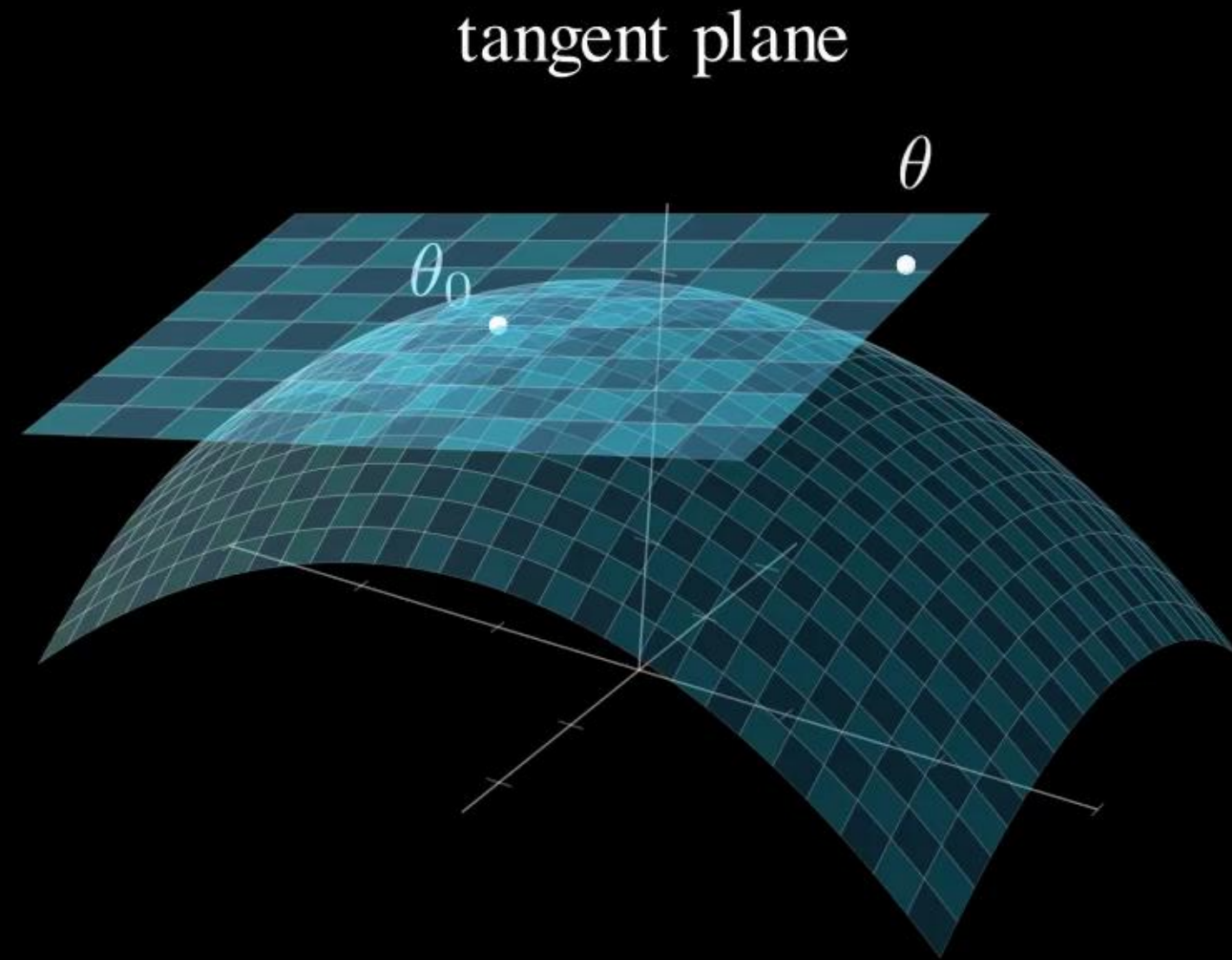
Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

So we linearize around a point:

$$f(\theta) \approx f(\theta_0) + \nabla_\theta f(\theta_0) (\theta - \theta_0)$$



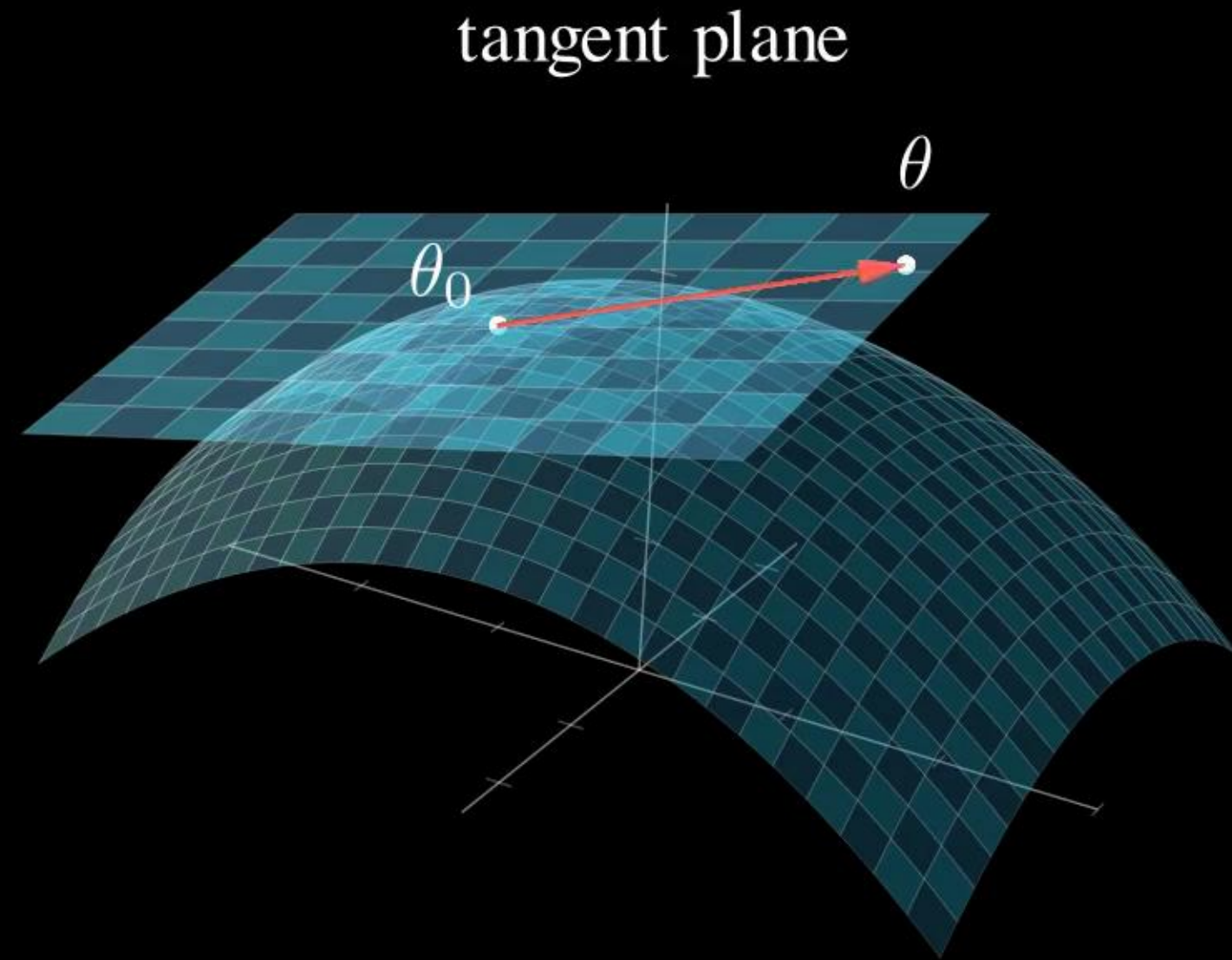
Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

So we linearize around a point:

$$f(\theta) \approx f(\theta_0) + \nabla_\theta f(\theta_0) (\theta - \theta_0)$$



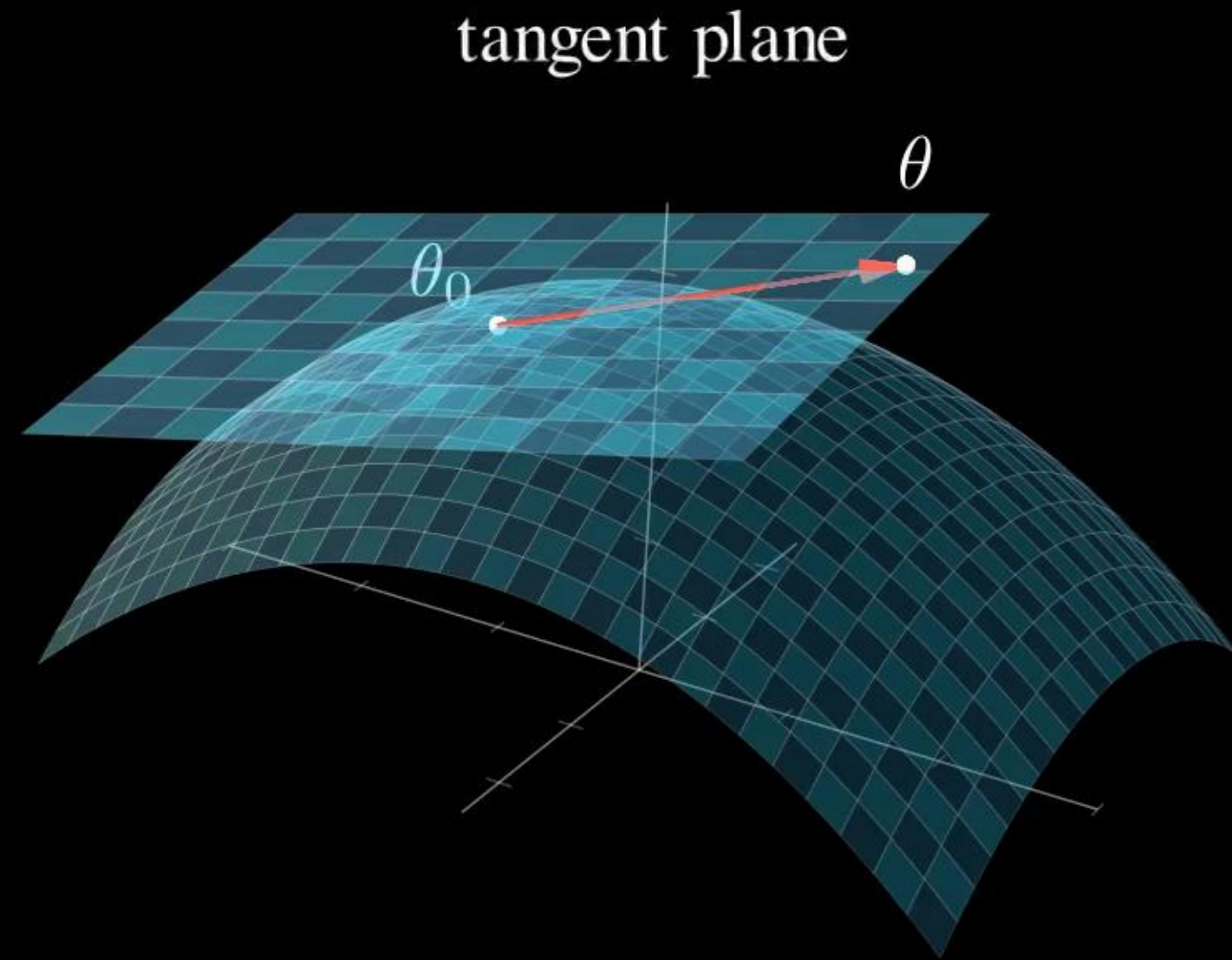
Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

So we linearize around a point:

$$f(\theta) \approx f(\theta_0) + \nabla_\theta f(\theta_0) (\theta - \theta_0)$$



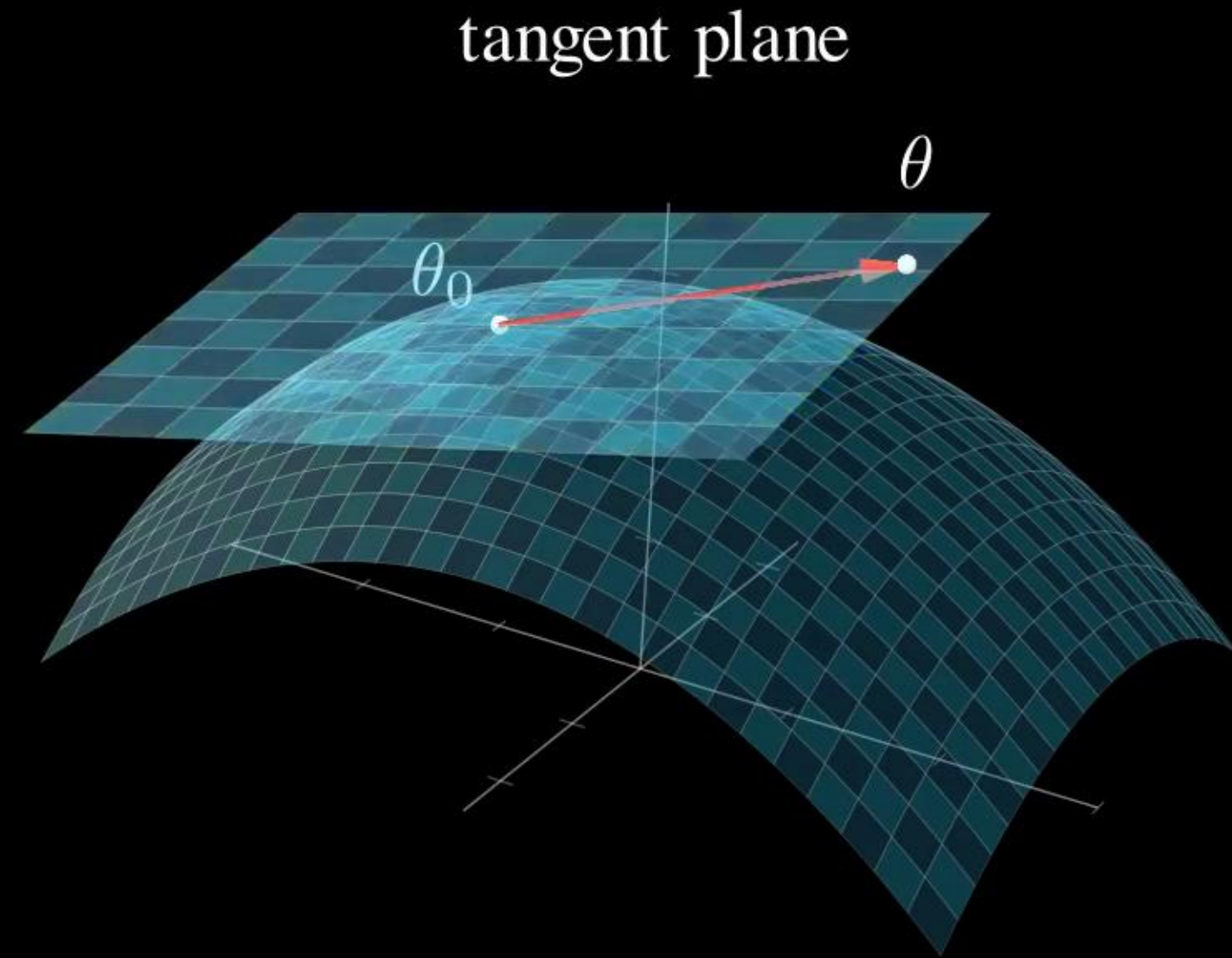
Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

So we linearize around a point:

$$f(\theta) \approx f(\theta_0) + \nabla_\theta f(\theta_0) (\theta - \theta_0)$$



Linearizing a function

The map

$$\theta \mapsto a \varphi(w^\top x), \quad \theta = (a, w)$$

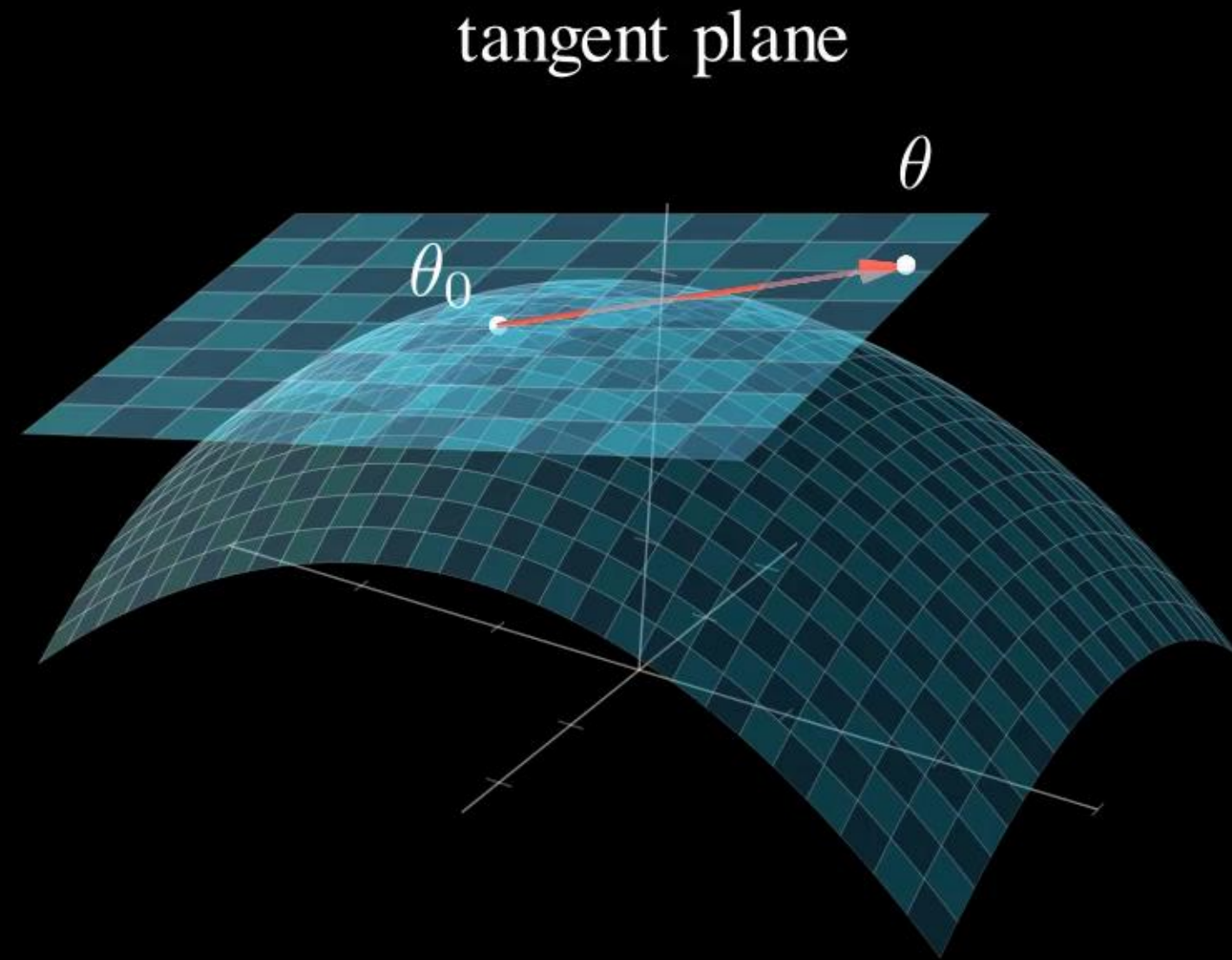
So we linearize around a point:

$$f(\theta) \approx f(\theta_0) + \nabla_\theta f(\theta_0) (\theta - \theta_0)$$

Then the residual dynamics become linear:

$$\dot{r}_t = -\Theta_t r_t$$

$$\mathbf{NTK} : \Theta_t(x, y) = \nabla_\theta f(x; \theta_t)^\top \nabla_\theta f(y; \theta_t)$$



Spectral bias

Spectral bias

- Constant NTK: $\dot{r} = -\Theta r$

Spectral bias

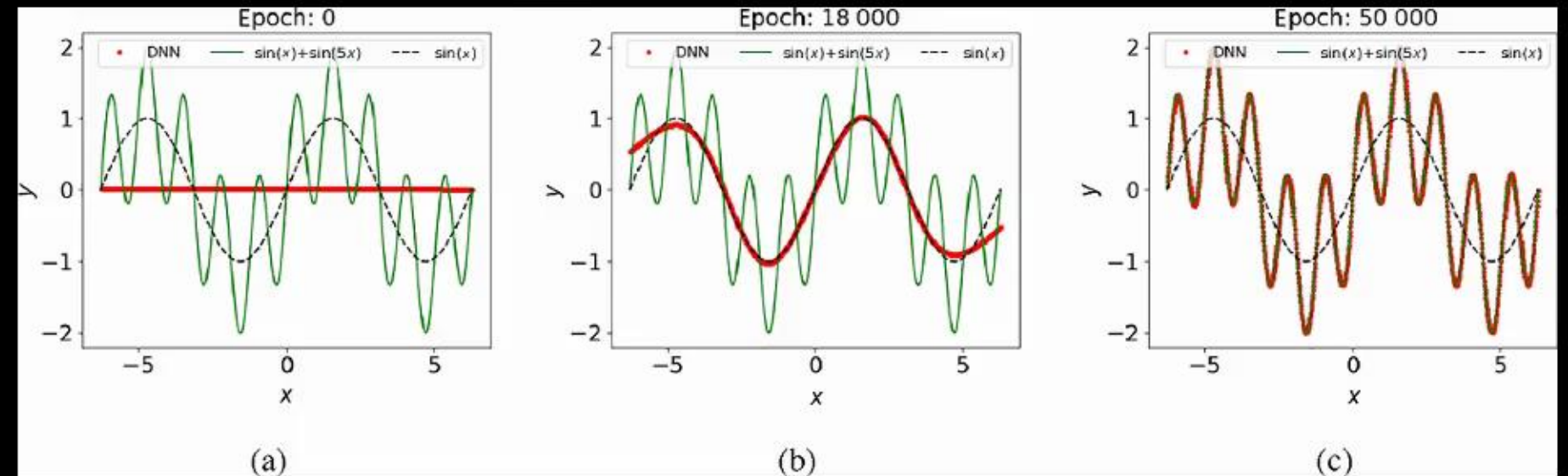
- Constant NTK: $\dot{r} = -\Theta r$
- **Spectrum** sets rates:
larger $\lambda \Rightarrow$ learned earlier

Spectral bias

- Constant NTK: $\dot{r} = -\Theta r$
- **Spectrum** sets rates:
larger $\lambda \Rightarrow$ learned earlier
- **Infinite width** isn't realistic,
so why care?

Spectral bias

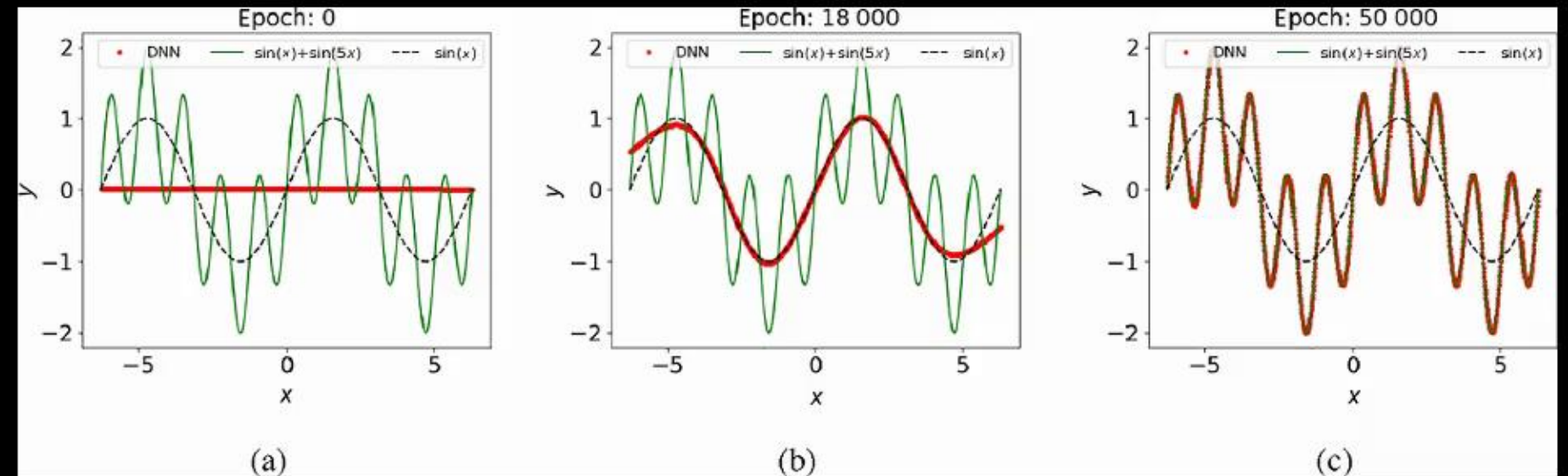
- Constant NTK: $\dot{r} = -\Theta r$
- **Spectrum** sets rates:
larger $\lambda \Rightarrow$ learned earlier
- **Infinite width** isn't realistic,
so why care?



Xu, Zhang & Luo (2022), Frequency Principle / Spectral Bias (overview).

Spectral bias

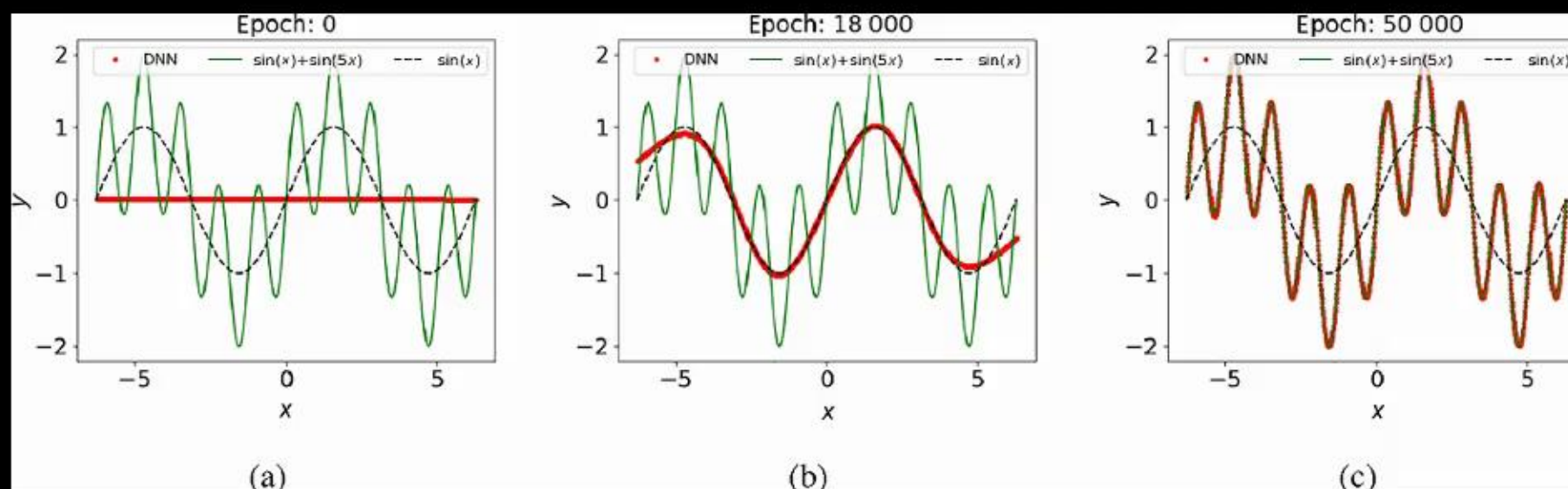
- Constant NTK: $\dot{r} = -\Theta r$
- **Spectrum** sets rates:
larger $\lambda \Rightarrow$ learned earlier
- **Infinite width** isn't realistic,
so why care?
- **Empirically** observed learning:
low freq $\rightarrow$ high freq (**frequency principle**)



Xu, Zhang & Luo (2022), Frequency Principle / Spectral Bias (overview).

Spectral bias

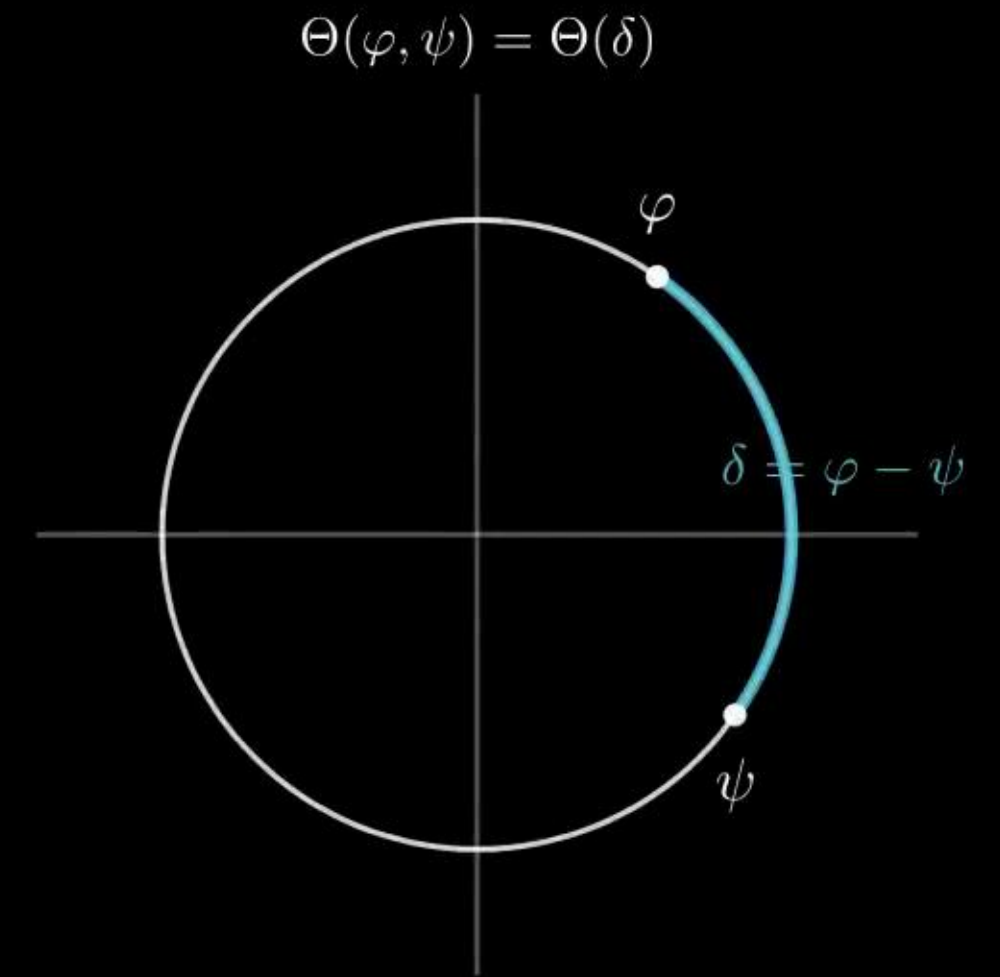
- Constant NTK: $\dot{r} = -\Theta r$
- **Spectrum** sets rates:
larger $\lambda \Rightarrow$ learned earlier
- **Infinite width** isn't realistic,
so why care?
- **Empirically** observed learning:
low freq $\rightarrow$ high freq (**frequency principle**)
- Beyond signals: **bias** persists without a frequency axis



Xu, Zhang & Luo (2022), Frequency Principle / Spectral Bias (overview).

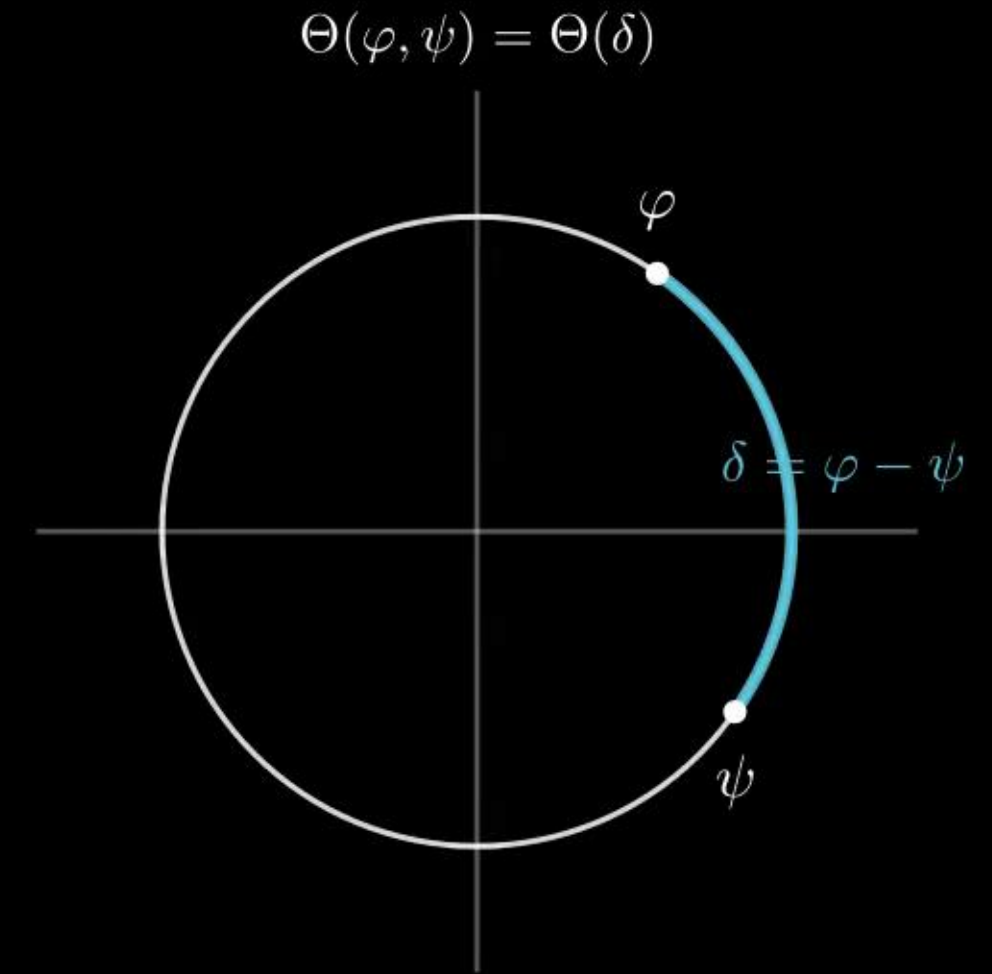
Frequency principle: Fourier diagonalization

Frequency principle: Fourier diagonalization



Frequency principle: Fourier diagonalization

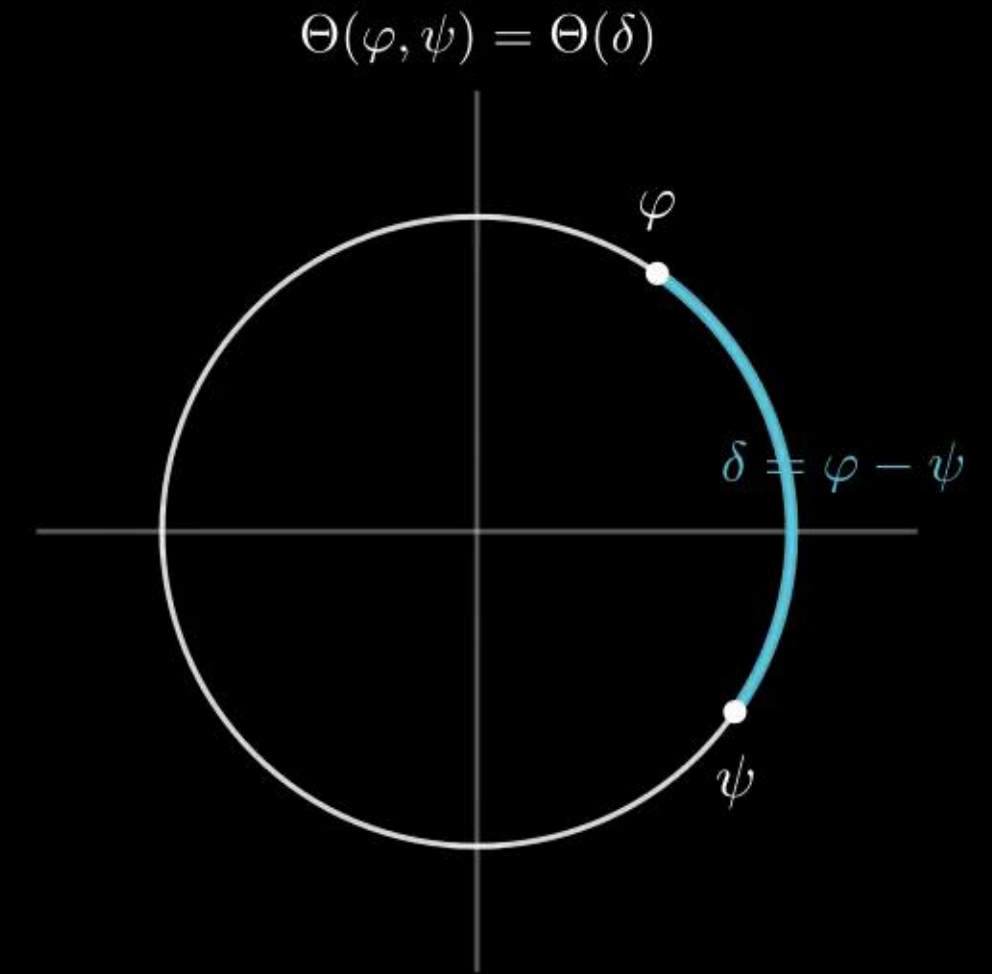
Setting: $\varphi \in S^1, \quad d\mu(\varphi) = \frac{d\varphi}{2\pi}$



Frequency principle: Fourier diagonalization

Setting: $\varphi \in S^1$, $d\mu(\varphi) = \frac{d\varphi}{2\pi}$

Translation-invariant NTK: $\Theta(x(\varphi), x(\psi)) = \Theta(\varphi - \psi) =: \Theta(\delta)$

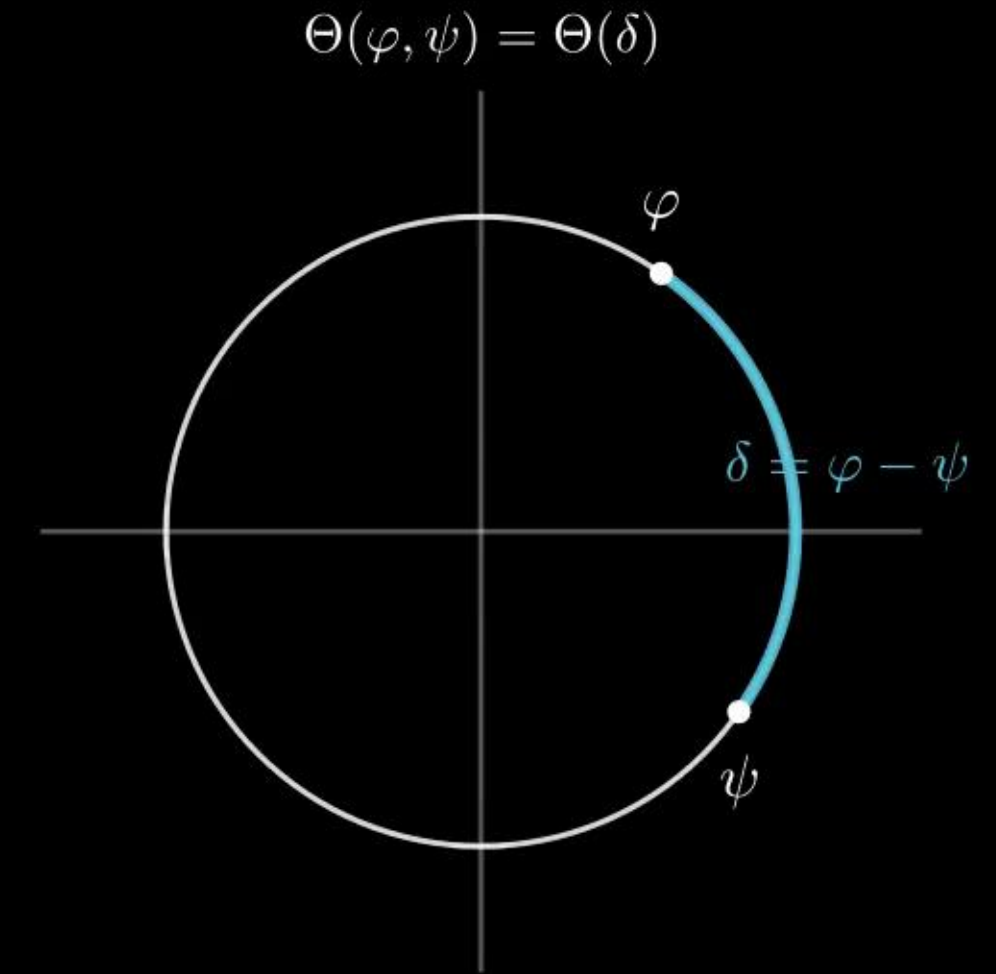


Frequency principle: Fourier diagonalization

Setting: $\varphi \in S^1$, $d\mu(\varphi) = \frac{d\varphi}{2\pi}$

Translation-invariant NTK: $\Theta(x(\varphi), x(\psi)) = \Theta(\varphi - \psi) =: \Theta(\delta)$

Convolution operator: $(T_{\Theta}f)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) f(\psi) d\mu(\psi)$



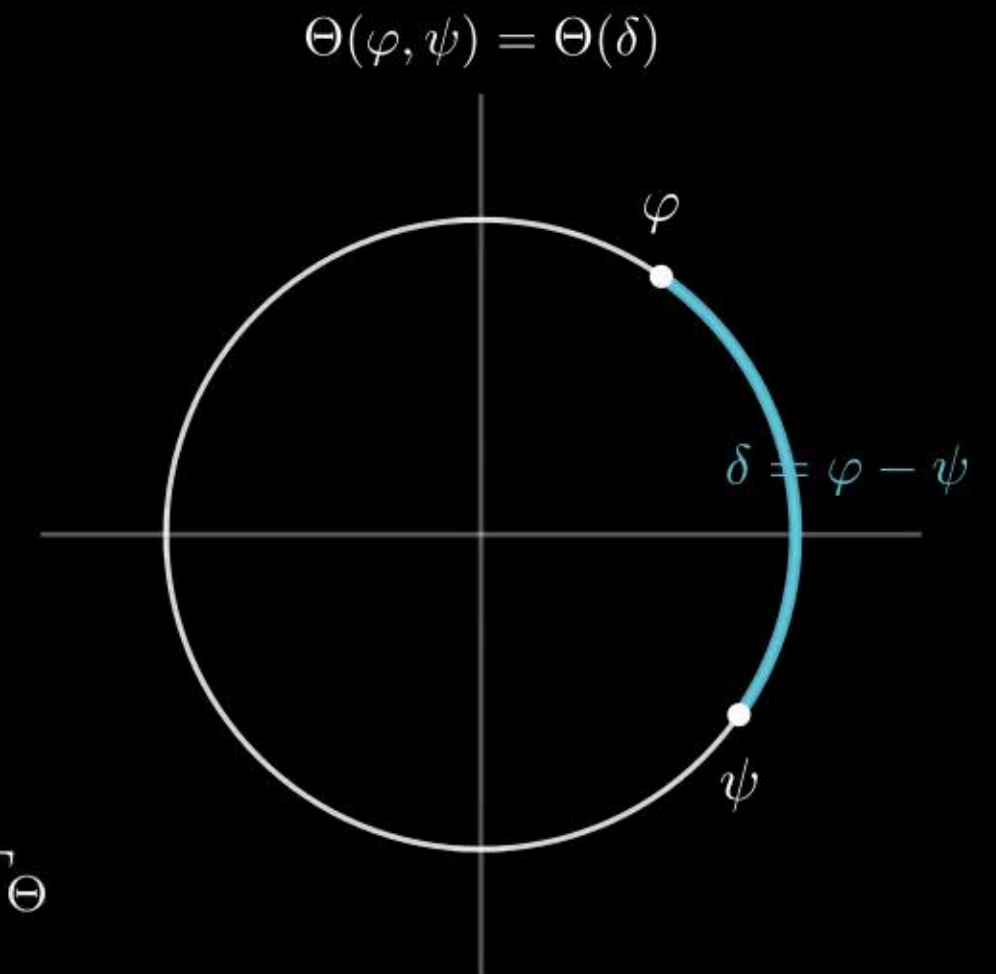
Frequency principle: Fourier diagonalization

Setting: $\varphi \in S^1, \quad d\mu(\varphi) = \frac{d\varphi}{2\pi}$

Translation-invariant NTK: $\Theta(x(\varphi), x(\psi)) = \Theta(\varphi - \psi) =: \Theta(\delta)$

Convolution operator: $(T_{\Theta}f)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) f(\psi) d\mu(\psi)$

Fourier diagonalization: $1, \cos(k\varphi), \sin(k\varphi)$ are eigenfunctions of T_{Θ}



Frequency principle: Fourier diagonalization

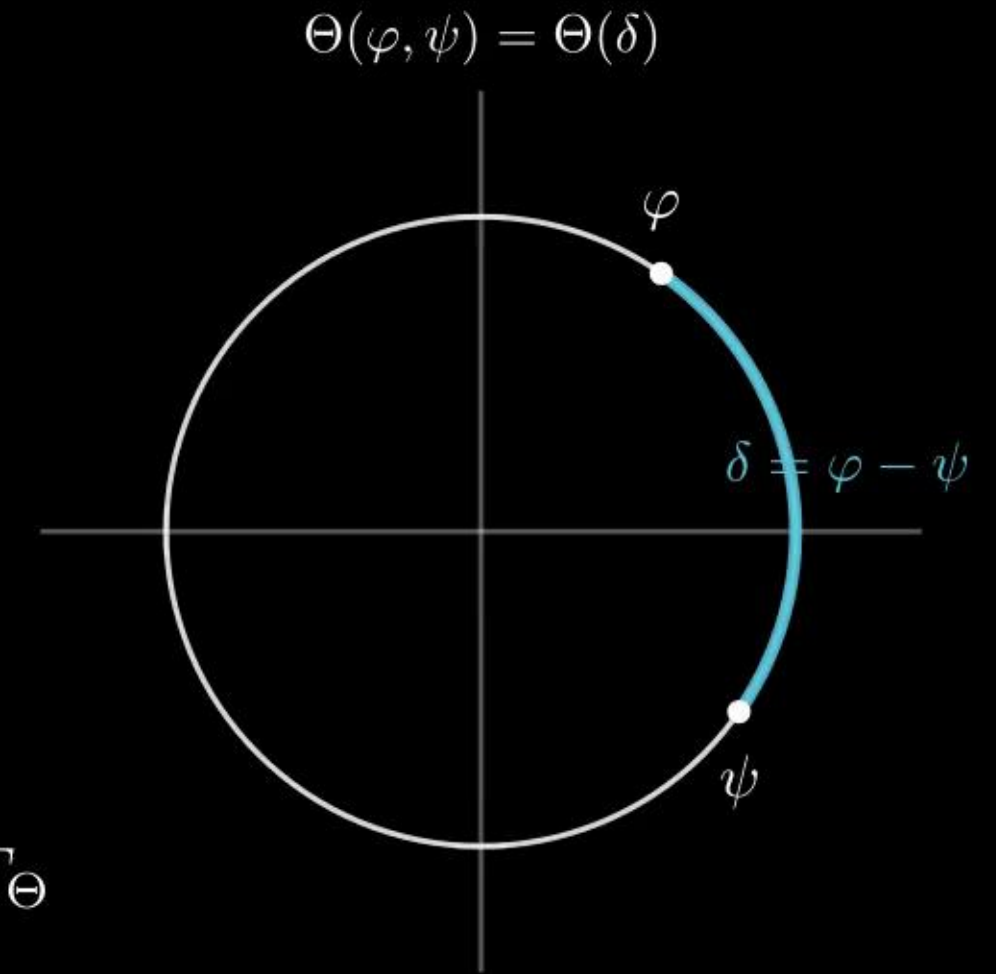
Setting: $\varphi \in S^1$, $d\mu(\varphi) = \frac{d\varphi}{2\pi}$

Translation-invariant NTK: $\Theta(x(\varphi), x(\psi)) = \Theta(\varphi - \psi) =: \Theta(\delta)$

Convolution operator: $(T_\Theta f)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) f(\psi) d\mu(\psi)$

Fourier diagonalization: $1, \cos(k\varphi), \sin(k\varphi)$ are eigenfunctions of T_Θ

Eigenvalues: $\lambda_k = \int_0^{2\pi} \Theta(\delta) \cos(k\delta) d\mu(\delta)$



Frequency principle: Fourier diagonalization

Setting: $\varphi \in S^1$, $d\mu(\varphi) = \frac{d\varphi}{2\pi}$

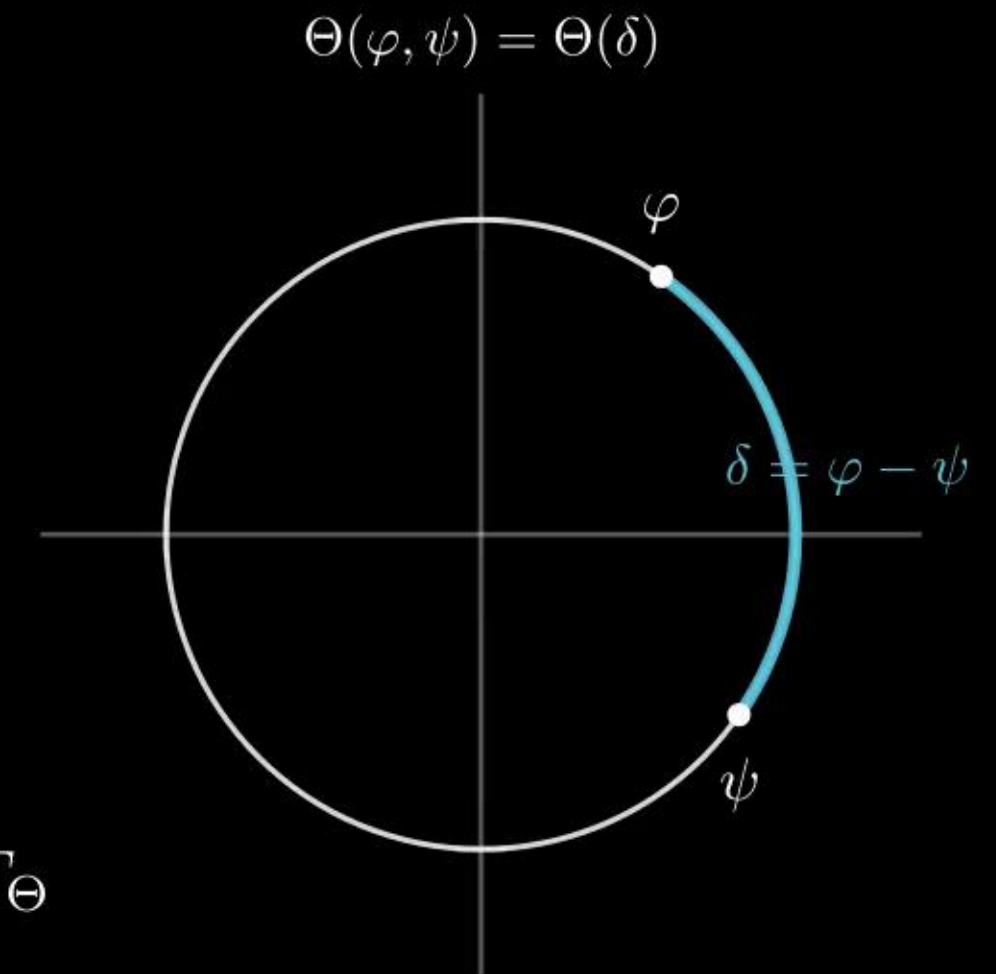
Translation-invariant NTK: $\Theta(x(\varphi), x(\psi)) = \Theta(\varphi - \psi) =: \Theta(\delta)$

Convolution operator: $(T_\Theta f)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) f(\psi) d\mu(\psi)$

Fourier diagonalization: $1, \cos(k\varphi), \sin(k\varphi)$ are eigenfunctions of T_Θ

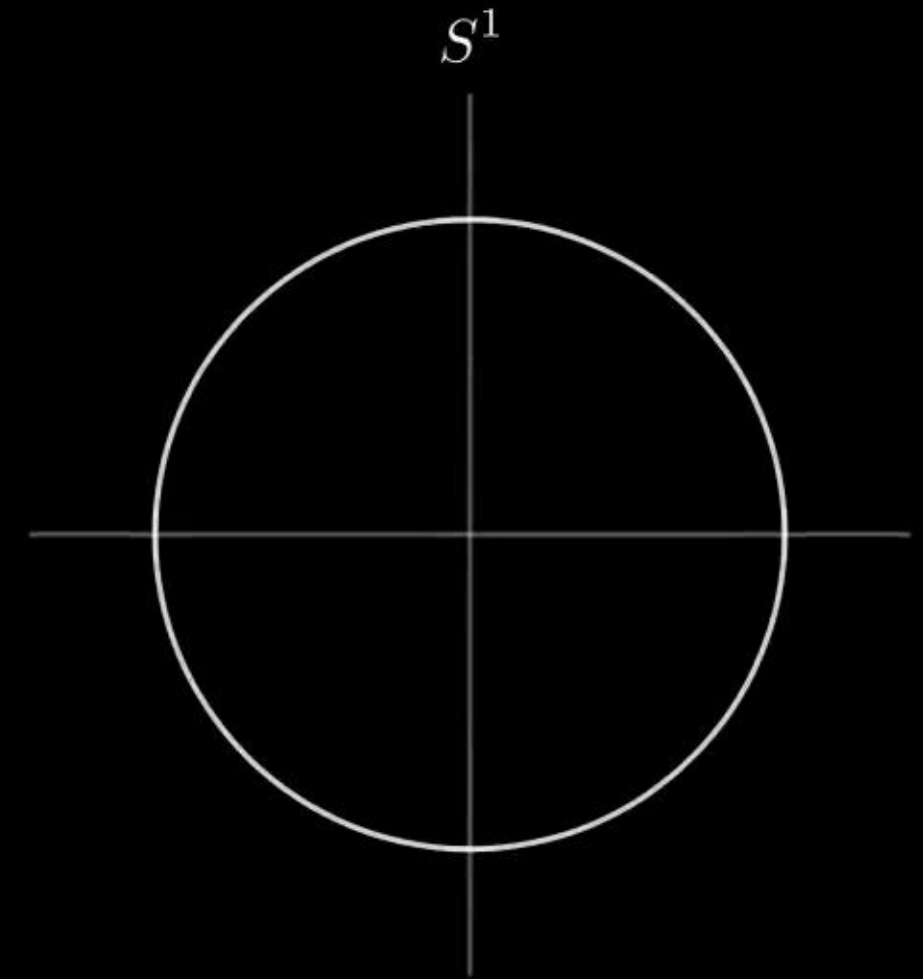
Eigenvalues: $\lambda_k = \int_0^{2\pi} \Theta(\delta) \cos(k\delta) d\mu(\delta)$

Mode-wise decay: $\dot{r} = -T_\Theta r \Rightarrow r_k(t) = e^{-\lambda_k t} r_k(0)$



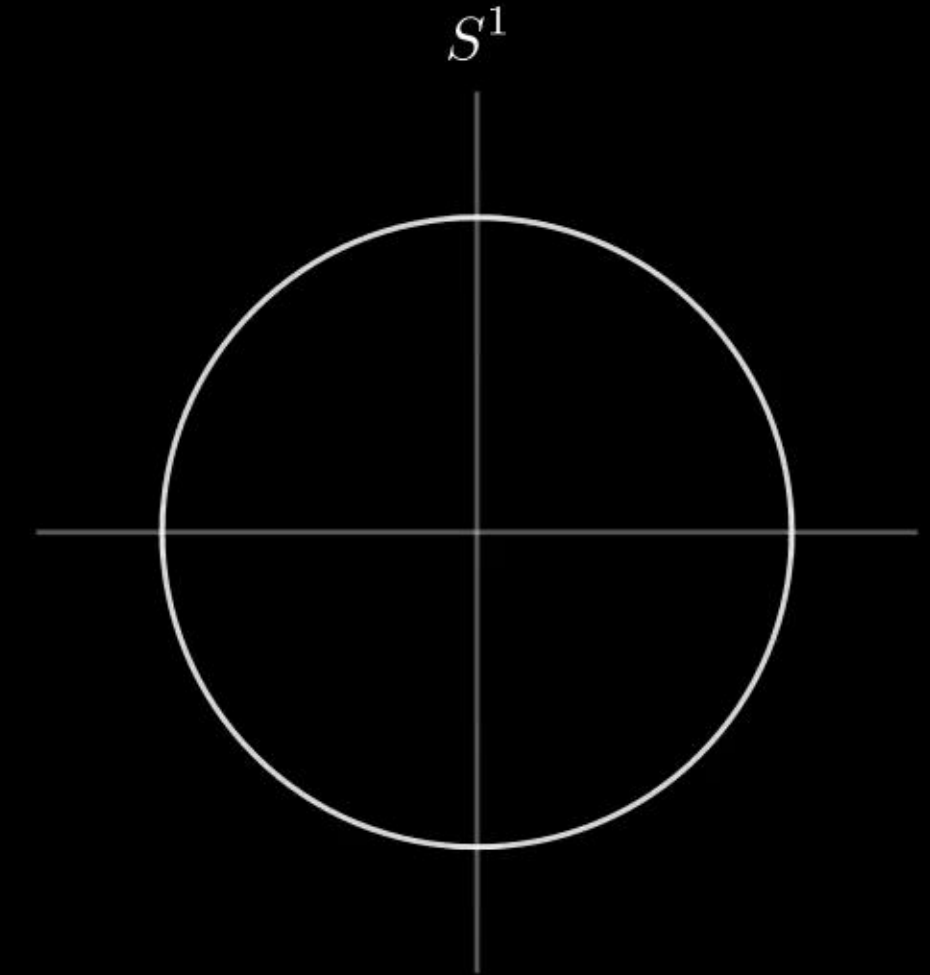
Finite samples: discretizing the circle

Finite samples: discretizing the circle



Finite samples: discretizing the circle

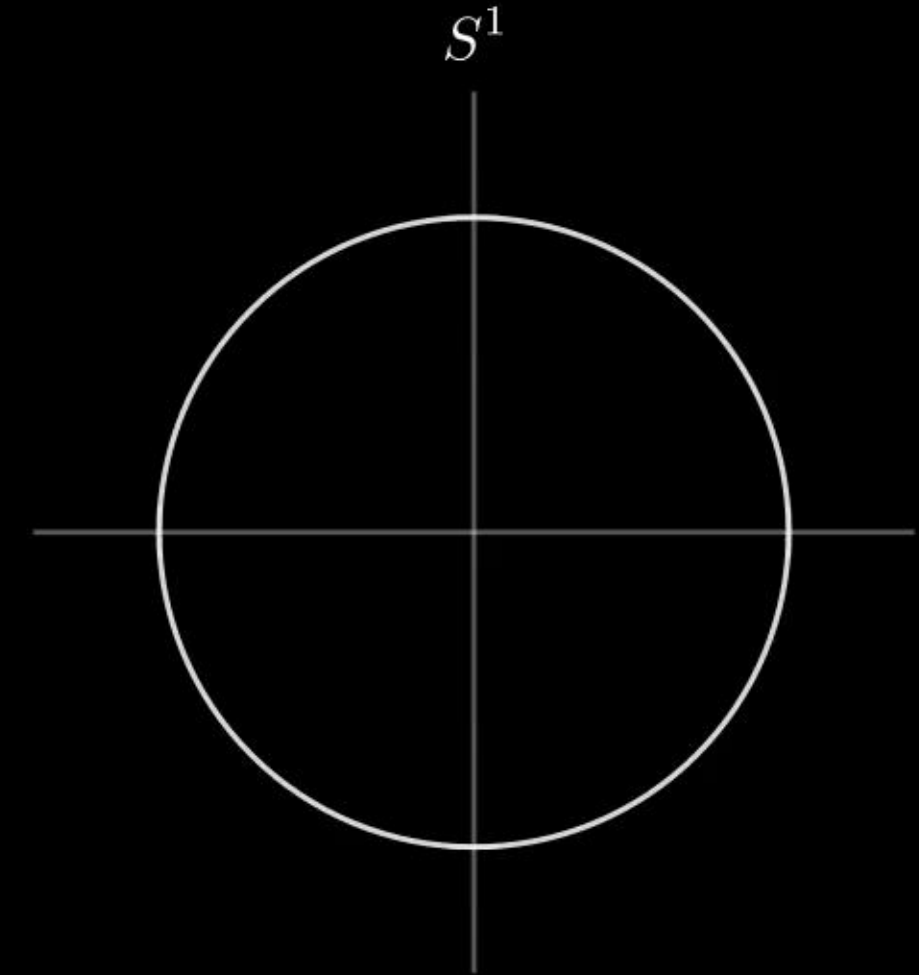
Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.



Finite samples: discretizing the circle

Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.

Discretize: Keep infinite width, but replace the continuum with uniformly sampled points.

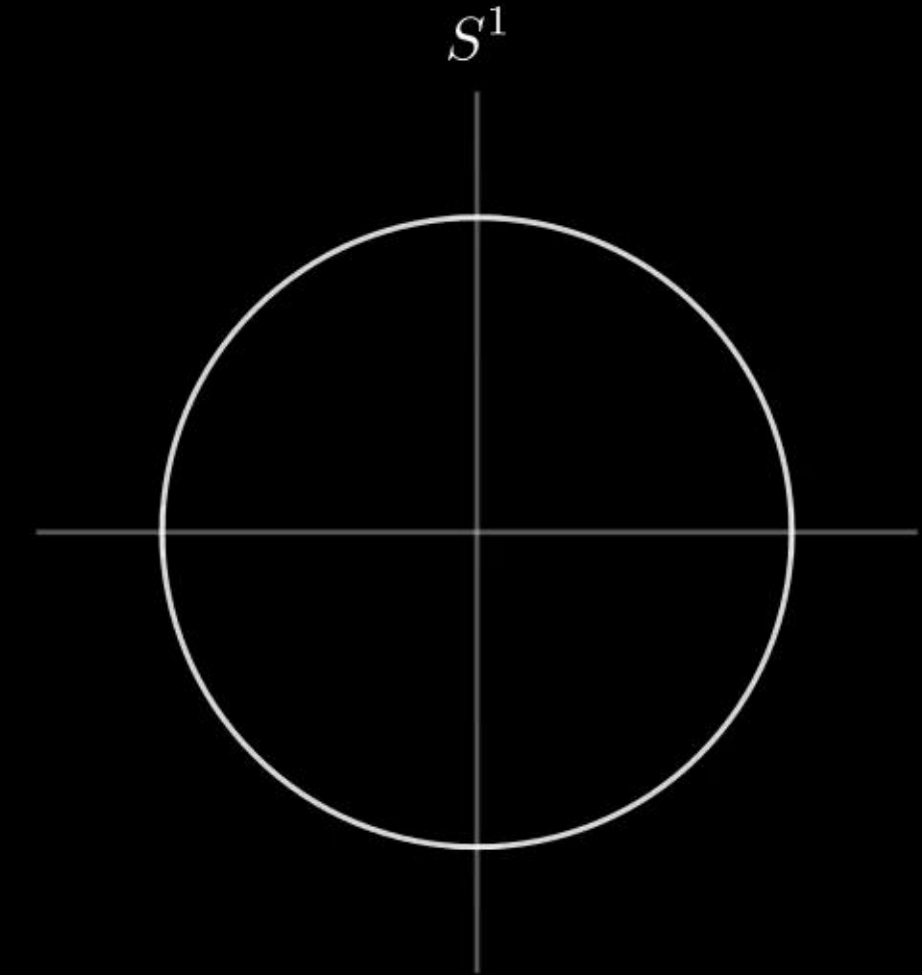


Finite samples: discretizing the circle

Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.

Discretize: Keep infinite width, but replace the continuum with uniformly sampled points.

Sampling: $\varphi_1, \dots, \varphi_n \sim \text{Unif}[0, 2\pi)$

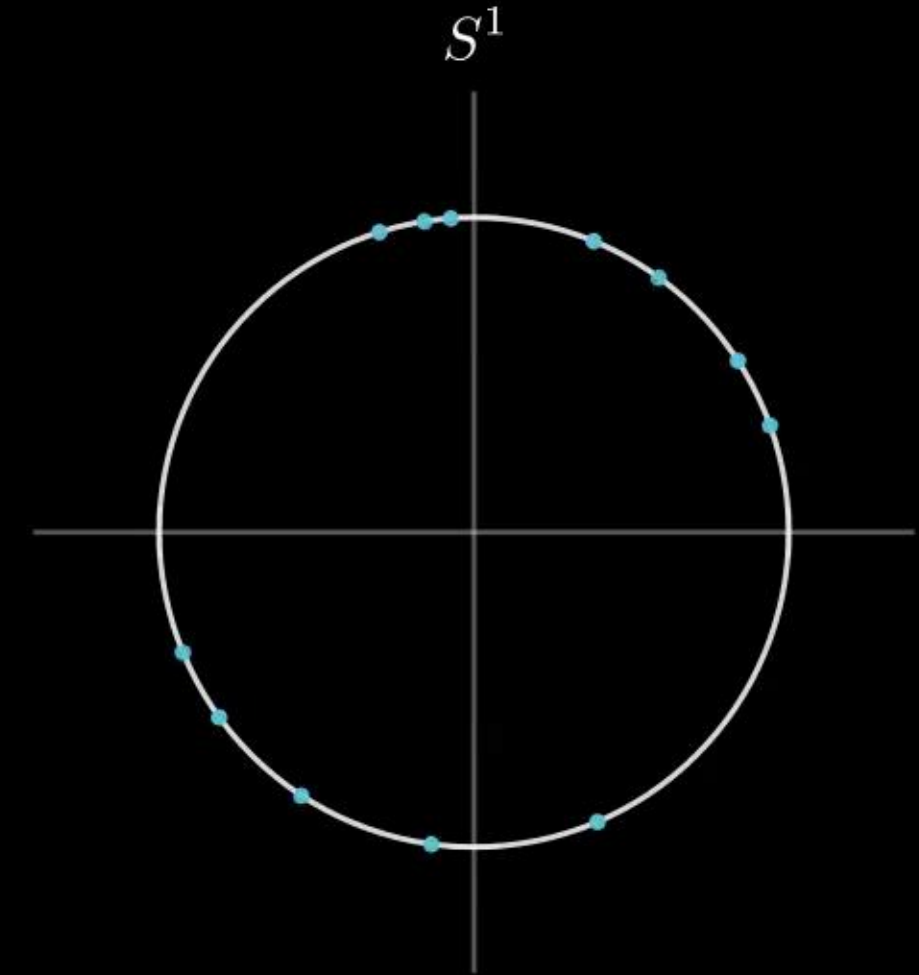


Finite samples: discretizing the circle

Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.

Discretize: Keep infinite width, but replace the continuum with uniformly sampled points.

Sampling: $\varphi_1, \dots, \varphi_n \sim \text{Unif}[0, 2\pi)$

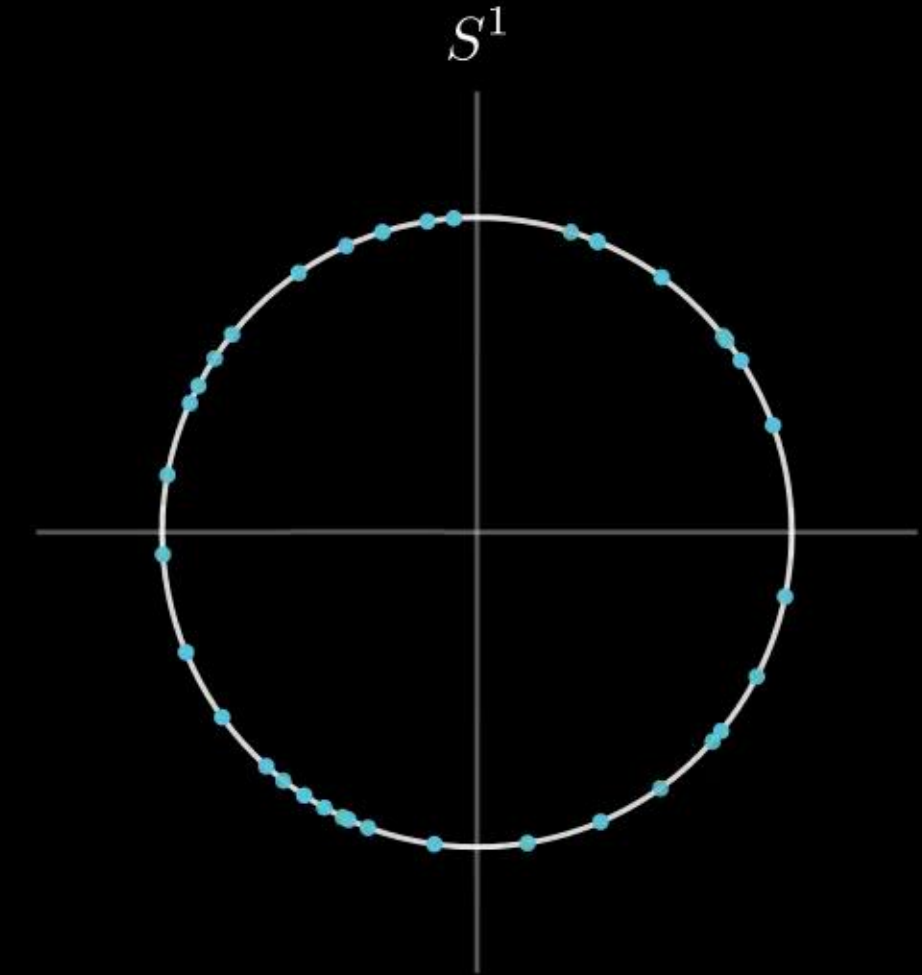


Finite samples: discretizing the circle

Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.

Discretize: Keep infinite width, but replace the continuum with uniformly sampled points.

Sampling: $\varphi_1, \dots, \varphi_n \sim \text{Unif}[0, 2\pi)$



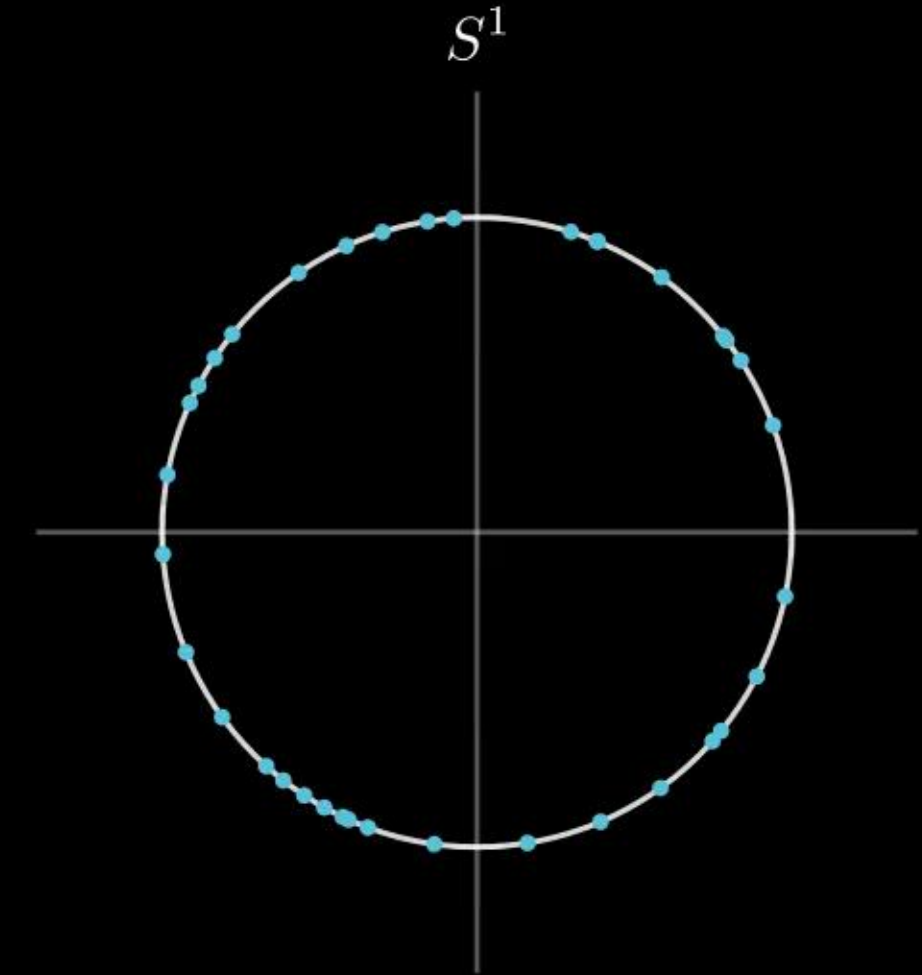
Finite samples: discretizing the circle

Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.

Discretize: Keep infinite width, but replace the continuum with uniformly sampled points.

Sampling: $\varphi_1, \dots, \varphi_n \sim \text{Unif}[0, 2\pi)$

Feature matrix: $\Phi_{i,p} = \varphi_p(\phi_i), \quad p \in \{0, (k, c), (k, s)\}$



Finite samples: discretizing the circle

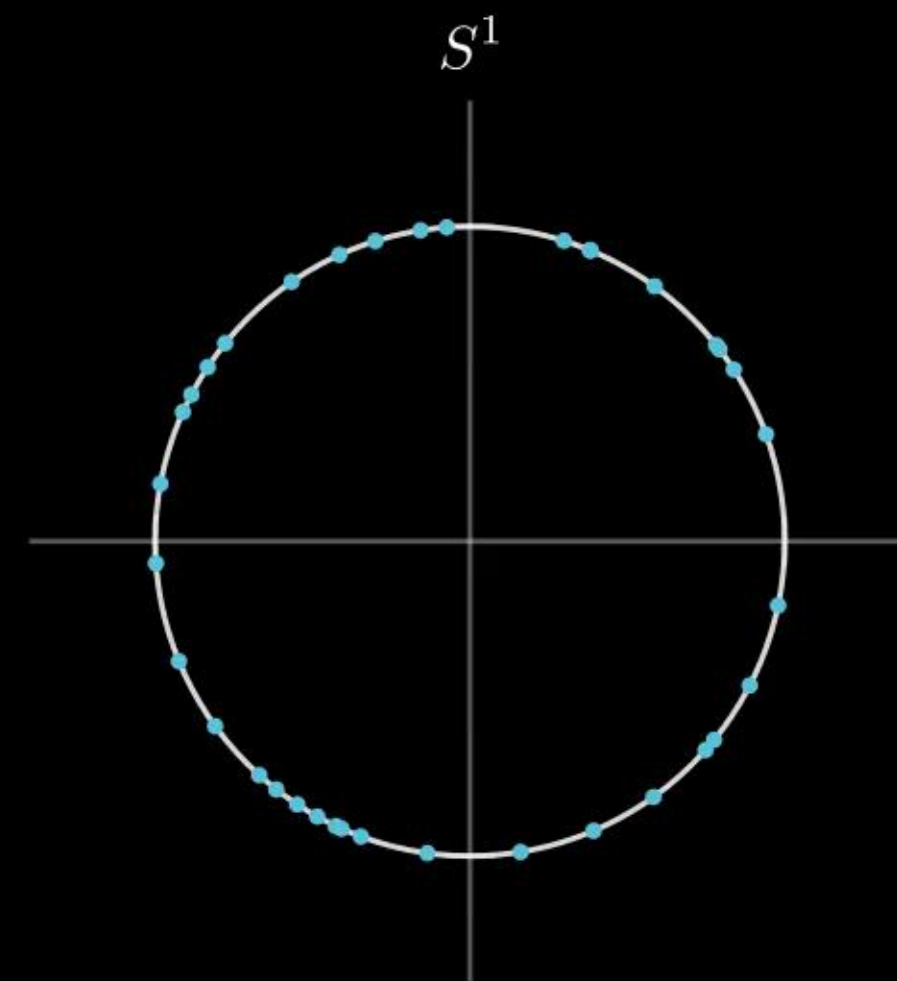
Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.

Discretize: Keep infinite width, but replace the continuum with uniformly sampled points.

Sampling: $\varphi_1, \dots, \varphi_n \sim \text{Unif}[0, 2\pi)$

Feature matrix: $\Phi_{i,p} = \varphi_p(\phi_i), \quad p \in \{0, (k, c), (k, s)\}$

$$\varphi_0(\phi) = 1, \quad \varphi_{k,c}(\phi) = \sqrt{2} \cos(k\phi), \quad \varphi_{k,s}(\phi) = \sqrt{2} \sin(k\phi), \quad k = 1, \dots, K.$$



Finite samples: discretizing the circle

Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.

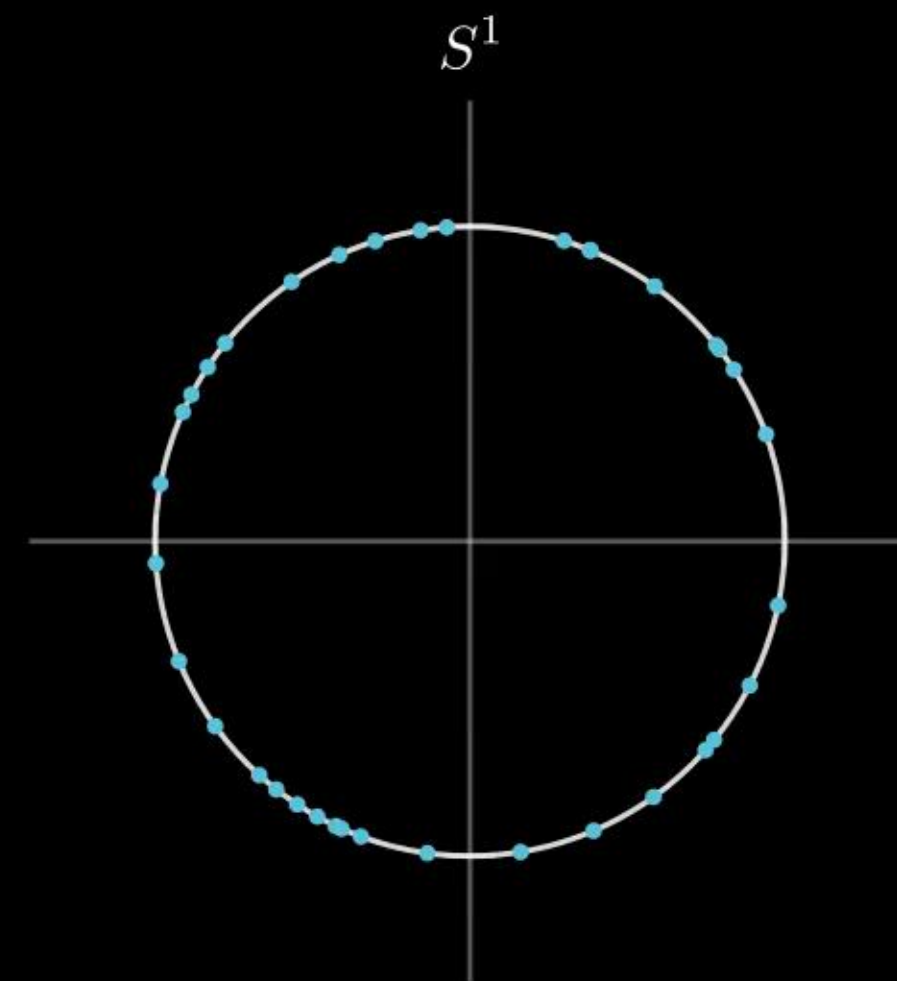
Discretize: Keep infinite width, but replace the continuum with uniformly sampled points.

Sampling: $\varphi_1, \dots, \varphi_n \sim \text{Unif}[0, 2\pi)$

Feature matrix: $\Phi_{i,p} = \varphi_p(\phi_i), \quad p \in \{0, (k, c), (k, s)\}$

$$\varphi_0(\phi) = 1, \quad \varphi_{k,c}(\phi) = \sqrt{2} \cos(k\phi), \quad \varphi_{k,s}(\phi) = \sqrt{2} \sin(k\phi), \quad k = 1, \dots, K.$$

Kernel: $M_{i,j} = \Theta(\varphi_i - \varphi_j)$ **Operator:** $A = \frac{1}{n} M$



Finite samples: discretizing the circle

Issue: Earlier picture assumes infinite width and training on the continuum $[0, 2\pi)$.

Discretize: Keep infinite width, but replace the continuum with uniformly sampled points.

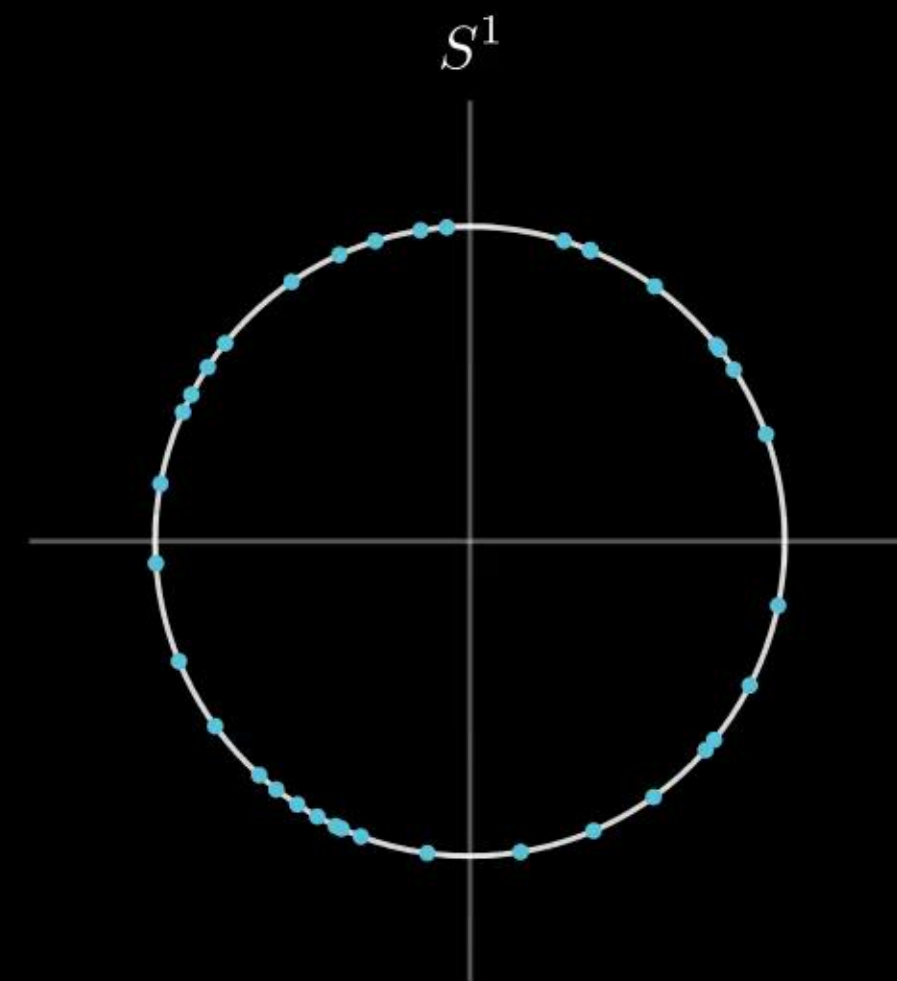
Sampling: $\varphi_1, \dots, \varphi_n \sim \text{Unif}[0, 2\pi)$

Feature matrix: $\Phi_{i,p} = \varphi_p(\phi_i), \quad p \in \{0, (k, c), (k, s)\}$

$$\varphi_0(\phi) = 1, \quad \varphi_{k,c}(\phi) = \sqrt{2} \cos(k\phi), \quad \varphi_{k,s}(\phi) = \sqrt{2} \sin(k\phi), \quad k = 1, \dots, K.$$

Kernel: $M_{i,j} = \Theta(\varphi_i - \varphi_j)$ **Operator:** $A = \frac{1}{n} M$

Goal: the low-frequency Fourier block still almost diagonalizes the sampled operator.



Finite samples: concentration

Finite samples: concentration

Notation: $d = 2K + 1, \quad \kappa = \sup_{\delta \in [0, 2\pi)} |\Theta(\delta)|$

Finite samples: concentration

Notation: $d = 2K + 1$, $\kappa = \sup_{\delta \in [0, 2\pi)} |\Theta(\delta)|$

Lemma A: $\mathbb{E}[G] = I_d$

$$\mathbb{P}(\|G - I_d\|_{\max} \geq \varepsilon) \leq 2d^2 \exp\left(-\frac{n\varepsilon^2}{8}\right)$$

Finite samples: concentration

Notation: $d = 2K + 1$, $\kappa = \sup_{\delta \in [0, 2\pi)} |\Theta(\delta)|$

Lemma A: Sampled Fourier features are almost orthonormal ($G \approx I$).

Finite samples: concentration

Notation: $d = 2K + 1$, $\kappa = \sup_{\delta \in [0, 2\pi)} |\Theta(\delta)|$

Lemma A: Sampled Fourier features are almost orthonormal ($G \approx I$).

Lemma B:

$$H = \frac{1}{n} \Phi^\top A \Phi$$
$$\lambda_p^{(n)} = \left(1 - \frac{1}{n}\right) \lambda_p + \frac{1}{n} \Theta(0), \quad \mathbb{E}[H] = \Lambda^{(n)}$$
$$\mathbb{P}(\|H - \mathbb{E}H\|_{\max} \geq \varepsilon) \leq 2d^2 \exp\left(-\frac{n\varepsilon^2}{32\kappa^2}\right)$$

Finite samples: concentration

Notation: $d = 2K + 1$, $\kappa = \sup_{\delta \in [0, 2\pi)} |\Theta(\delta)|$

Lemma A: Sampled Fourier features are almost orthonormal ($G \approx I$).

Lemma B: The compressed operator is almost diagonal on low modes ($H \approx \Lambda$).

Finite samples: concentration

Notation: $d = 2K + 1$, $\kappa = \sup_{\delta \in [0, 2\pi)} |\Theta(\delta)|$

Lemma A: Sampled Fourier features are almost orthonormal ($G \approx I$).

Lemma B: The compressed operator is almost diagonal on low modes ($H \approx \Lambda$).

Lemma C:
$$\mathbb{P}\left(\|A\Phi - \Phi\Lambda^{(n)}\|_{\max} \geq \varepsilon\right) \leq 2nd \exp\left(-\frac{n\varepsilon^2}{4\kappa^2}\right), \quad (n \geq 2)$$

Finite samples: concentration

Notation: $d = 2K + 1$, $\kappa = \sup_{\delta \in [0, 2\pi)} |\Theta(\delta)|$

Lemma A: Sampled Fourier features are almost orthonormal ($G \approx I$).

Lemma B: The compressed operator is almost diagonal on low modes ($H \approx \Lambda$).

Lemma C: Low Fourier modes act like eigenvectors ($A\Phi \approx \Phi\Lambda$), uniformly).

Finite samples: concentration

Notation: $d = 2K + 1$, $\kappa = \sup_{\delta \in [0, 2\pi)} |\Theta(\delta)|$

Lemma A: Sampled Fourier features are almost orthonormal ($G \approx I$).

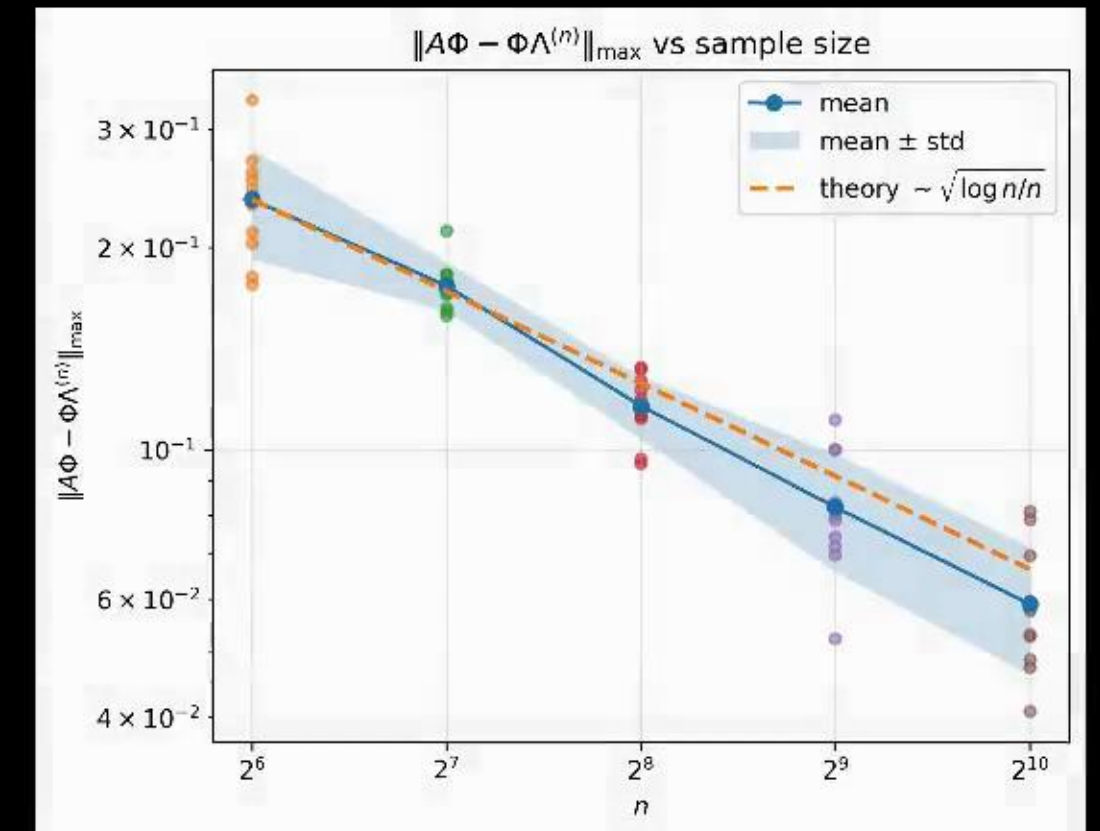
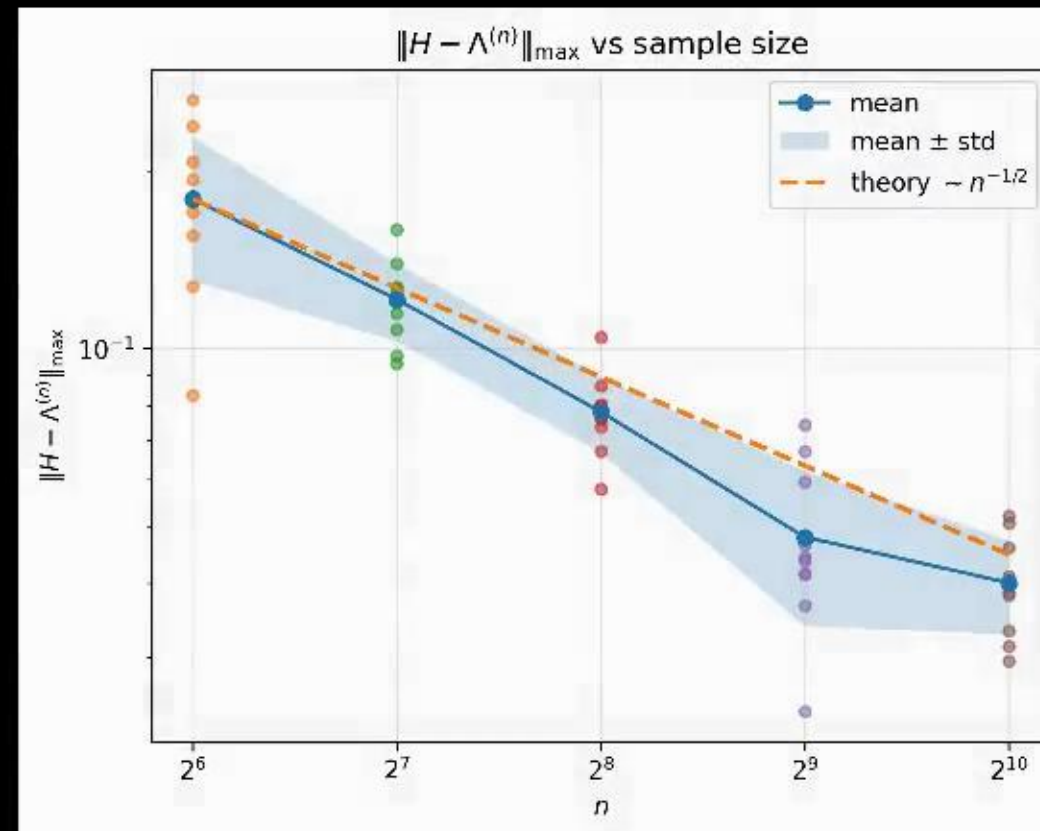
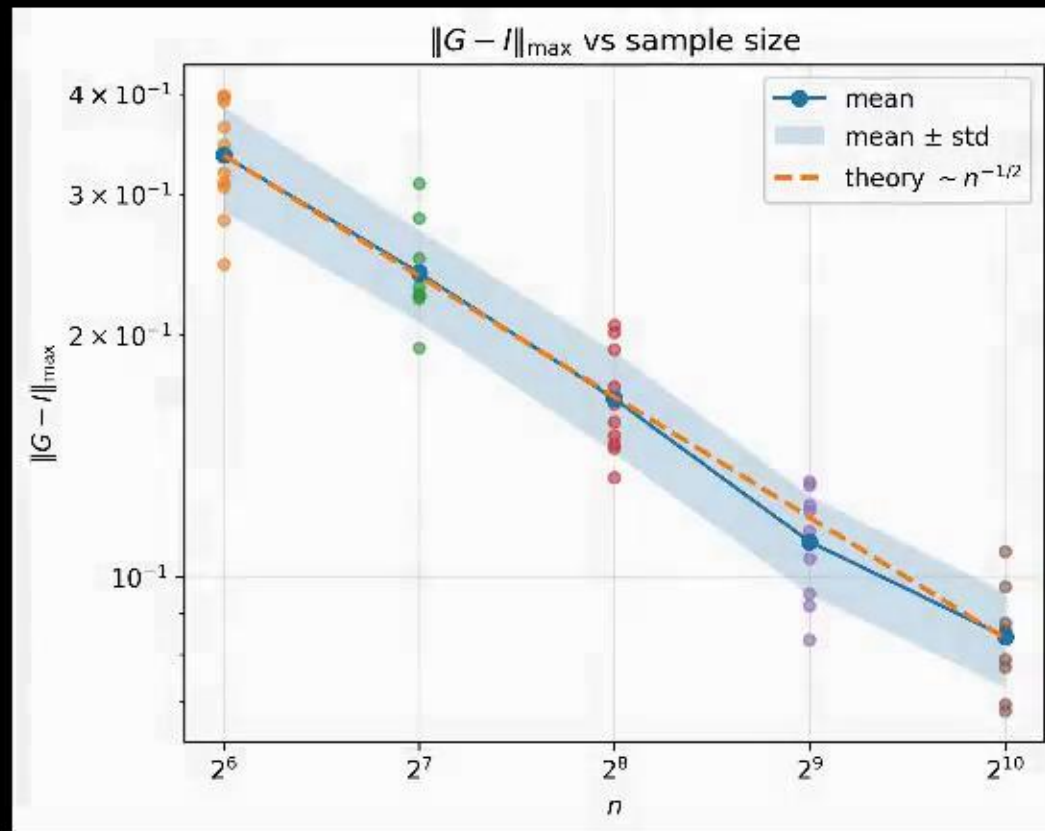
Lemma B: The compressed operator is almost diagonal on low modes ($H \approx \Lambda$).

Lemma C: Low Fourier modes act like eigenvectors ($A\Phi \approx \Phi\Lambda$), uniformly).

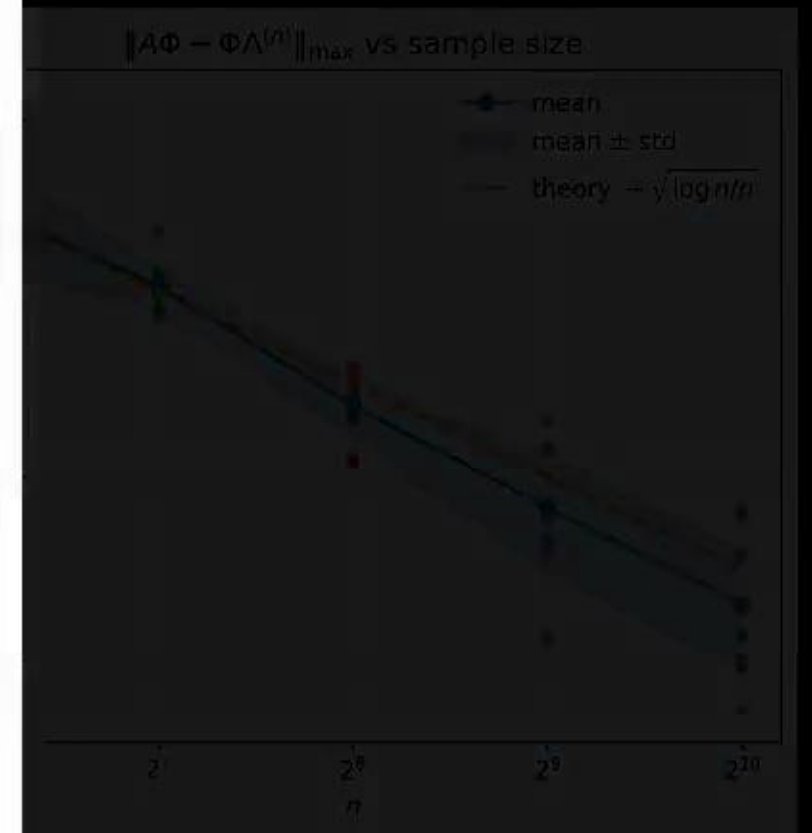
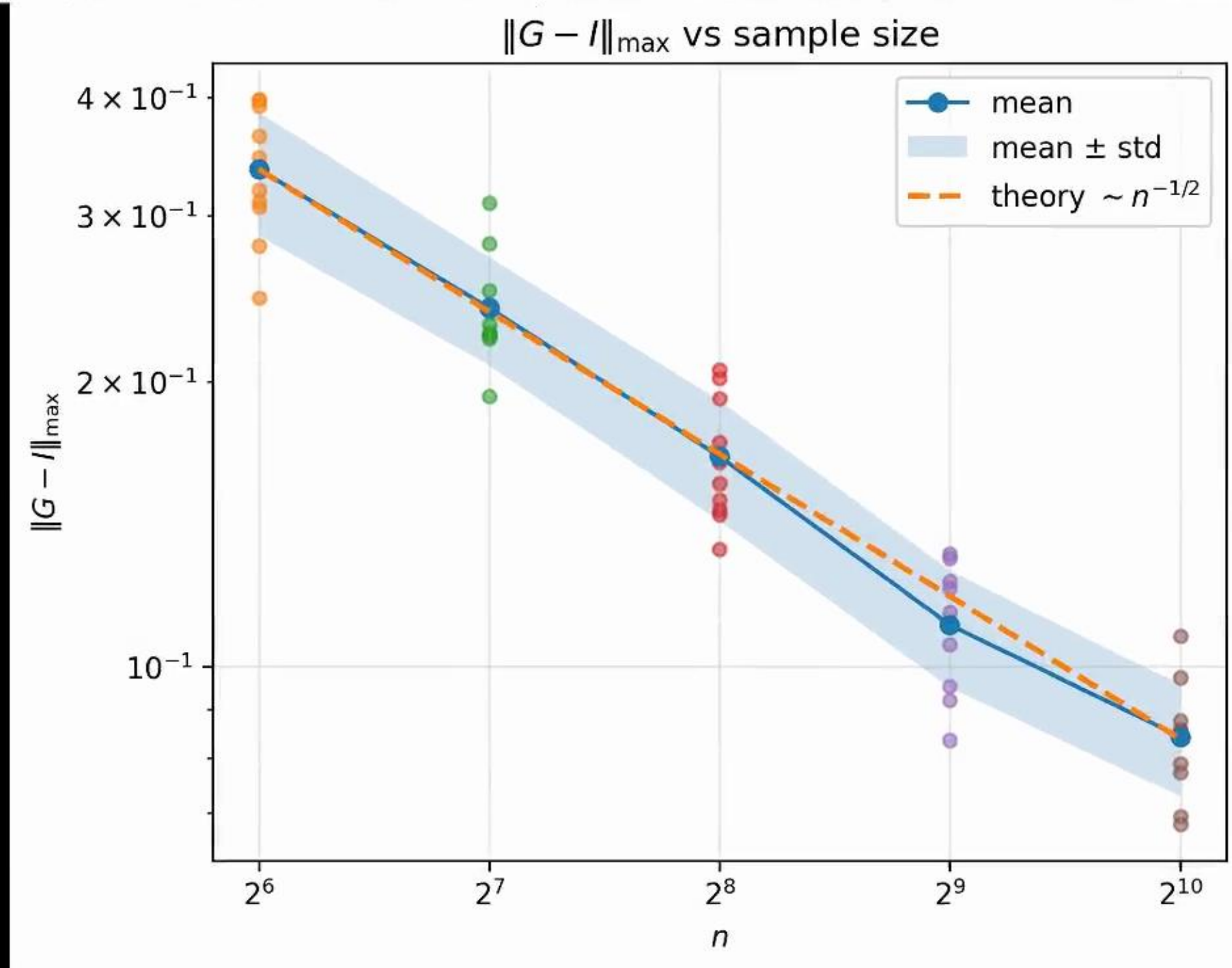
Low-mode diagonalization and decay rates survive finite sampling.

Empirical results: concentration with sample size

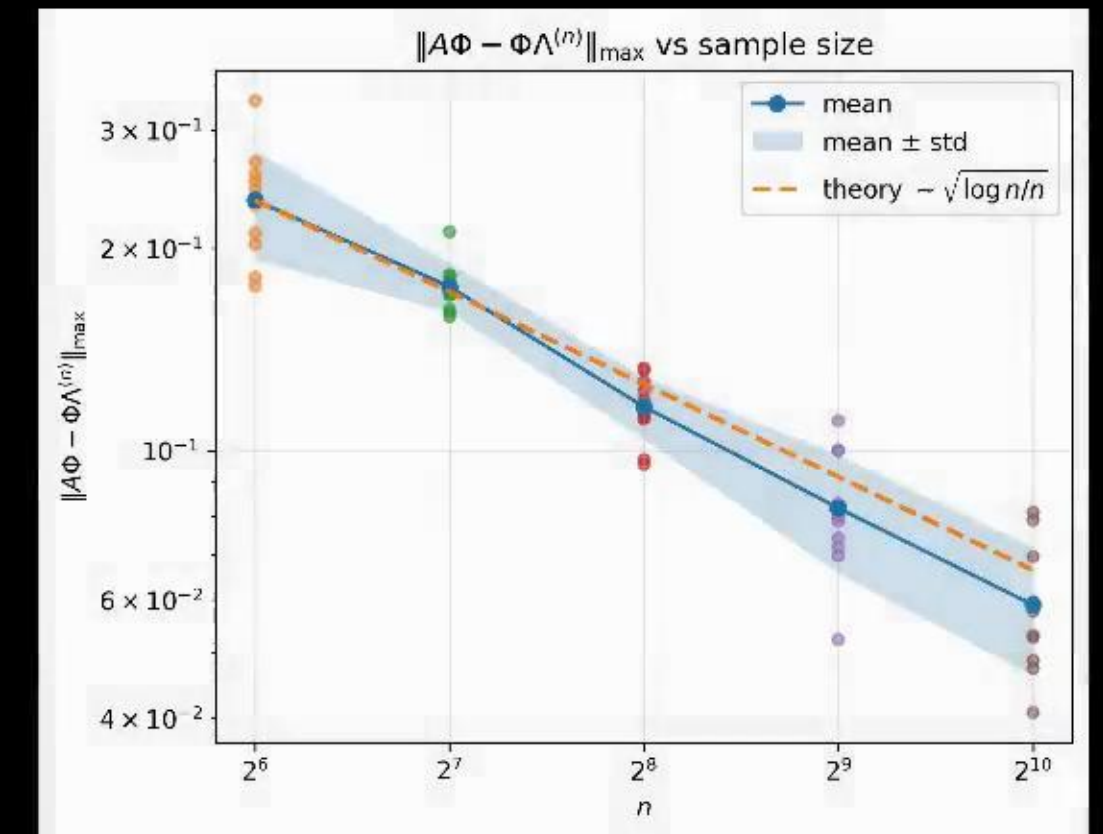
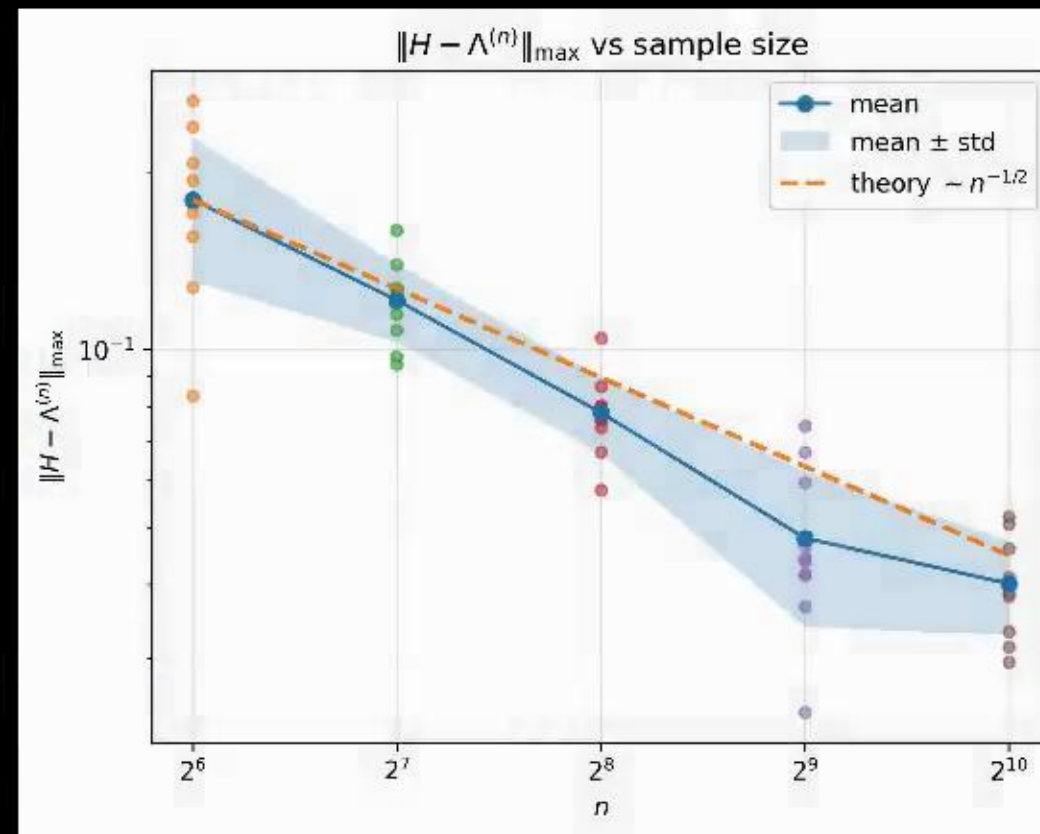
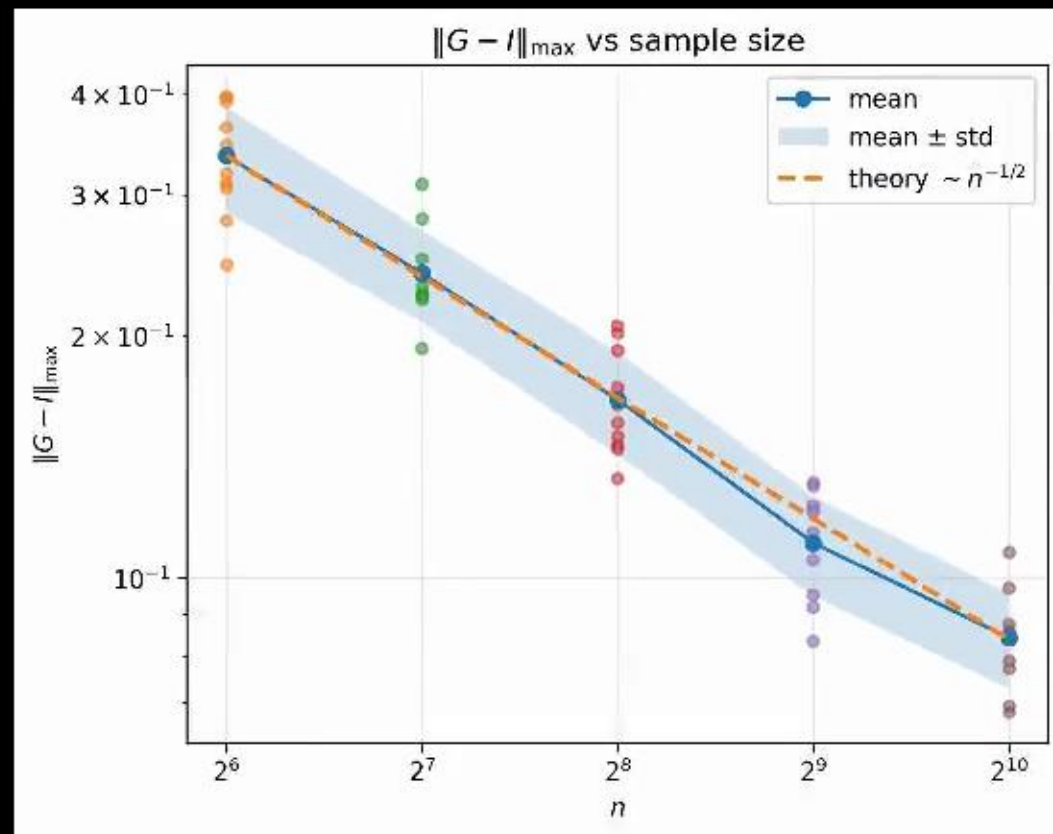
Empirical results: concentration with sample size



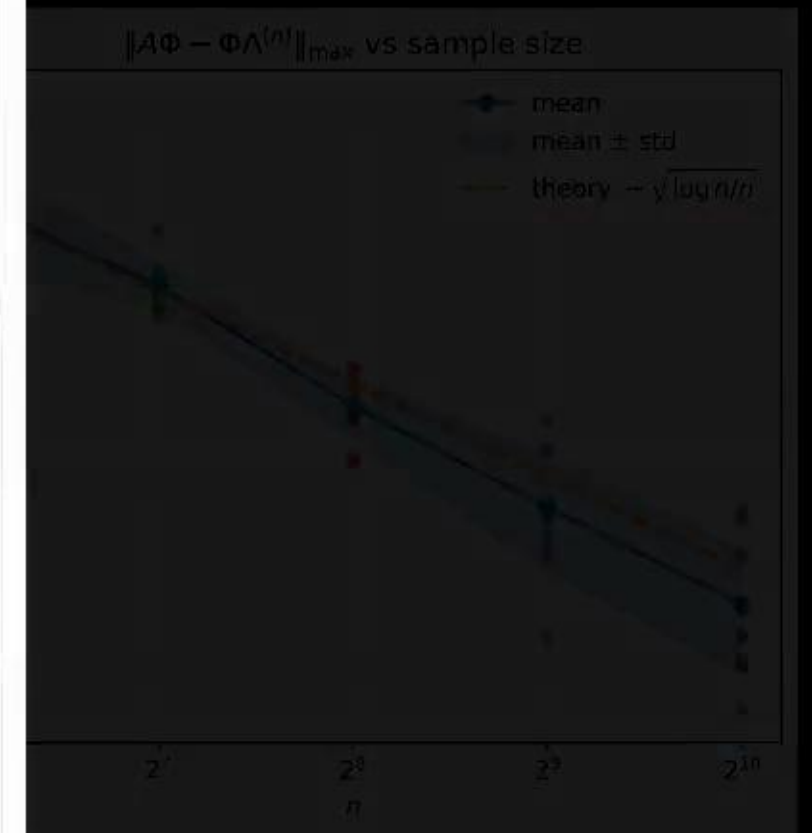
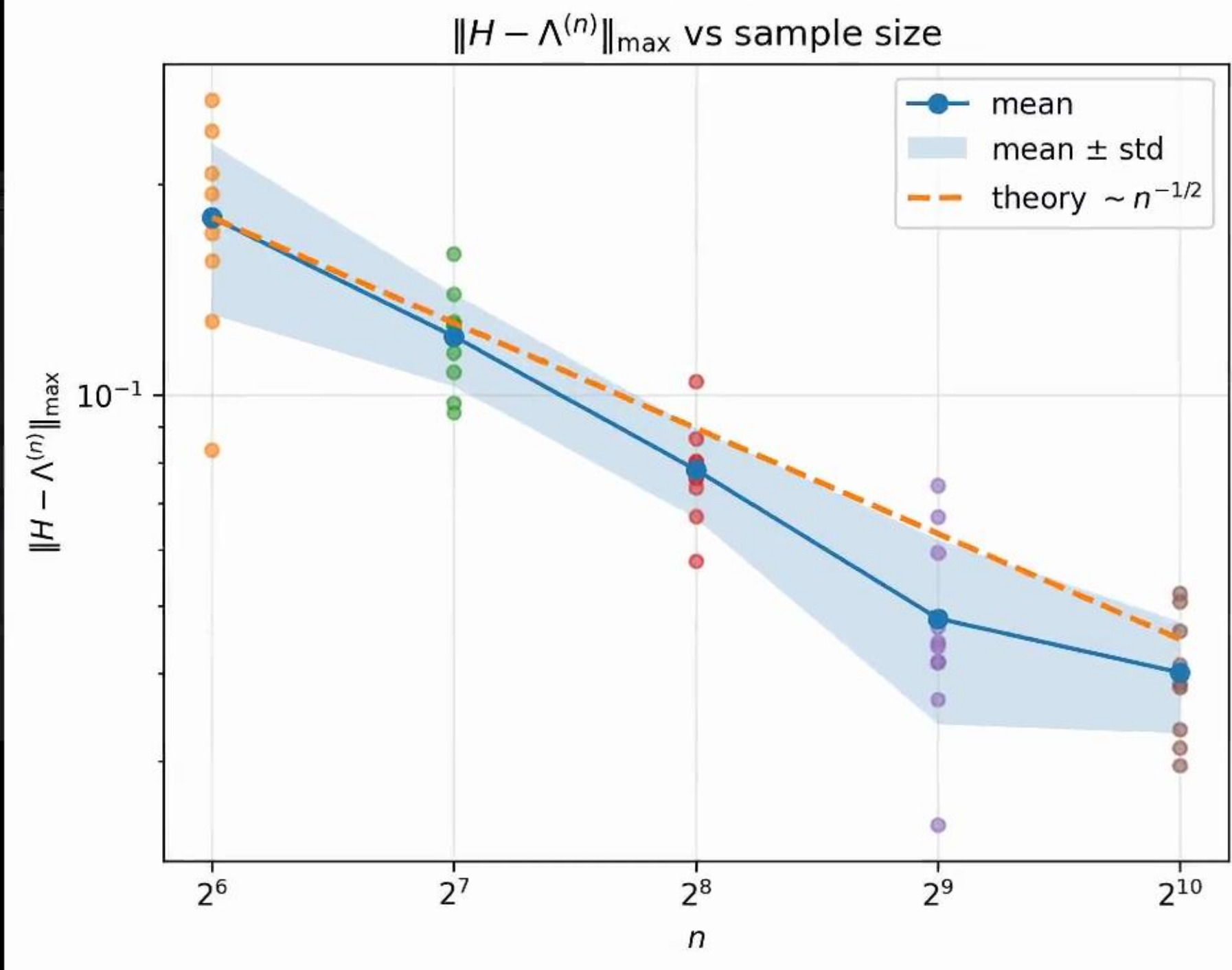
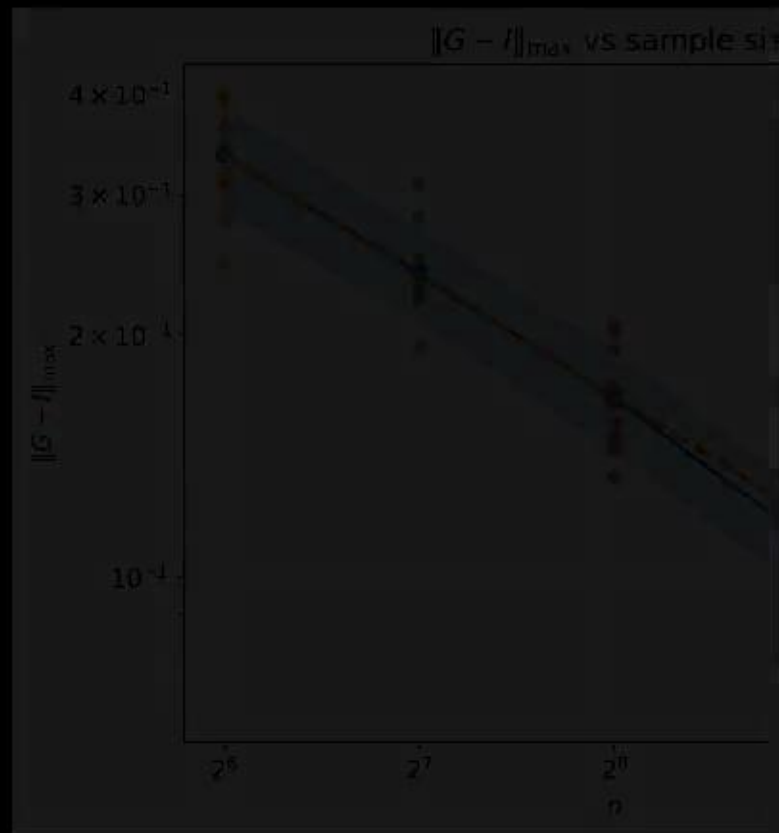
Empirical results: concentration with sample size



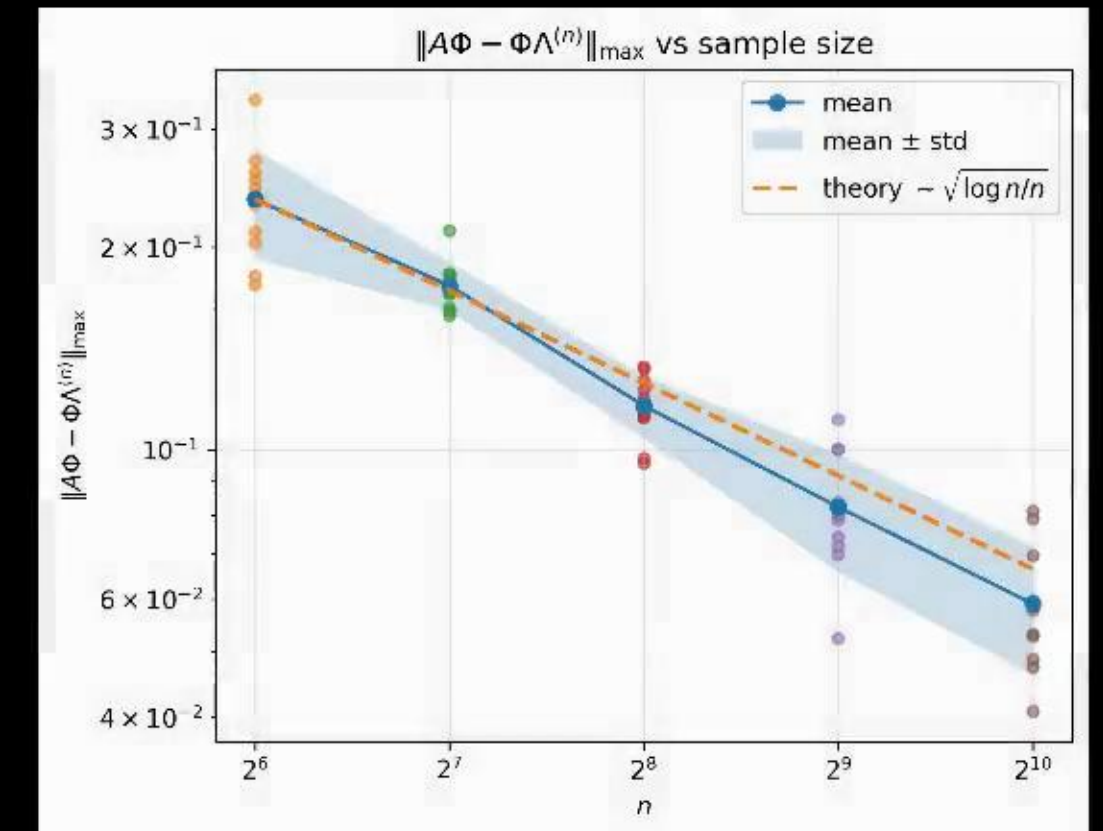
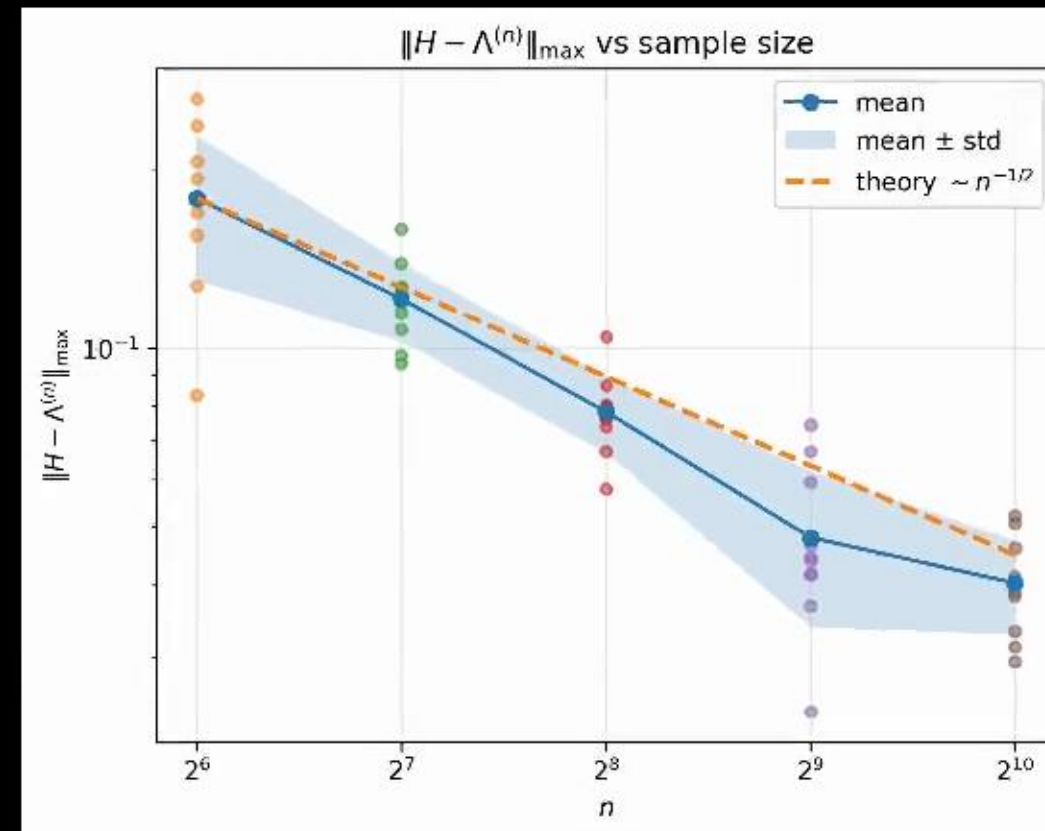
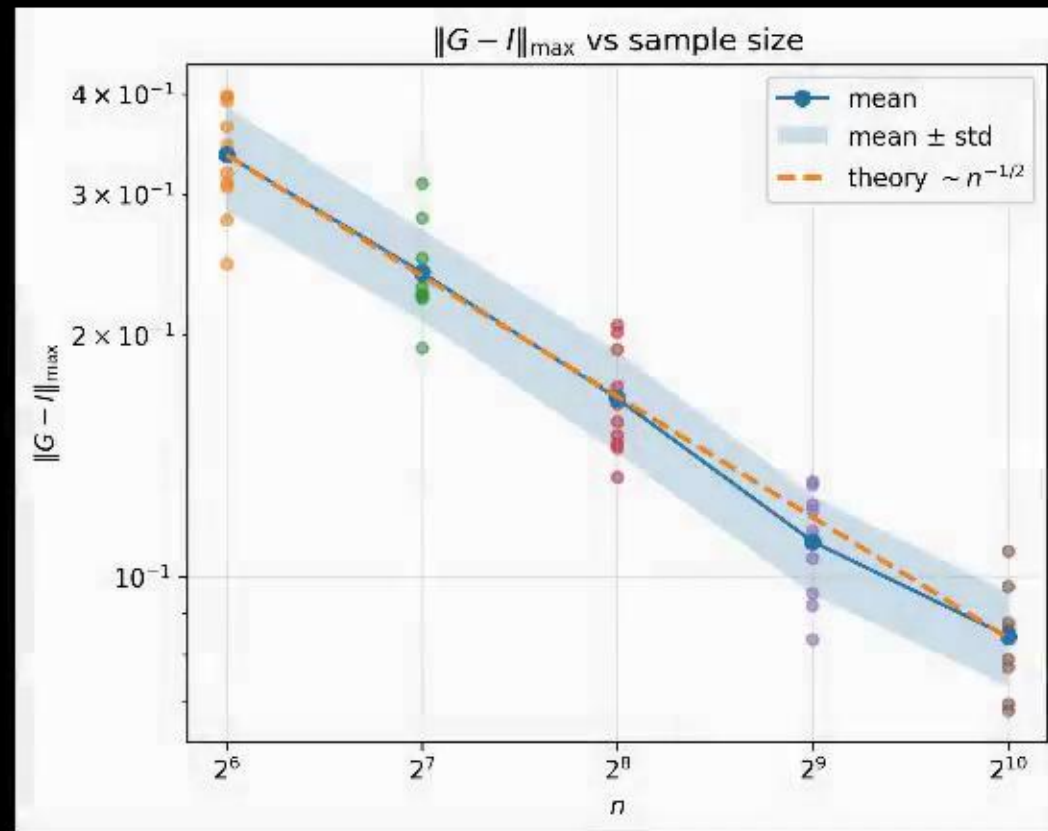
Empirical results: concentration with sample size



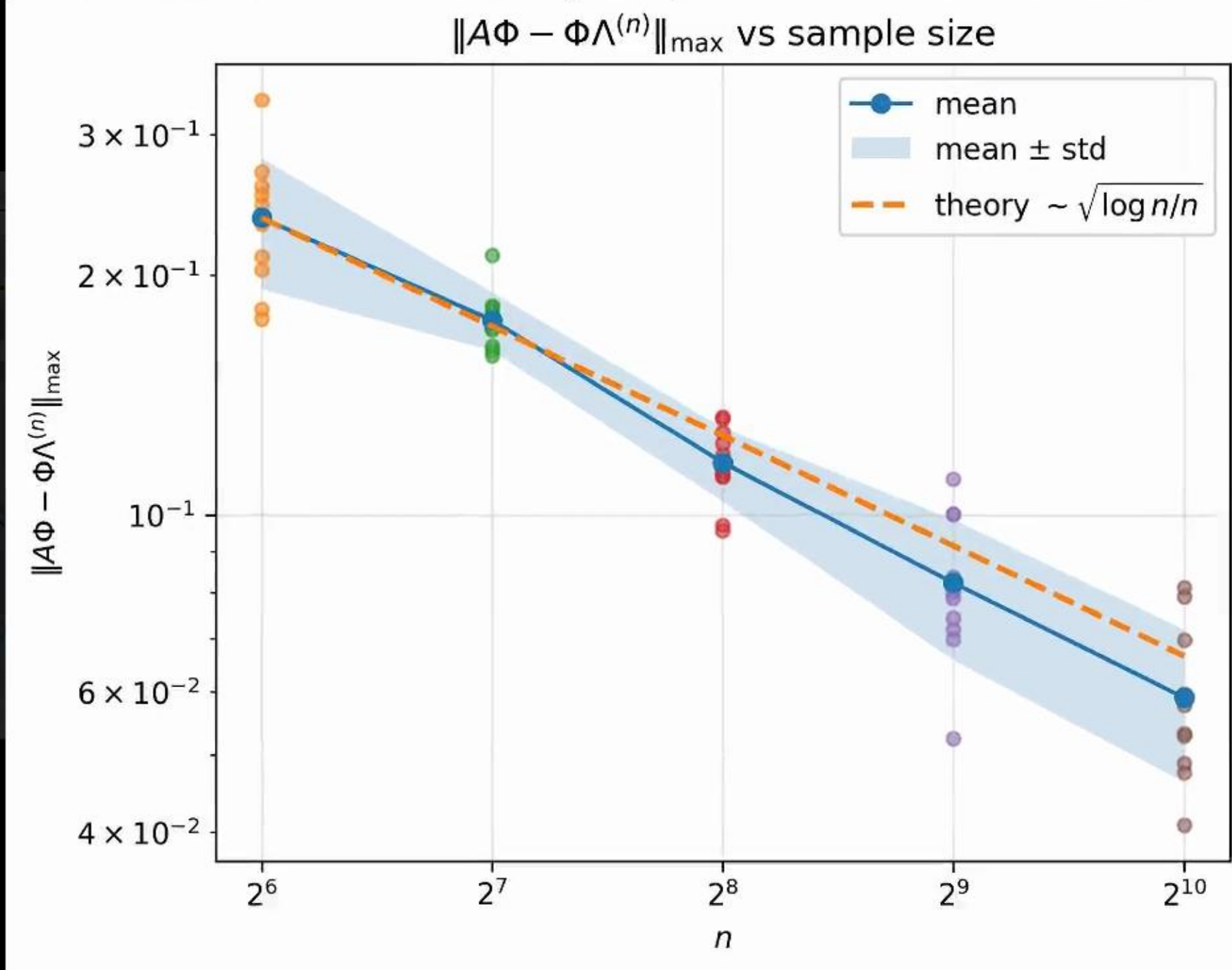
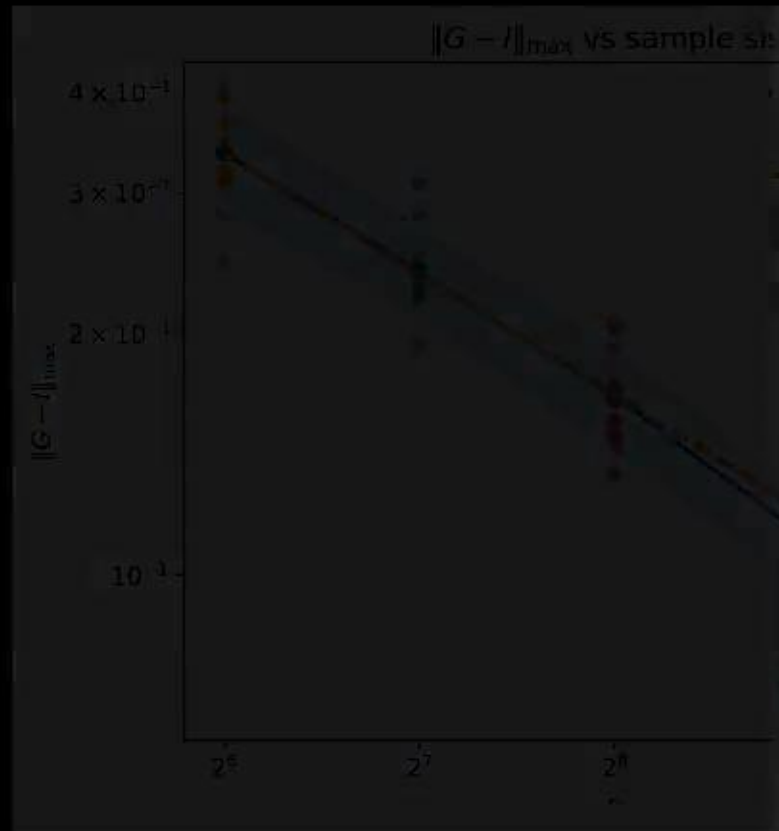
Empirical results: concentration with sample size



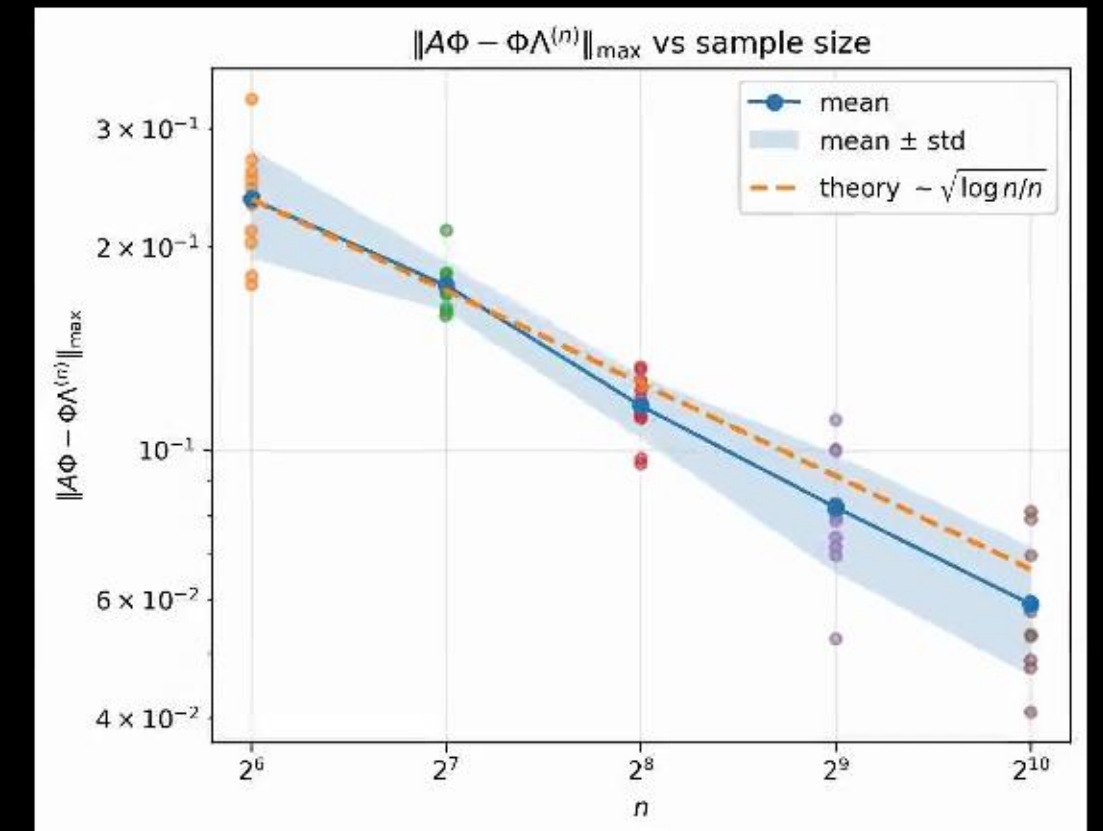
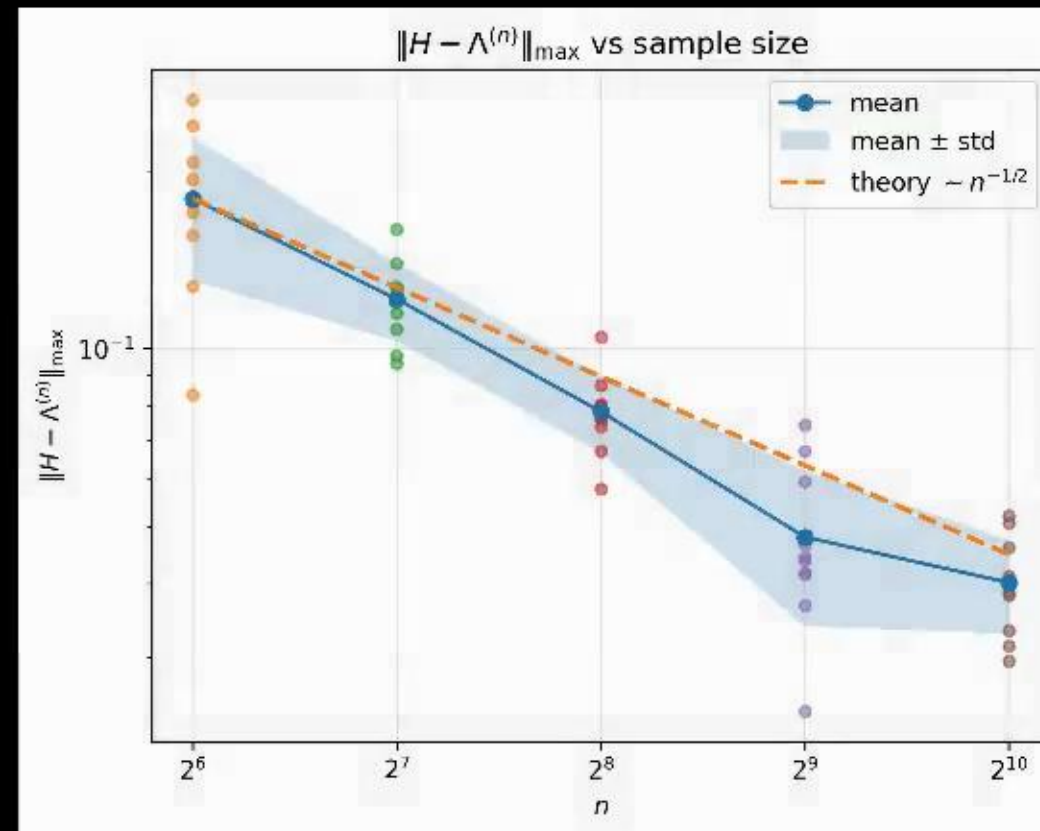
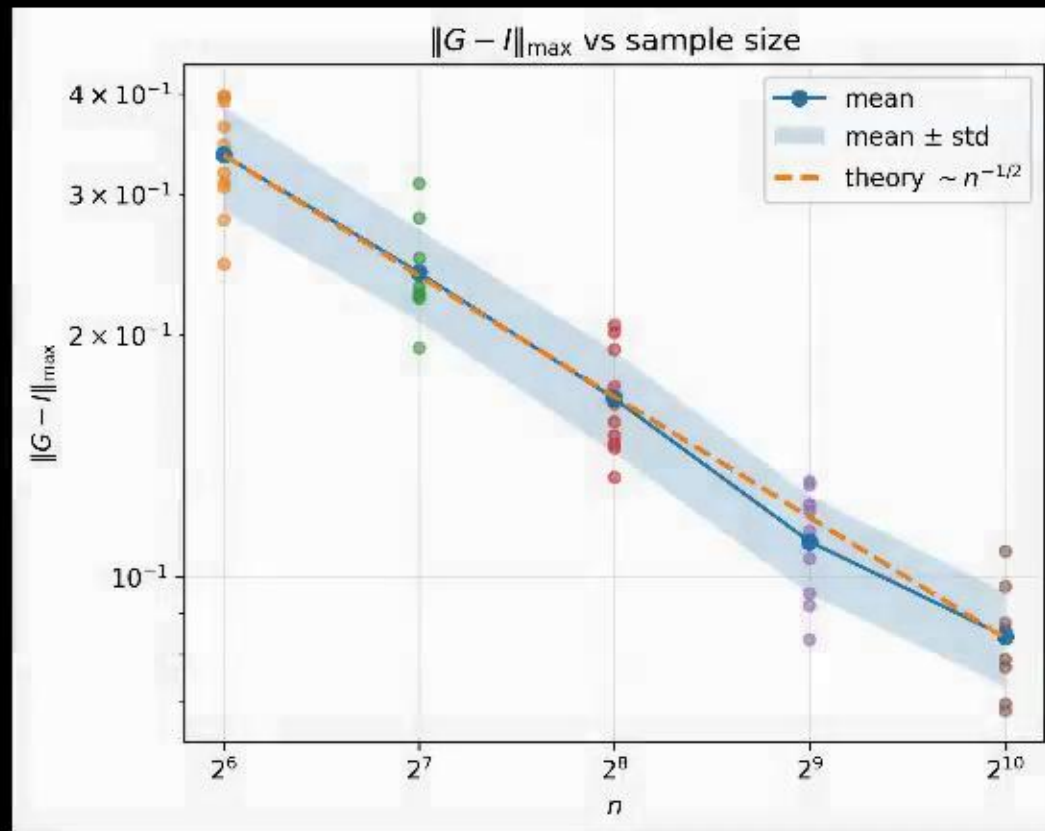
Empirical results: concentration with sample size



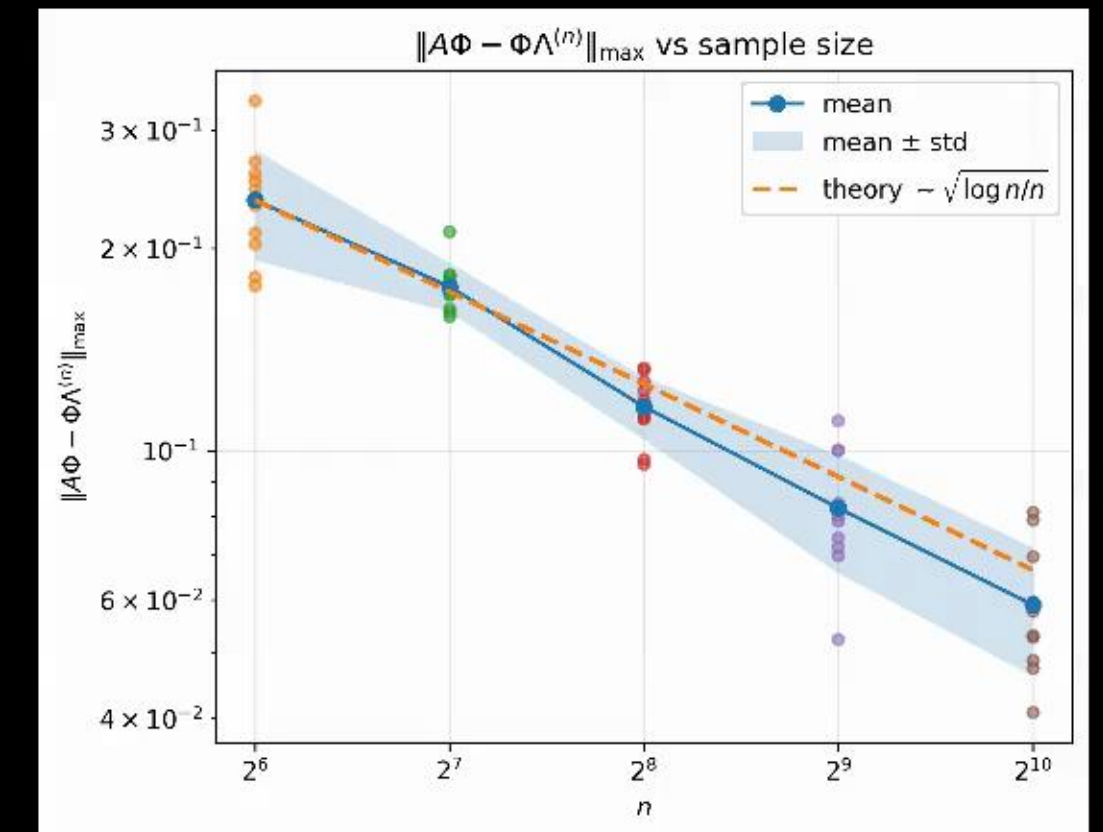
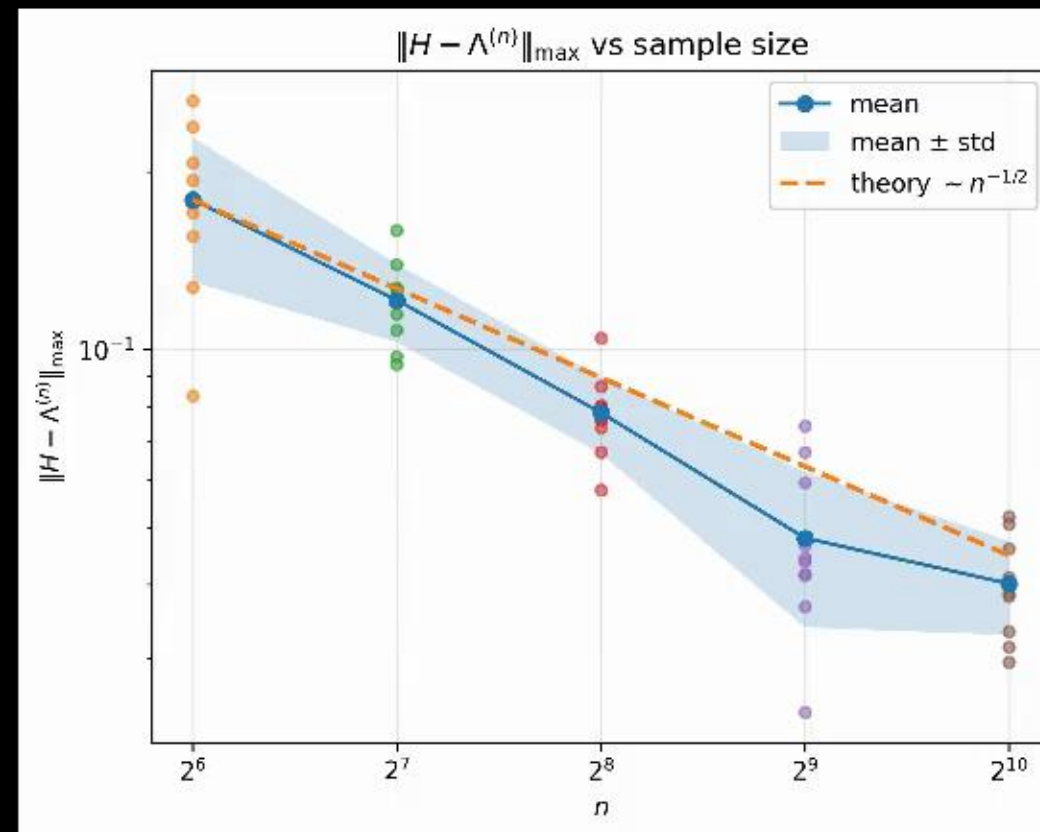
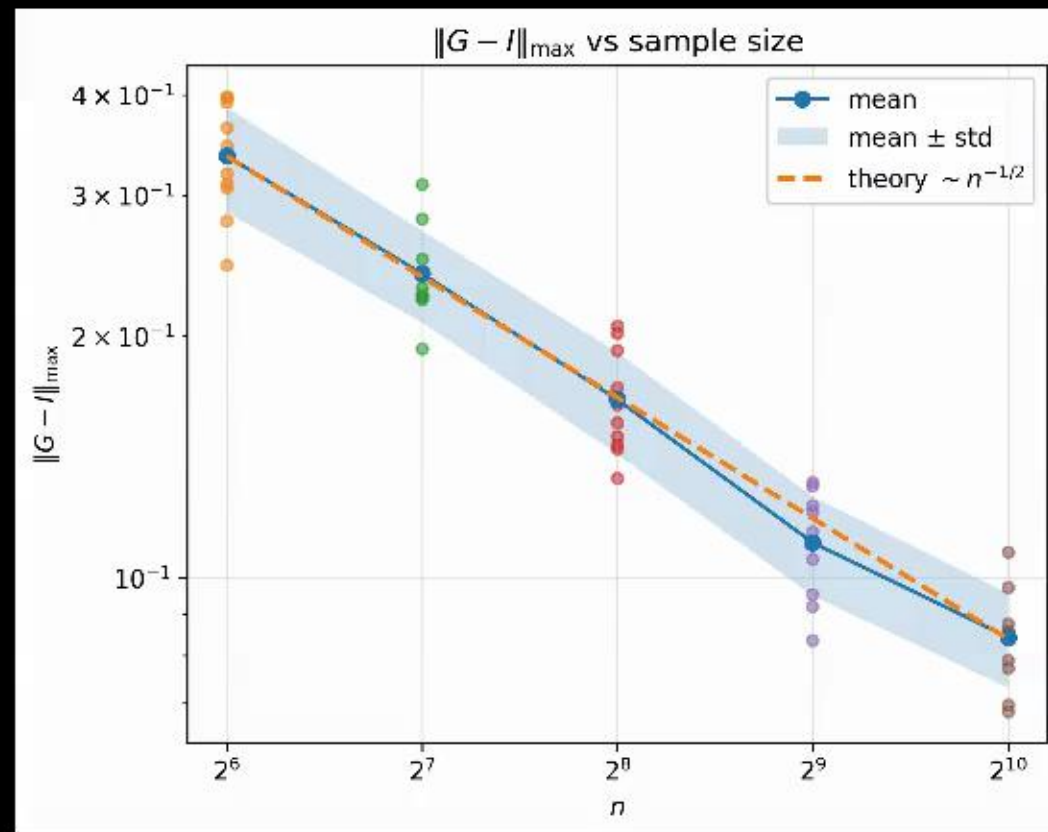
Empirical results: concentration with sample size



Empirical results: concentration with sample size



Empirical results: concentration with sample size



Finite sampling: low modes behave like the continuum once n is big enough.

Conclusion and next steps

Conclusion and next steps

Takeaway: for low modes, finite sampling preserves the continuum spectral picture.

Conclusion and next steps

Takeaway: for low modes, finite sampling preserves the continuum spectral picture.

- Tighten the concentration bounds
(using sharper tools than crude union bounds).

Conclusion and next steps

Takeaway: for low modes, finite sampling preserves the continuum spectral picture.

- Tighten the **concentration bounds**
(using **sharper tools** than crude union bounds).
- **Precondition** learning across modes:
rescale steps so all **eigendirections** converge similarly.

Conclusion and next steps

Takeaway: for low modes, finite sampling preserves the continuum spectral picture.

- Tighten the **concentration bounds**
(using **sharper tools** than crude union bounds).
- **Precondition** learning across modes:
rescale steps so all **eigendirections** converge similarly.
- **Finite width**:
Gaussian concentration, **Chernoff**-style bounds).

Conclusion and next steps

Takeaway: for low modes, finite sampling preserves the continuum spectral picture.

- Tighten the **concentration bounds**
(using **sharper tools** than crude union bounds).
- **Precondition** learning across modes:
rescale steps so all **eigendirections** converge similarly.
- **Finite width**:
Gaussian concentration, **Chernoff**-style bounds).

Thanks!